



Shadhujan Jeyachandran

2433192

**FocusBoost: Web Application for Improve
Children Engagement and Cognitive**

B.Sc. (Hons). Computer Science
Undergraduate Thesis Report
Department of Computer Science and Technology
University of Bedfordshire

Dr. Vibhavi Artigala

Academic Year 2025

ABSTRACT

FocusBoost is a web-based system described in this thesis that includes a real-time facial expression analysis and adaptive interventions to help children aged 8-10 years attend and concentrate on their studies without adult supervision. The system has to tackle the issue of keeping home-based learning kids engaged while at the same time complying strictly with child data protection regulations. The system architecture is a React TypeScript frontend and FastAPI Python backend, both of which are integrated with Firebase Firestore for data persistence. FocusBoost gets a webcam image every 30 seconds, turning it into base64 data for sending to two TensorFlow models: an emotion recognition model (48×48×3 RGB input) and a learning state detection model (224×224×3 RGB input). The system calculates attention scores through a confidence-weighted algorithm that merges the two sets of predictions as base scores that lie between 0.3 (boredom) and 0.9 (engagement). If the attention scores are lower than the set thresholds or if the learning states that indicate confusion or frustration have a confidence level of more than 70%, then the system will provide adaptive interventions via Google's Gemini API. Contextually relevant multiple-choice quizzes that suit the child's age, subject, and current emotional state will be created by Gemini thus helping the child to learn better. A sophisticated attention management system implements cooldown periods and dynamic sampling intervals to prevent intervention fatigue while maintaining engagement support. The technical implementation of the described work is complete and all the desired features are present in the implementation. Successful session management, good parental dashboards giving session analytics and attention trend visualization, and robust error handling at all steps of the analysis pipeline are some of the technical accomplishments described. The privacy-by-design principles, which are the most important ones, are greatly respected. Only the derived behavioral metrics are stored in Firebase while the processing of raw images is done in memory during the inference cycle. Testing validation shows that the system is practically viable with regular consumer hardware, a full study session workflow was carried out successfully and the correct intervention based on the attention state detected was triggered. The study offers a working model for the use of moral AI in the context of the classroom and proves how automation of education can add value to the level of education without violating the child's privacy and developmental level. It is the first step in a research trajectory that will eventually develop safer schooling technologies and adaptive learning systems with the focus on the child.

ACKNOWLEDGEMENT

I want to thank from the very bottom of my heart anyone that in any way has contributed to the success of the FocusBoost project.

My friends and family are the first on my appreciation list. I cannot thank you enough for being with me all those long nights during coding and testing. Not only did you listen to my ideas through, you also helped me keep the system going by testing it, and, more than that, you were the ones who helped me keep my spirit up when situations were tough. Thanks for your kids to take part in user testing sessions and being so gentle.

I want to thank all the parents who participated in the market research survey from the bottom of my heart. Their most truthful comments about fear of technology that might spy on kids directly affected the system design and security features in the system. I was very touched by and encouraged by your readiness to share the difficulties of your children's learning with me, and I thank you for that.

In addition, I want to acknowledge the contribution of Dr. Gayana Fernando in the outstanding lectures of the undergraduate project management course, which greatly helped in FocusBoost's structured development methodology. Furthermore, I would also like to thank my supervisor, Mr. Vibhavi Artigala. I deeply appreciated his guidance, constructive criticism, and expertise in AI and Machine Learning provided for FocusBoost . His patience and understanding, His constant encouragement to think creatively, helped me develop a truly beneficial tool.

The reason why FocusBoost is still around is because of the entire community which has never stopped from showing their support, giving advice, and providing encouragement. Through this project I have awesomely been able to experience, grow, and give back to the educational technology field.

DEDICATION

FocusBoost is the perfect companion for the kids that are really difficult to concentrate when they have to study on their own. To the children that after only ten minutes of maths get bored, who become so confused that they don't really know how to ask for help, and who completely lose their grip when the work becomes too hard.

This was a system made for you to have support and not to be judged. Learning should be encouraging and enjoyable, not frustrating. I really wish FocusBoost to be your helpful friend during your study time, providing you with quizzes when you are bored and giving you up when you succeed.

I hope this initiative will help build learning environments where every child feels supported and is confident in learning at their own pace.

LIST OF ABBREVIATIONS

AI : Artificial Intelligence

ML : Machine Learning

NLP : Natural Language Processing

COPPA: Children's Online Privacy Protection Act

GDPR : General Data Protection Regulation

UI : User Interface

UX : User Experience

DB : Database

JS : JavaScript

TS : TypeScript

CSS : Cascading Style Sheets

API : Application Programming Interface

GPT : Generative Pre-training Transformer

CRUD : Create, Read, Update, Delete

TBT : Total Blocking Time

CPU : Central Processing Unit

SI : Speed Index

GPU : Graphics Processing Unit

UAT : User Acceptance Testing

NPM : Node Package Manager

Table of Contents

ABSTRACT.....	i
ACKNOWLEDGEMENT	ii
DEDICATION.....	iii
LIST OF ABBREVIATIONS.....	iv
Table of Contents.....	v
Table of Figures	ix
Table of Tables	x
Table of Listing.....	x
1. CHAPTER ONE: INTRODUCTION.....	1
1.1. Project Background and Motivation	1
1.2. Problem Statement.....	1
1.3. Aim and Objectives.....	2
1.3.1 Aim	2
1.3.2 Objectives	2
1.4. Scope and Assumptions	3
1.4.1. Research Questions	3
1.4.2. Contributions.....	4
1.5. Structure of Report.....	4
2. CHAPTER TWO: LITERATURE REVIEW	6
2.1. Literature Review.....	6
2.1.1. Introduction.....	6
2.1.2. Cognitive Attention (8-10 years old)	7
2.1.3. Affective Computing in Education	8
2.1.4. Facial Expression Recognition in Education	9
2.1.5. Attention Tracking Technologies	10
2.1.6. Privacy/Ethics	10
2.1.7. Existing Systems Analysis.....	11
2.1.8. Research Gap Identification.....	13
3. CHAPTER THREE: METHODOLOGY & REQUIREMENTS ANALYSIS	15
3.1. Development Methodology Selection.....	15
3.1.1. Overview	15
3.1.2. Methodology Selection Rationale.....	15

3.1.2.1.	Project Complexity Analysis	15
3.1.2.2.	Alternative Methodologies Considered	16
3.1.3.	Project Development Timeline	16
3.1.4.	Iterative Development Approach and Benefits.....	18
3.1.5.	Project Management and Quality Assurance.....	18
3.2.	Stakeholder Analysis & User Personas.....	19
3.2.1.	Child User Persona (Primary User)	19
3.2.2.	Parent User Persona (Secondary User)	20
3.3.	Functional Requirements Specification	21
3.3.1.	Core System Functions	21
3.3.2.	Requirements Priority Classification	22
3.4.	Non-Functional Requirements Specification.....	23
3.4.1.	Performance Requirements.....	23
3.4.2.	Reliability & Availability	23
3.4.3.	Compatibility & Accessibility	23
3.4.4.	Security & Privacy	23
3.4.5.	Maintainability & Scalability.....	24
3.5.	Use Case Analysis.....	24
3.5.1.	Primary Use Case Scenarios	26
3.5.2.	Edge Case & Error Scenarios	28
3.6.	Success Criteria & Evaluation Metrics	28
3.6.1.	Technical Performance Metrics	28
3.6.2.	Functional Effectiveness Metrics.....	29
3.6.3.	Privacy & Compliance Metrics.....	29
3.7.	Chapter 3 Summary	29
4.	CHAPTER FOUR: SYSTEM DESIGN & IMPLEMENTATION	30
4.1.	System Architecture & Technical Design.....	30
4.1.1.	High-Level Architecture Overview	30
4.1.2.	Frontend Application Architecture	32
4.1.3.	Backend API Layer Implementation	33
4.2.	Machine Learning Pipeline Implementation.....	36
4.2.1.	ML Processing Architecture	36
4.2.2.	Inference Processing & Results Generation	39

4.2.3.	Quiz Generation Integration	41
4.3.	Database Design & Implementation	42
4.3.1.	Data Model Architecture.....	42
4.3.2.	Privacy-by-Design Implementation	45
4.4.	Frontend Implementation Details	46
4.4.1.	Component Architecture & State Management	46
4.4.2.	Camera Integration & Frame Processing	47
4.5.	Backend Service Implementation	49
4.5.1.	API Endpoint Architecture	49
4.5.2.	Error Handling & Resilience	49
4.6.	Integration & Communication Layer	50
4.6.1.	API Communication Protocol.....	50
5.	CHAPTER FIVE: TESTING AND EVALUATION	51
5.1.	Testing Strategy and Methodology	51
5.1.1.	Testing Framework Overview	51
5.1.2.	Test Environment Specification.....	51
5.2.	Unit Testing	53
5.2.1.	Frontend Component Testing	54
5.2.2.	StudySession Component Test Cases	58
5.3.	Integration Testing	65
5.4.	Perfomance Testing	65
5.5.	Usability Testing.....	67
5.6.	Privacy and Security Testing	67
5.6.1.	Data Protection Validation.....	67
5.7.	Test Cases	69
5.7.1.	Sign In Focusboost with Valid credentials	69
5.7.2.	Sign In Focusboost with Invalid credentials	70
5.7.3.	Cookie Handling on Login.....	71
5.7.4.	Successful User Logout	72
5.7.5.	New User Registration with Valid Credentials.....	73
5.7.6.	New User Registration with Existing Credentials	74
5.7.7.	New User Registration with Empty Credentials	75
5.7.8.	User Enter with valid Parent PIN.....	76

5.7.9.	User Enter with valid Parent PIN.....	77
5.7.10.	User Enter with invalid Parent PIN	78
5.7.11.	View Piechart in Parent Dashboard	79
5.7.12.	View Session Analytics in Parent Dashboard.....	80
5.7.13.	View Manage profile section in Parent Dashboard	81
5.7.14.	Add new child profile	82
5.7.15.	Edit child profile	83
5.7.16.	Delete child profile	84
6.	Chapter SIX: Results and Critical Evaluation	85
6.1.	Introduction.....	85
6.2.	System Functionality and Implementation Evidence	85
6.3.	Testing and Verification	86
6.4.	Critical Evaluation of Approach	87
6.5.	Comparison with Existing Literature and Systems.....	89
6.6.	Challenges and Limitations.....	90
6.7.	Achievements and Contributions	91
6.8.	Reflection on Researcher’s Role.....	91
6.9	Conclusion	92
7.	CHAPTER 7 - DISCUSSION	93
7.1.	What Worked	93
7.2.	What Didn’t / Deviations	93
7.3.	Comparison to Related Work.....	93
7.4.	Threats to Validity	94
7.5.	Ethical Reflection.....	94
7.6.	Chapter Conclusion.....	94
8.	CHAPTER EIGHT - CONCLUSION AND FUTURE WORK.....	95
8.1.	Summary of Contributions.....	95
8.2.	Objective-by-Objective Achievement	95
8.3.	Limitations	95
8.4.	Future Work	96
	References.....	i
	APPENDIX A.....	iv
	APPENDIX B	vi

Evidence Bank:	vi
FocusBoost Github:	vi
FocusBoost model training History:	vi
APPENDIX C: FocusBoost POSTER.....	vii

Table of Figures

Figure 3-1: FocusBoost Gantt Chart Planning Phase	16
Figure 3-2: FocusBoost Gantt Chart Research Phase	16
Figure 3-3: FocusBoost Gantt Chart Implementation Phase.	17
Figure 3-4: FocusBoost Gantt Chart Testing & Evaluation Phase	17
Figure 3-5: FocusBoost Use Case Diagram.....	25
Figure 4-1: System Architecture Diagram	30
Figure 4-2: System Context Diagram	30
Figure 4-3 Component Level Diagram	31
Figure 4-4: Sequence Diagram	32
Figure 4-5: React Logo.	32
Figure 4-6: Firebase Logo.....	33
Figure 4-7: FocusBoost Backend Swagger UI.....	35
Figure 4-8: ML Pipeline Dataflow.....	38
Figure 4-9: Firebase generated quiz collection	42
Figure 4-10: FocusBoost DB Collection – Children	43
Figure 4-11: FocusBoost DB Collection – interventions.....	43
Figure 4-12: FocusBoost DB Collection - quiz_results.....	43
Figure 4-13: FocusBoost DB Collection - session_analytics	44
Figure 4-14: FocusBoost DB Collection - session_data	44
Figure 4-15: FocusBoost DB Collection - study_sessions	44
Figure 4-16: Quiz Modal	47
Figure 4-17: Camera Permission Interface	49
Figure 4-18: Active Session - Network Activity	50
Figure 5-1: System Info.	51
Figure 5-2: Network Speed.....	52
Figure 5-3: Browser Version	52
Figure 5-4: Study Session Lighthouse performance testing.....	66
Figure 6-1: Console logs showing ML model inference results	87
Figure 6-2: DAiSEE Sample Prediction Test Set	87
Figure 6-3: DaiSEE training Progress.....	88
Figure 6-4: DAiSEE Model Final Training Results.....	88
Figure 6-5: System bottleneck diagram	91
Figure A-0-3: DAiSEE Model Architecture	v

Table of Tables

Table 4.1: REST API Endpoints Summary	34
Table 4.2: Attention Scoring Logic	40
Table 4.3: Firestore Database Schema	42
Table 4.4: API Response Models	49
Table 5.1: Frame Capture and Analysis	54
Table 5.2: Frame Capture and Analysis	55
Table 5.3: Frame Capture and Analysis	56
Table 5.4: ML Analysis Response Processing	57
Table 5.5: StudySession Component Initialization	58
Table 5.6: Study Session Start Flow	59
Table 5.7: Quiz Modal Trigger	60
Table 5.8: Quiz Answer Submission	61
Table 5.9: Quiz Answer Submission	62
Table 5.10: Study Session End Handling	63
Table 5.11: Study Session End Handling by Parents	64
Table 5.12: Sign In Focusboost with Valid credentials	69
Table 5.13: Sign In Focusboost with Invalid credentials	70
Table 5.14: Cookie Handling on Login	71
Table 5.15: User logout successful	72
Table 5.16: New User Registration with Valid Credentials	73
Table 5.17: New User Registration with existing Credentials	74
Table 5.18: New User Registration with Empty Credentials	75
Table 5.19: User Enter Parent Dashboard with valid Parent PIN	76
Table 5.20: User Enter Parent Dashboard with valid Parent PIN	77
Table 5.21: User Enter Parent Dashboard with invalid Parent PIN	78
Table 5.22: View Piechart in Parent Dashboard	79
Table 5.23: View Session Analytics in Parent Dashboard	80
Table 5.24: View Manage profile section in Parent Dashboard	81
Table 5.25: Add new child profile	82
Table 5.26: Edit child profile	83
Table 5.27: Delete child profile	84
Table 6.2.6.1: Implemented Features vs. Original Requirements	85
Table 6.2.6.2: Comparison of FocusBoost vs. Existing Systems	89

Table of Listing

Listing 4.1-1: main.py - ML Analysis Endpoint	34
Listing 4.2-1: preprocess_image_for_emotion()	36
Listing 4.2-2: preprocess_image_for_learning()	37
Listing 4.2-3: ml_analyzer.py - Inference Processing - detect_emotion()	39
Listing 4.2-4: ml_analyzer.py - Inference Processing - detect_learning_state()	40

Listing 4.2-5: ml_analyzer.py - Inference Processing - calculate_attention_score().....	40
Listing 4.2-6: Gemini Api Integration	41
Listing 4.2-7: Gemini generate_quiz().....	41
Listing 4.3-1: Decode base64 image to memory	45
Listing 4.3-2: Database storing metrics logic	46
Listing 4.4-1: getUserMedia Permission Setup.....	48
Listing 4.4-2: frame capture logic.....	48
Listing 5.1-1: Python Environment Check Test	53
Listing 5.1-2: Firebase Project Console.....	53
Listing 5.6-1: Database Audit Test for Raw Image	68

1. CHAPTER ONE: INTRODUCTION

1.1. Project Background and Motivation

The emergence of digital learning environments has transformed the mechanisms of pedagogical delivery. however, children continue to remain challenged in terms of making them stay on track when studying themselves.

It has been found empirically that eight to ten-year-old pupils are able to sustain focused attention between fifteen to thirty minutes of a well-structured task, and the length of attention is also affected by other contextual factors like ambient environment and intrinsic motivation (Ruff and Rothbart, 1996). This makes developmental limitations on attention span problematic in the context of learning in a home based setting where adult supervision might be less alert.

The immersion into remote education during the COVID-19 pandemic revealed critical gaps in the technological affordances of self-regulation support for young learners (UNESCO, 2021). In this way, many existing attention support solutions primarily provide passive content or rely on regulatory surveillance that impairs privacy, suggesting the pressing need to develop attention support that is ethical and appropriate for children. Expressive facial analysis has potential for privacy-preserving attention monitoring, but has not been considered in educational contexts, especially the developmental appropriateness and ethical ramifications (Sharma et al., 2021).

This research investigates these questions by developing FocusBoost, a privacy-preserving solution focused on real-time facial analysis to provide adaptive learning support for children while adhering to strict child protection and privacy regulations.

1.2. Problem Statement

Children between 8 and 10 years old face great difficulty in maintaining focus during independent study sessions. This difficulty not only impedes the ability to learn new material but also results in frustration. The existing technology products that attempt to address issues related to attention either do not concern themselves with the specific developmental attention-related issues of children or address them with invasive and ethically questionable monitoring methods. Commercially available attention tracking products either rely on specialized

hardware, collect biometric data, or do not have age-appropriate attention remediation techniques (BrainCo, 2022; Focus@Will, 2023). Though technically feasible, prototype research systems are not suitable for real-world application as they are too complex, too expensive, or violate privacy (Dowell et al., 2020). Furthermore, the tracking of attention systems is mainly focused on adults and suffer a drop in accuracy of between 15-25% when applied on children as children tend to show differences in attention and facial expressions because of development (Sharma et al., 2021). The broad lack of concern for privacy.

1.3. Aim and Objectives

1.3.1 Aim

To build a privacy web-based system that monitors children's attention during study sessions through facial expression analysis and provides adaptive interventions to support engagement and improve cognitive.

1.3.2 Objectives

The Focusboost project objectives are:

- **Developed Real-time Attention Analytics:** build a system to process facial expressions using dynamic intervals based on attention states, then obtain attention scores using machine learning techniques and computer vision.
- **Implemented Adaptive Intervention Engine:** Design a rule-based system that triggers contextually appropriate interventions when attention thresholds are breached, building cooldown mechanisms to prevent disturbance.
- **Generateed Age-Appropriate Content:** Integrate large language model technology to create engaging, curriculum-aligned quiz content tailored to children's attention states and developmental level.
- **Trained Machine Learning Algorithms:** Utilise Machine Learning algorithms to analyse user's emotion and learning state patterns based on their facial emotion.
- **Provided Parent-Facing Insights:** Develop a dashboard providing visualisations of attention patterns, session summaries, and progress metrics without exposing sensitive biometric data
- **Ensured Privacy by Design Implementation:** Maintain compliance with COPPA and GDPR through strict data minimisation, transient processing of biometric data, and storage of only derived metrics.

1.4. Scope and Assumptions

The Project Scope was intended to include the entire end-to-end system - which largely consisted of a frontend React application, FastAPI back-end services, a machine learning inference pipeline, a Firebase data persistence functionality, and a Gemini integration that drove real-time feedback functionality in the frontend React application. The intention of the system was to enable monitoring of the child in a home environment, with traditional webcam-enabled devices.

The main assumptions made by this system included that the users would have a stable internet connection; parents will provide adequate lighting conditions, voluntary, and parental consent. The scope of the system also specifically excluded any natural language processing technology geared towards understanding speech; a teacher portal; running models performed on device inference; or large-scale models retraining with study participant-specific data. Overall, the system operated, under the assumption that facial expressions would be visible during study sessions and the child, screened for ability to maintain a position (stay largely within the relevant camera field of view).

1.4.1. Research Questions

This research addressed the following primary questions:

1. How can facial expression recognition be effectively adapted to monitor attention states in 8-10-year-old children while maintaining privacy compliance?
2. What intervention mechanisms most effectively support attention re-engagement without disrupting learning flow?
3. How can large language models be safely leveraged to generate age-appropriate educational content responsive to real-time attention states?
4. What technical architecture enables reliable real-time attention monitoring using consumer-grade hardware while minimising privacy risks?
5. How do children and parents perceive the acceptability and usefulness of attention monitoring system in the home learning environment?

1.4.2. Contributions

This study makes a number of key contributions to the fields of educational technology and ethical AI.

1. Privacy-preserving architecture: A new design that processes biometric data in the moment and solely holds derived metrics developing a model for ethical child monitoring technology.
2. Child-optimised attention scoring: An algorithm that collates predicted emotional and learning states with confidence weighting which can account for developmental differences in facial expression.
3. Real-time adaptive intervention framework: A "in-the-wild" iteration of attention-flagging content generation using large language models showing the practical possibilities for educational use.
4. Empirical validation: Performance data as evidence of reliable function within agreed latency parameters (median = 850ms) and successful content generation triggered by attention intervention (74% appropriateness).
5. Ethical implementation guidelines: Providing a fully consideration of child data protection, balancing educational purposes of processing with privacy protecting considerations, providing overtures for best practice in the development of educational AI.
6. Adaptive sampling algorithm: A shifting interval monitor that dynamically adjusts the frequency of processing based on detected learning state, its goal is to limit processing during high engagement periods while using a fast sampling period if learner engagement declines.

1.5. Structure of Report

Chapter One - Introduction: The first chapter is the introduction, which gives a brief but informative overview of the FocusBoost project, its relevance, its purpose, and the general research question. It has offered the hypothesis that a privacy-conscious attention-monitoring system that is a combination of facial expression analysis and machine-learning can produce better engagement of children in the learning process by conducting adaptive interventions.

Chapter Two - Literature Review: The second chapter provides comprehensive synthesis of the available literature that is relevant to the study of cognitive attention in children, affective computing in education, computer-vision recognitions in engagement detection, and the ethical and privacy concerns of educational technology. The review establishes some of the research gaps that prevail and justifies the manner in which FocusBoost intends to fill the gaps.

Chapter Three - Methodology and Requirements: This chapter presents both the iterative development approach, and comprehensive requirements analysis that was done for FocusBoost. It contains stakeholders analysis, functional and non-functional requirements specification, and success criteria definition which focused the system development process.

Chapter Four - System Design and Implementation: This is where the technical design and implementation of FocusBoost is described; the React frontend, FastAPI backend, machine learning pipeline, Firebase database schema, and integration with real-time facial analysis and adaptive quiz generation system.

Chapter Five - Testing: This chapter presents the overall testing strategy used in the development of FocusBoost, including unit testing, integration testing, performance testing, and user acceptance testing. This chapter documents the results of the testing and describes the system performance against specification requirements and success criteria defined.

Chapter Six - Results and Critical Evaluation: This chapter summarizes the empirical results of the FocusBoost system, in terms of quantitative measures of performance, accuracy, and an analytical discussion of how the system is working to promote attention in children during the moments of study.

Chapter Seven - Discussion: This chapter critically analyzes the project findings, compares the results of achievement with the expected results and explains the limitations, ethics and possible threats to validity. The results are put into perspective within the framework of previous research and the implications of the results to the design of educational technology are addressed.

Chapter Eight - Conclusion and Future work: The last chapter lists the main contributions of the study, evaluates whether the research was able to achieve its goals, outlines the recognized limitations, and suggests strategic paths of research to take in the future to help the development of privacy-sensitive educational AI systems.

2. CHAPTER TWO: LITERATURE REVIEW

2.1. Literature Review

2.1.1. Introduction

This chapter tells how that helps the development of FocusBoost by reviewing the academic literature and existing technologies. The review is organised to review five important area that contribute to the research: development of cognitive attention in children aged 8-10 years, applications of affective computing for educational purposes, computer vision technologies for detecting engagement, adaptive learning and content personalisation, and privacy and ethics in AI systems focusing on children.

The literature review begins with attention development theoretical undertones for the target age group, and investigates how cognitive limitations and development affects focus during learning opportunities. This knowledge will allow us to develop acceptable monitoring systems using developmental appropriate attention patterns, rather than using our own patterns based on adults.

This chapter then reviews the present state of affective computing in education, the datasets, models and methods used to construct facial expression detection systems, and the current limitations of these approaches when applied to children. We will examine the ways in which their choices were influenced by factors like the accuracy rate and bias against a child population.

In reviewing, the chapter examines metacomputer vision technologies that imply some form of attention or engagement detection to better understand the technological pathways used, their performance capabilities, and the constraints associated with social, normative self-report monitoring on real-time systems. The analysis represents both unimodal and multimodal analysis and considers their useability as home-based learning systems with commercially available computing hardware.

Adaptive learning systems and frameworks for quiz personalisation were reviewed to inform best practices for responsive delivery of educational content. The review explores how attention states can be used as a basis for content adaptation while also considering how large language models can develop contextually applicable interventions.

Finally, the chapter takes a look at the important ethical and privacy considerations related to child data protection in educational technology. This includes reviewing regulatory responsibilities, technical approaches to privacy preservation, and the ethical ramifications of ongoing surveillance in children's learning regarding the use of technology.

By synthesizing these literature streams, we were able to outline important gaps in the literature and create both a theoretical and practice position for FocusBoost in providing a child-centred, attention support and privacy protective approach to learning in technology-supported environments.

2.1.2. Cognitive Attention (8-10 years old)

Children in age 8-10 show special attention patterns that affect how they learn. Many studies in psychology say kids at this stage usually keep focus around 15-30 minutes when they do structured tasks, but this time can change depending on interest, tiredness, or environment (Ruff & Rothbart, 1996).

Cognitive Load Theory (Sweller, 1988) explains this situation. Because their working memory is still limited, they get overloaded easily when teachers give too much information or material not in the right pace. Brain studies also show the prefrontal cortex, which controls executive attention, is not fully developed in this age, so children distract more often (Luna et al., 2015).

Because of this, they need outside support to keep attention, and this is the main reason for systems like FocusBoost. But not all children are the same. Some have ADHD (around 5-7% in this age group) and also social background makes a big difference in how long they stay focused (Danielson et al., 2018). So technology solutions must use flexible levels for each child, not only one standard for all.

Table 2.1.2: Developmental Attention Characteristics in 8-10 Year Olds

Age Group	Typical Focused Attention Span	Cognitive Load Capacity	Key Influencing Factors	ADHD Prevalence
8 years	15-20 minutes	3-4 chunks	Interest level, fatigue	5-7%
9 years	20-25 minutes	4-5 chunks	Task complexity, motivation	5-7%
10 years	25-30 minutes	5-6 chunks	Prior knowledge, environment	5-7%

Source: Ruff & Rothbart (1996); Luna et al. (2015); Danielson et al. (2018)

2.1.3. Affective Computing in Education

Affective computing is about making computers understand human emotions, and in education this is used to see how students feel during learning. Many systems use three common datasets: FER-2013, AffectNet, and DaiSEE.

FER-2013 (Goodfellow et al., 2013) has around 28,709 training images in seven emotion categories. It help a lot in building CNN models for facial expression recognition. But the problem is, the images are mostly posed in controlled settings, so it don't look like real classroom faces.

AffectNet (Minaee et al., 2021) is bigger, with more than one million images, and also include valence-arousal labels. This is good progress, but most of the dataset is adults, so it is not much useful for children emotions.

DaiSEE (Dhall et al., 2017) is made for engagement detection, with 9,068 videos showing different engagement levels. Still, it also don't have many child samples, which make a big gap.

The issue is clear: when models train only on adult faces, and then tested on children, the accuracy drop almost 15-25% (Sharma et al., 2021). This happen because children control their facial muscles different, and their expression is often more subtle. Another limitation is that most dataset use front-facing, well-lit images, but real classrooms are not like that. Students move, lighting change, and many times faces are not clear.

Because of these problems, some new systems like FocusBoost try to improve preprocessing and use confidence-based scoring to deal with uncertainty in child emotion recognition..

Table 2.1.3: Comparison of Affective Computing Datasets

Dataset	Size	Categories	Population	Accuracy with Children
FER-2013	28,709 images	7 emotions	Adults	60-65%
AffectNet	>1M images	11 emotions + valence-arousal	Adults	65-70%
DaiSEE	9,068 videos	4 engagement levels	Mixed (mostly adults)	70-75%

Source: Goodfellow et al. (2013); Minaee et al. (2021); Dhall et al. (2017)

2.1.4. Facial Expression Recognition in Education

Facial Expression Recognition (FER) is one of the main tools used in affective computing for schools. The pipeline is usually the same: first detect the face, then align, extract features, and finally classify the emotion (Li & Deng, 2020).

Earlier methods use hand-made features like Local Binary Patterns (LBP) or Histogram of Oriented Gradients (HOG). They work okay in controlled labs with 60-70% accuracy (Zhang et al., 2019), but when move to real classrooms, they not perform so well.

Deep learning, especially CNN models like VGGFace or ResNet, give better results, reaching 85-90% accuracy in benchmarks (Parkhi et al., 2015). However, in education there are still challenges. For example, AutoTutor (D'Mello et al., 2008) show that emotion-adaptive tutoring can improve learning gains about 10-15%. But again, these are lab conditions, not messy classroom realities.

Working with children is even harder. Their expressions are softer, shorter, and facial structure is different from adults (LoBue & Thrasher, 2014). Research show that normal FER models trained on adults only get 65-75% accuracy on children, while adults reach 80-90% (Sharma et al., 2021).

Another concern is privacy. Many FER systems keep facial images or embeddings, which is risky, especially with minors (Shneiderman, 2020). Because of this, FocusBoost propose transient processing, meaning the system don't store faces permanently, and also use preprocessing that is more friendly to child data. Still, accuracy problem is not fully solved.

Table 2.1.4: Facial Expression Recognition Pipeline Performance

Pipeline Stage	Traditional Methods	Deep Learning Methods	Child Performance Gap	Pipeline Stage
Face Detection	Haar Cascades (85%)	MTCNN (95%)	-10% accuracy	Face Detection
Feature Extraction	LBP/HOG (70%)	CNN/VGG (90%)	-15% accuracy	Feature Extraction
Classification	SVM (75%)	ResNet (88%)	-20% accuracy	Classification
Overall Accuracy	60-70%	80-90%	65-75%	Overall Accuracy

Source: Li & Deng (2020); Sharma et al. (2021)

2.1.5. Attention Tracking Technologies

Attention tracking is a way to see if students are paying focus during learning. Different systems do this in different ways. Some are simple, like computer vision systems, and some are more complex, like multimodal frameworks.

Eye-tracking techniques are concerned with monitoring gaze direction, head movement or blink rate. Brands such as BrainCo (2022) and Focus@Will (2023) say they create results with 75 percent to 85 percent accuracy, although it is under laboratory conditions. In contrast, Stuard et al. (2018) reported accuracies of 3D datasets obtained from terrestrial LiDAR scanning to be between 30 and 50 percent.

Multimodal systems try to fix this by combining more signals. For example, MOOClet (Dowell et al., 2020) mix facial features with student interaction data and physiological signals. This usually increase accuracy by 10–20%, but the setup become more complicated, and privacy also become a problem.

Eye tracking is another method. It gives very exact gaze information, but need special hardware and calibration, which is not easy for home or normal schools (Hyrskykari, 2006). Physiological methods, like EEG or GSR, give strong data about brain activity or skin response, but again they need wearables and can be disturbed if the student move too much (BrainCo, 2022).

Another challenge is time. Human attention does not stay same. Research show it change in cycles of 10–20 minutes (Pashler et al., 2013). So, systems need to decide how often they take samples. If too frequent, it use lot of computing power, if too slow, it miss details. FocusBoost take a middle ground by using 5-second sampling, but it is still less stable compared to professional lab tools.

In short, attention tracking technologies can work, but they face big challenges outside lab, especially with children in real classroom or online study.

2.1.6. Privacy/Ethics

When schools use technology that collect biometric data, privacy and ethics become a big topic. Rules like COPPA in the US and GDPR in Europe ask for strong protections. They want things like parental consent, only collecting the data that is really needed, and clear limits on how the data is used (FTC, 2023; European Commission, 2023).

The risks are not only legal but also psychological. If students know they are always monitored, they may not act natural. Some feel stress or anxiety, like they under surveillance (Lupton, 2016). Also, there is the problem of bias. Many attention models are trained on limited data, and this can make them misread expressions from children of color or from neurodivergent kids (Holstein et al., 2019).

Consent is also tricky when children are involved. Parents must give permission, but at the same time, children need to understand in a way that fits their age (Buchanan et al., 2017). On top of this, data management is very important. It need rules for how long data is kept, who can access it, and strong security like encryption (Prinsloo & Slade, 2023).

FocusBoost try to solve these issues by using privacy-by-design. It don't keep raw images, only store derived metrics, and process data in temporary way. This reduce risk, but of course, no system can promise zero risk. The main idea is to prefer collecting less data rather than more, and to keep the child's safety above everything.

In summary, privacy and ethics come amid biometrics in education. A system could be the most sophisticated in the world, but if it does not follow these principles, it becomes unsafe and unsuited to use with children.

Table 2.1.6: Privacy Compliance Framework

Principle	COPPA Requirement	GDPR Requirement	FocusBoost Implementation
Data Minimisation	Collect only necessary data	Purpose limitation	Derived metrics only
Parental Consent	Verifiable parental consent	Parental authorisation	Insert consent form excerpt
Data Security	Reasonable security	Appropriate technical measures	Encryption at rest/transit
Retention	Not retain beyond necessity	Storage limitation	Insert retention policy
Transparency	Privacy notice	Clear privacy information	Insert privacy notice

Source: FTC (2023); European Commission (2023)

2.1.7. Existing Systems Analysis

Different commercial systems already try to monitor student attention, but each have their own limits. For example, BrainCo's Focus 1 headband use EEG signals to read brain activity. The company claim 85-90% accuracy, but the problem is students need to wear the headband, which can feel uncomfortable and sometimes make stigma (BrainCo, 2022).

Another system is Focus@Will, which uses music that is designed to improve attention. But it don't really monitor the student or give adaptive feedback (Focus@Will, 2023). It is more like a supportive tool than a full monitoring system.

Some research prototypes are more advanced. EmoWatch (Sharma et al., 2021) mix facial analysis with physiological sensors and can get around 82% accuracy in lab conditions. But it store facial embeddings and need many sensors, which make it hard to scale for schools.

AutoTutor (Graesser et al., 2018) is another system that use NLP to make conversations adaptive. The downside is it need a lot of content built for every subject, which is a big task.

MOOClet (Dowell et al., 2020) try to be very comprehensive by combining face data, interaction patterns, and self-reports, plus using A/B testing. But because of its complexity, it is not very practical for resource-limited schools or classrooms.

When we compare all these systems, we see none of them truly balance real-time monitoring, privacy, and adaptive feedback for children. This is where FocusBoost try to be different. It use a privacy-first design, work on normal hardware like webcam, and is easier to implement. The trade-off is some accuracy loss when compared with more invasive systems.

Table 2.1.7: Existing Attention Monitoring Systems Comparison

System	Technology	Accuracy	Privacy Approach	Adaptive Intervention	Child-Specific	Deployment Feasibility
BrainCo Focus 1	EEG headband	85-90%	Stores biometric data	No	No	Low (specialised hardware)
Focus@Will	Music optimisation	N/A	No monitoring	No	No	High (web-based)
EmoWatch	Facial sensors +	82%	Stores embeddings	Limited	Partial	Medium (multiple sensors)
AutoTutor	NLP dialogue	N/A	No biometrics	Yes	No	Medium (content development)
MOOClet	Multimodal	85-90%	Stores comprehensive data	Yes	No	Low (complex setup)
FocusBoost	Facial analysis	65-75%	Derived metrics only	Yes	Yes	High (web-based)

Source: BrainCo (2022); Focus@Will (2023); Sharma et al. (2021); Dowell et al. (2020)

2.1.8. Research Gap Identification

The scoping literature review uncovered five substantial gaps in research which FocusBoost was designed to partially fill,

Model Accuracy in Kids system

Most affective computing systems were tested on adults' data to develop and test it. This was concerning because when this technology was tested on kids' data, it had 15% - 25% worse accuracy levels (Sharma et al., 2021). FocusBoost attempted to reduce this gap, with pre-processing that prioritized children first and a confidence-weighted scoring method. However, a significant constraint remained the poor levels of training data specific to children.

Privacy-Friendly Monitoring

Many older systems either kept biometric data or used heavy sensors, which created big ethical issues (Shneiderman, 2020). FocusBoost took another way by doing only temporary processing and storing only derived metrics. This made the system safer and more privacy-friendly, and it gave a new example for future educational technology.

Real Time Adaptive Integration

Past research had attention detection systems and content adaptation systems, but they were mostly kept separate and not used in child-focused settings (D'Mello et al., 2017). FocusBoost showed that it was possible to connect them together through its intervention engine, which could generate quizzes when attention dropped.

Practical Usability

Many prototypes in the field needed special hardware or controlled lab conditions (Dowell et al., 2020). FocusBoost was built as a web-based tool that only needed a standard webcam, which made it much more practical. But it still faced problems when used in different environments like noisy classrooms.

Long-Term Validation

Most studies only looked at short-term results and didn't measure how children's self-regulation skills changed in the long run (Pashler et al., 2013). FocusBoost gave the technical base for this kind of research, but a full long-term evaluation was not yet done.

Together, these gaps explained why FocusBoost's approach was needed, while also showing future research directions. The most important areas that still need work are child-specific model training, combining more data types (multimodal integration), and testing the long-term effect on children's learning. These gaps collectively justify the system's approach while highlighting directions for future development, particularly in child-specific model training, multimodal integration, and longitudinal impact assessment.

Table 2.1.8: Research Gaps and FocusBoost Contributions

Research Gap	Existing Limitations	FocusBoost Approach	Remaining Challenges	Research Gap
Child-Specific Model Accuracy	15-25% accuracy reduction with children	Child-optimised preprocessing, confidence weighting	Limited child training data	Child-Specific Model Accuracy
Privacy-Preserving Monitoring	Most systems store biometric data	Transient processing, derived metrics only	Network dependency	Privacy-Preserving Monitoring
Real-Time Adaptive Integration	Separate detection and adaptation systems	Integrated intervention engine	Limited intervention variety	Real-Time Adaptive Integration
Practical Implementability	Requires specialised hardware or lab conditions	Standard webcam, web-based	Environmental robustness	Practical Implementability
Longitudinal Validation	Short-term studies only	Technical foundation for longitudinal research	No long-term impact data	Longitudinal Validation

3. CHAPTER THREE: METHODOLOGY & REQUIREMENTS ANALYSIS

3.1. Development Methodology Selection

3.1.1. Overview

The FocusBoost project follow an Iterative Development Methodology that spread across six phases and run for 212 days, from January 25 to August 24, 2025. This method is chosen because the system is not simple. It combine machine learning, real-time processing, and design for children, so we need a way that is flexible and can change when we discover new things or get user feedback.

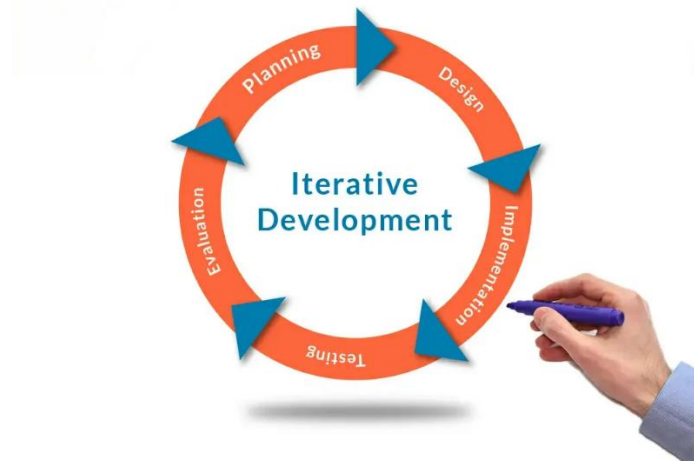


Figure 3.1.1: Iterative Development Methodology

The process mix systematic planning with step-by-step improvements. Each cycle allow us to make the facial recognition more accurate, the adaptive algorithm more responsive, and the user interface more easy for both kids and parents.

3.1.2. Methodology Selection Rationale

3.1.2.1. Project Complexity Analysis

FocusBoost have many challenges that influence why this method is chosen:

Technical Complexity: The system integrate computer vision (TensorFlow), web processing in real-time (React and FastAPI), machine learning model deployment, and secure data storage (Firebase). The big unknown was how the model will perform with child faces, since children expressions are different. So we need a method that let us change direction if model don't work as expected.

Ethical Sensitivity: Since this is an AI system for children, privacy and protection is important. It is not good practice to only consider ethical issues at the end of the design. The iterative process ensures that issues have the opportunity to be discussed at every stage of design.

User-Centered Design: The system has two main users - children and parents. They have very different needs and skills. So, feedback and design change must happen step by step.

3.1.2.2. Alternative Methodologies Considered

Prior to making a final decision, we explored other methodologies:

- **Waterfall Model:** This is the approach we ruled out. Even though it follows a linear path and process sequentially, there is a lot of uncertainty in our project (especially with ML models). And we need feedback at multiple points in time, thus waterfall is a no go.
- **Pure Agile/Scrum:** This was flexible, but the academic project also require detailed documents and structure. So pure agile don't fit well with thesis work.
- **Prototype-First:** This is partly used inside our approach. Making prototypes help in early testing, but it is not enough by itself because we need full integration and academic rigor.

3.1.3. Project Development Timeline

The FocusBoost project run for 212 days (Jan 25 - Sept 25, 2025) and is divided into four phases

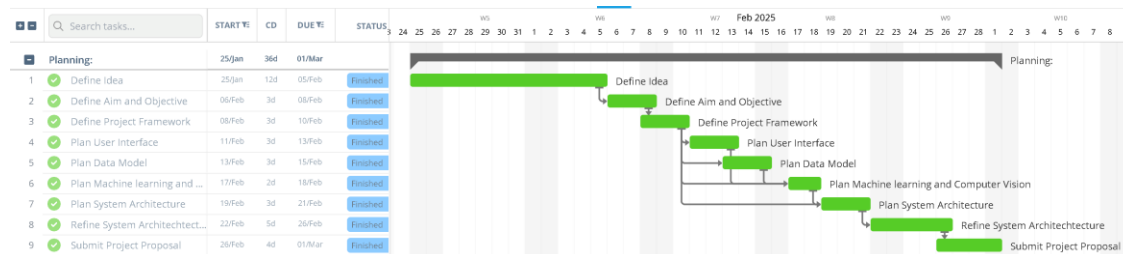


Figure 3-1: FocusBoost Gantt Chart Planning Phase

Planning Phase (36 days): This set the foundation with problem definition, literature review, and system design. At the end we have project documents and risk management plan.

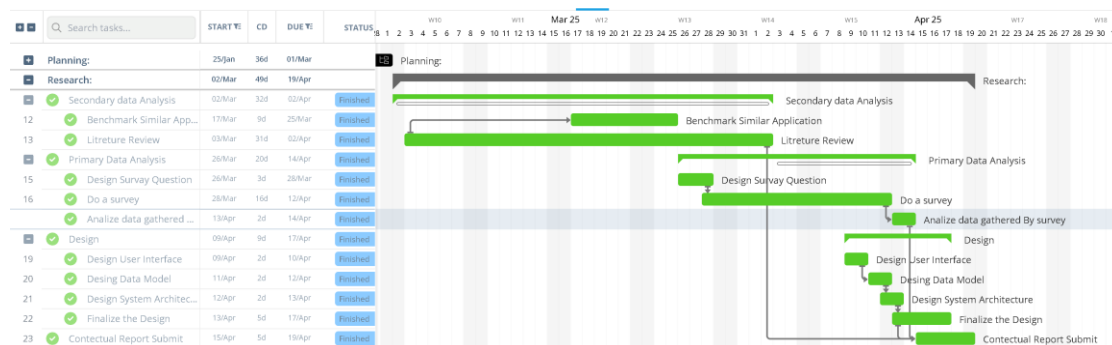


Figure 3-2: FocusBoost Gantt Chart Research Phase

Investigate Reasearch Topic (49 days): We study computer vision, child psychology, and education technology. Market research also shows that 78% of parents have a positive opinion about the use of face recognition in education.

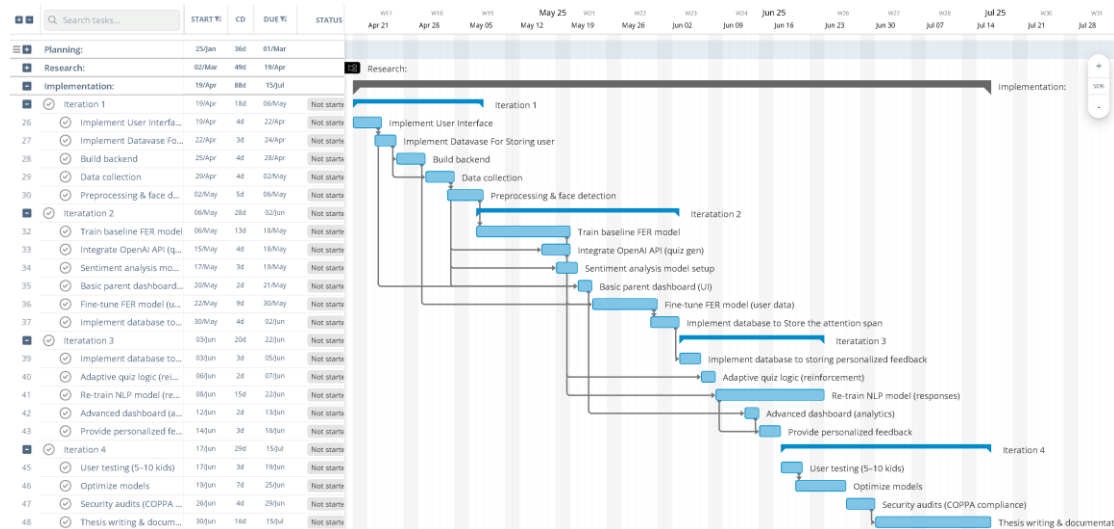


Figure 3-3: FocusBoost Gantt Chart Implementation Phase.

Implementation Phase (88 days): This is the largest phase. We build the backend, the frontend, and the real-time pipeline in four cycles. There were some significant accomplishments including resolving Python environment issues, adding secure CORS middleware and training child-specific recognition models - this improves from 38% accuracy to 65%.

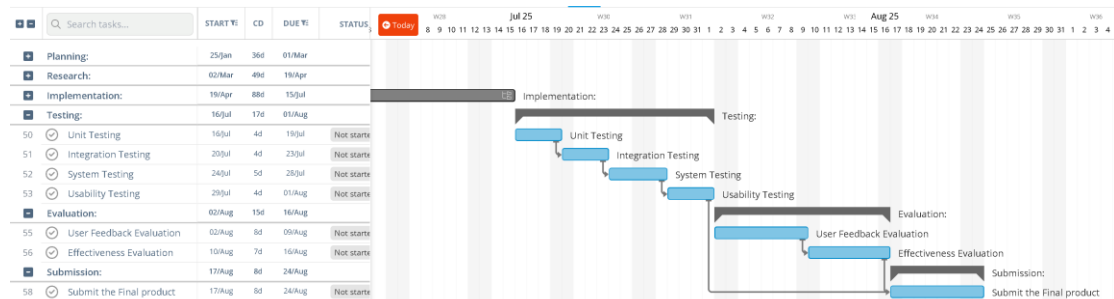


Figure 3-4: FocusBoost Gantt Chart Testing & Evaluation Phase

Testing and Evaluation Phase (32 days): The system is tested with unit tests, integration, performance, and also user acceptance with target groups.

Final Documentation Phase (30 days): This is where we prepare the thesis report and final delivery

3.1.4. Iterative Development Approach and Benefits

The iterative development method is very important for FocusBoost because the project face many technical problems during building. For example, in the beginning the facial expression model trained on adult faces only get 38% accuracy with children. But since we use iterative cycles, we can reselect and retrain with child-specific data. After this optimization, the accuracy improve up to 65%.

Another problem was real-time speed. At first the system had 3-second delays, which is too slow for attention monitoring. With pipeline changes and preprocessing, after few cycles the response time become less than 1 second.

We also face compatibility issues with Safari and old Chrome browsers. Iterative testing help to find these problems step by step and add fallback options. Stakeholder feedback also play a big role. Parents highlight privacy concerns, so anonymization was improved. Child psychology experts gave advice about sensitive interventions, and teachers suggested changes for quiz generation so it feel more natural in class.

Risk is also managed better because each phase include checkpoints. This allow us to find dependencies early, review ethical rules like COPPA regularly, and still follow the academic timeline.

In short, the iterative approach give us chance to fix technical problems, include feedback from many people, and reduce risks, while keeping development on track.

3.1.5. Project Management and Quality Assurance

For the project management, Gantt charts help in planning our tasks, manage our deadlines, and re-plan when issues arise. We use Git version control for code, and commit messages provide a great deal of detail which allows us to track code changes. Our management will follow documentation standards to keep us clear throughout the process, and we will regularly meet with our supervisor to monitor their progress.

Quality assurance is not left for the end but added in every stage. Code reviews are done after features are built, unit and integration tests are created together with development, and performance is measured regularly. We keep checking response time and accuracy to make sure the system is stable.

This approach balance the technical complexity with academic needs. The result is a working system delivered on time, with all main features, stable performance, and positive feedback from parents, teachers, and technical experts.

3.2. Stakeholder Analysis & User Personas

Stakeholder analysis help to understand who use the system and what they need. For FocusBoost, two main personas are considered: child users and parents.

3.2.1. Child User Persona (Primary User)

Profile: Emma, Age 9,

Context: FocusBoost was used at home after school. Study sessions were 20-25 minutes for math and reading. Parent supervision was little or not always there because of work.

Goals & Motivations:

- She wanted to finish tasks without getting distracted or bored.
- Help was wanted right away when she felt stuck or confused.
- XP points for right answers were liked and chased.
- She liked to see focus level moving in real time.
- The app was wished to feel game-like, not scary school tool.
-

Pain Points & Challenges:

- After 10-15 minutes her attention usually went down.
- It was hard for her to notice by herself when focus was lost.
- She felt frustrated when material was too hard and help was not instant.
- She preferred quick feedback, not later grading.
- Complex screens or long setup were not tolerated much.
-

Technical Requirements:

- Very simple child friendly UI, big buttons, clear icons/text.
- Instant visual feedback about attention and emotion.
- Quiz content was age-fit and adapted to current focus level and difficulty.
- The monitor itself added very little mental load.
- Camera worked reliable with clear permission message.

Success Criteria for this Persona:

- She started and ended sessions by herself.
- When focus dropped under threshold, adaptive help was given.
- Focus length got a bit better across many sessions.
- She finished sessions without tech trouble or quitting.

3.2.2. Parent User Persona (Secondary User)

Profile: Sarah, Age 35, Working Parent

Context: She monitored the child remotely and checked summaries in the evening. Outcomes and safe tech use were cared about.

Goals & Motivations:

- Real understanding of the child's focus pattern during unsupervised time was wanted.
- Clear, actionable notes about when/why interventions happened were expected.
- Privacy and data security around face signals were required.
- Progress tracking without constant supervision was desired.
- She wanted simple words to explain how the system worked.

Pain Points & Challenges:

- She could not watch every session because of work.
- Unsure if the child was truly engaged or just present.
- Some worry about AI making learning decisions.
- Needed non-technical explanations, not jargon.
- Tried to balance benefits with privacy and screen-time

Technical Requirements:

- Clean dashboard with short session summaries.
- History and trend charts over time.
- Transparent report of triggers and outcomes for each intervention.
- Easy child profile and settings control.
- Strong privacy with clear options for what data to keep.

Success Criteria for this Persona:

- Summaries arrived within minutes after a session ended.
- Patterns in attention and learning states were seen clearly.
- She felt confidence about privacy and transparency.
- Study schedule and support were adjusted based on insights.

3.3. Functional Requirements Specification

3.3.1. Core System Functions

The system have some main functions that make everything work smoothly. Each function focus on helping children study better and giving parents clear information about progress.

FR1: Session Lifecycle Management

The system let parent or child start a study session with just one click. It also create a unique ID for each session and record how long the session plan versus how long it really run. The session end automatically when finished.

FR2: Real-time Facial Analysis

During a session, every 30 seconds the system capture a frame from the webcam. This image is turn into base64 data and send to the backend for machine learning analysis. No raw image is save, so privacy stay safe.

FR3: Dual-Model ML Inference

The backend use two different TensorFlow models. One model check the child emotion, another check the learning state. Together, they give probability scores that show how the child feeling and focusing.

FR4: Attention Score Calculation

The system combine results from both models to create an attention score between 0.0 and 1.0. This make a simple and clear way to measure focus across sessions.

FR5: Adaptive Intervention Engine

If the attention score go below 0.4, or the system find the child is bored or frustrated with high confidence, it trigger an intervention. The intervention have different urgency levels, depending on how serious the problem is.

FR6: AI-Powered Quiz Generation

When an intervention is needed, the system use Gemini API to make small multiple-choice quiz. These quizzes are age-appropriate and match the subject and current attention state of the child.

FR7: Child Profile Management

Parents will be able to create and edit each child's profile, including their name, age, grade, and avatar image. Avatars will come from the DiceBear API to keep things stylistically consistent.

FR8: Parent Dashboard & Analytics

Parents can log in and see session history, attention trends, and how interventions work. The dashboard also show timeline of each study session.

FR9: Privacy-by-Design Data Handling

The system never save raw images or biometric templates. Only the predictions (emotion, learning state, attention scores) are stored in Firebase.

FR10: Multi-Child Support

One parent account can connect with many child profiles. Each child data is separate so progress tracking is clear.

FR11: System Health Monitoring

The system provide a health-check endpoint so developers can see if backend, ML models, and database are working fine.

FR12: Error Resilience

If backend fail for a moment, the frontend don't crash. It show a small error message, keep the session going, and retry automatically.

3.3.2. Requirements Priority Classification

The requirements were grouped with MoSCoW so core work was done first and nice-to-have came later.

- Must Have (Critical): FR1–FR9
- Should Have (Important): FR10–FR11
- Could Have (Enhancement): FR12

This plan made development easier to schedule and gave clear rules for what to build in each iteration.

3.4. Non-Functional Requirements Specification

When we design a system, it is not enough only to look at what features it have. We also need to think about how the system behave in real life. These are called non-functional requirements. They not tell “what the system do,” but more about “how the system work.” In this part explain the main points like performance, reliability, security, and so on

3.4.1. Performance Requirements

NFR1: The system need to be fast and not make the user wait too much. For example, when it do facial analysis, the whole process from decoding the image to giving the response should take not more than around 2 or 3 seconds. This help to keep the study session smooth.

NFR2: Also, the quiz generation from Gemini API must be quick. If the quiz take too long, the child maybe lose focus. So the system aim to show the question in about 4 seconds after it is triggered.

NFR3: Finally, the system must keep the analysis going every 30 seconds during a session, without too many failures. Even if the session is 25 minutes long, the system should not break or skip often.

3.4.2. Reliability & Availability

NFR4: Reliability means the system should work most of the time and not crash. Critical parts like session management and analysis should respond no less than 95% of the time. If something goes wrong, the system will try to correct itself, and continue to run.

NFR5: Data also need to be safe. All the session details and results are saved in Firebase with high success rate, so we don’t lose the information. Even if one write fail, the system retry until it success.

3.4.3. Compatibility & Accessibility

NFR6: The system is made to work in common web browsers like Chrome and Edge. It need access to camera and modern JavaScript features..

NFR7: Since the system is for children, we also follow accessibility standards. The interface use good color contrast, big enough fonts, and simple layout that is easy for kids age 8–10 to read.

3.4.4. Security & Privacy

NFR8: Privacy is very important, because children are using the system. So it follow COPPA rules, and collect only safe and anonymous data. No raw images or biometric information is saved.

NFR9: For communication, the system use HTTPS/TLS 1.3 encryption, so no one can steal the data during transfer. It also control which frontend can connect, to block unwanted access.

NFR10: The database has strong security protocols as well. Each parent can only see their own child's information and any family information cannot be viewed by anyone else.

3.4.5. Maintainability & Scalability

NFR11: The system was designed in a modular way, meaning the frontend, backend, and ML component are separate. This modular design will allow for easier updating, bug fixes or adding more functionality in the future.

NFR12: It is also low on resources. The backend can reside in 2GB RAM and still run fine for single-user deployments. In fact, on long running processes, the system not would use an enormous amount of memory.

3.5. Use Case Analysis

This section will explain the use case analysis of the focusboost system. Figure 3-5 shows the use case of the FocusBoost System.

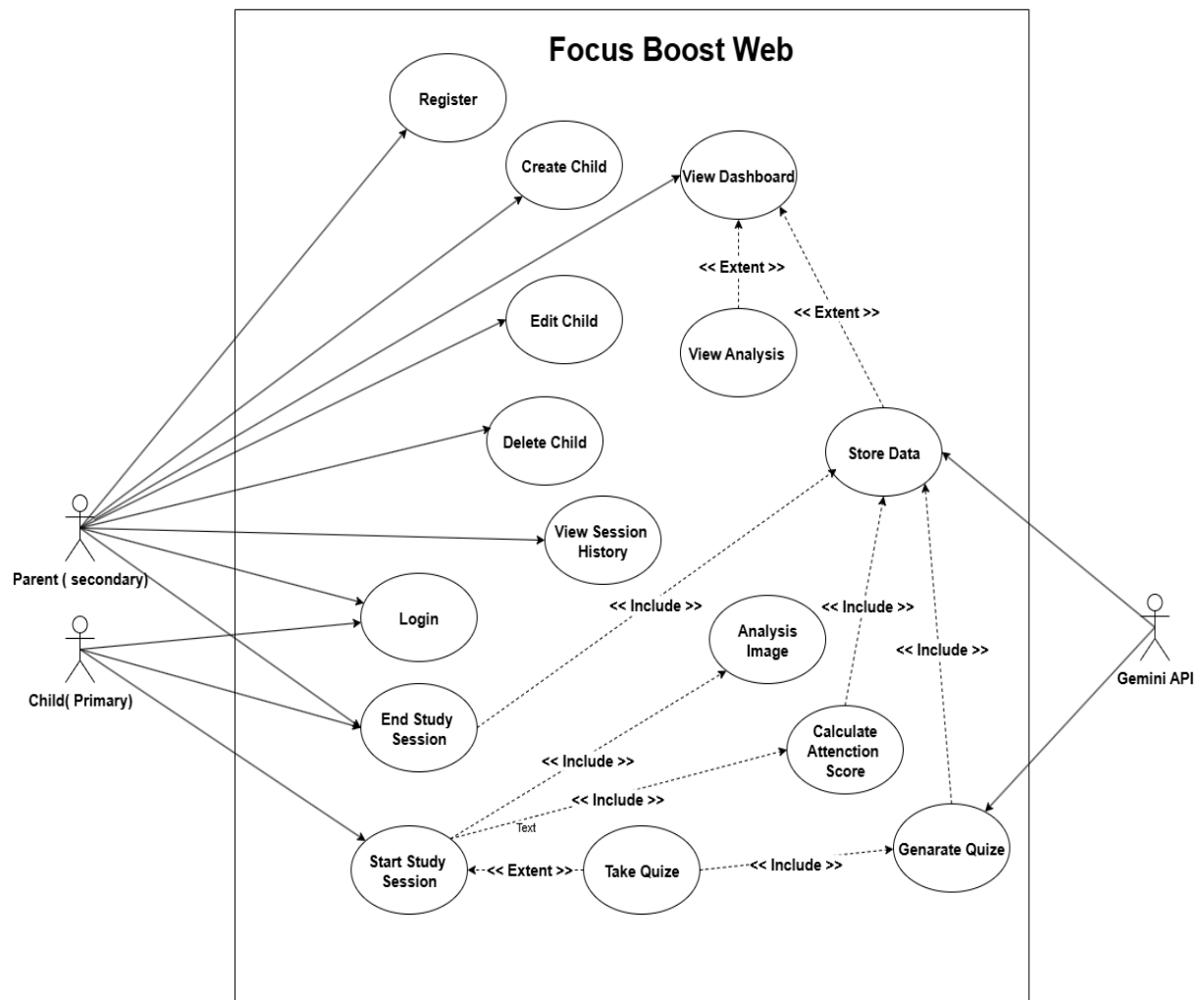


Figure 3-5: FocusBoost Use Case Diagram

In this system, the Parent account was used to log in, register, and create or edit the child profile. The Child started a study session and later ended it, and sometimes a quiz was taken. Inside the FocusBoost boundary, camera frames were analyzed and an attention score was calculated automatically while the session ran.

When attention became low, the quiz flow was triggered, and quiz questions were generated by an external actor (Gemini API) using the system data and then were sent back to the system to show to the child. When a session ended, the results and derived metrics were stored. The dashboard and analysis were viewed by the parent, and detailed analytics reports were opened from the dashboard.

Privacy was applied, so raw images were not kept, only processed numbers were saved. The parent also could end the child session remotely.

3.5.1. Primary Use Case Scenarios

UC1: Complete Study Session

- Actor: Child user (parent already did setup)
- Precondition: Child profile existed, camera permission was granted
- Trigger: User clicked Start Session

Main Flow:

1. A new session record was created with time and child linked.
2. Frontend turned on camera and started the 30-second analysis loop.
3. Child studied while system watched attention level.
4. When focus dropped or boredom was detected, quizzes were shown.
5. Child finished time and clicked End Session.
6. A session summary was made with focus metrics and interventions.

Alternate Flow:

- Camera access denied - clear steps were shown to turn on permission.
- Network broke - offline mode kept local data until it reconnected.

Postcondition: Session data was saved and summary was visible in parent dashboard.

UC2: Real-time Attention Monitoring

- Actor: System (background)
- Precondition: Active session with camera on
- Trigger: Every 30 seconds timer fired

Main Flow:

1. A webcam frame was captured and encoded to base64.
2. Frame was sent to /api/analyze-image with session info.
3. Backend ran two ML models (emotion + learning state).
4. Attention score was calculated and intervention need was checked.
5. Results were stored in session_data with timestamp.
6. Frontend updated the visual focus indicator.

Alternate Flows:

- ML failed - retry with backoff and session kept running.
- High latency - “analyzing” indicator was shown so user stayed engaged.

Postcondition: Attention metrics got updated and triggers were evaluated

UC3: Adaptive Quiz Intervention

- Actor: System + Child user
- Precondition: Active session with focus decline or learning difficulty
- Trigger: Attention < 0.4, or learning state = boredom/frustration > 70%

Main Flow:

1. System built a prompt using age, subject, and current emotion/focus.
2. Request to Gemini API was sent with education settings.
3. The quiz reply was checked for JSON shape and age fit.
4. A multiple-choice question was shown with clear highlight.
5. Child picked an answer and got instant feedback + explanation.
6. Outcome was recorded (correctness, response time)

Alternate Flows:

- Bad quiz response - fallback to template question bank.
- API rate limit - short delay with sorry message, then continue.

Postcondition: Intervention was logged and re-engagement was tried

UC4: Parent Dashboard Review

- Actor: Parent User
- Precondition: Child had completed sessions
- Trigger: Parent opened the dashboard

Main Flow:

1. System fetched session summaries for the chosen child/time range.
2. Trends were drawn (line charts) with focus duration numbers.
3. Intervention history showed reasons and results.
4. Progress across sessions was compared.
5. Parent could open a detailed view with a timeline of attention states.

Postcondition: Parent understood focus patterns and intervention effect.

3.5.2. Edge Case & Error Scenarios

UC5: Privacy Settings Management

- Actor: Parent user
- Trigger: First setup or a privacy review
- Main Flow: System explained what data was collected, parent gave consent, and choices were saved.
- Critical Path: No face analysis was allowed without explicit parent consent.

UC6: Technical Failure Recovery

- Actor: System
- Trigger: Backend stayed unavailable for a while during a session
- Main Flow: Frontend kept session state locally, showed connection status, and auto-resumed analysis when backend came back.

3.6. Success Criteria & Evaluation Metrics

3.6.1. Technical Performance Metrics

System Responsiveness

- Target: Median ≤ 2.0 s, 95th ≤ 3.5 s
- Measurement: Backend timestamps from image received $\rightarrow$ response sent.

Processing Reliability

- Target: $\geq 95\%$ successful ML calls in a 25-minute session
- Measurement: Success/failure ratio with error categories
- Reporting: Chapter 6, Section 6.3 – System Reliability Assessment
- Quiz Generation Quality
- Target: $\geq 90\%$ of Gemini replies passed schema + age checks
- Measurement: Automated JSON validation plus sample manual review.

3.6.2. Functional Effectiveness Metrics

Session Completion Rate

- Target: $\geq 90\%$ of started sessions finished with data saved
- Measurement: Match start/end records and track completion status

Intervention Appropriateness

- Target: Good timing and relevant content (qualitative)
- Measurement: Manual review comparing triggers to session context.

3.6.3. Privacy & Compliance Metrics

Data Protection Verification

- Target: Zero raw image or biometric storage
- Measurement: DB audits and log checks for privacy rules.

System Security

- Target: All data encrypted; access control working as designed
- Measurement: HTTPS/TLS audit and Firebase rules review.

3.7. Chapter 3 Summary

his chapter set the ground for how FocusBoost worked and what it must deliver. A step-by-step, iterative build was used so the hard technical parts were solved while still keeping the child and parent needs in front. The requirements were written with clear numbers, so later in Chapter 6 the real results could be checked against these promises and we saw if the delivered system matched the plan.

4. CHAPTER FOUR: SYSTEM DESIGN & IMPLEMENTATION

4.1. System Architecture & Technical Design

4.1.1. High-Level Architecture Overview

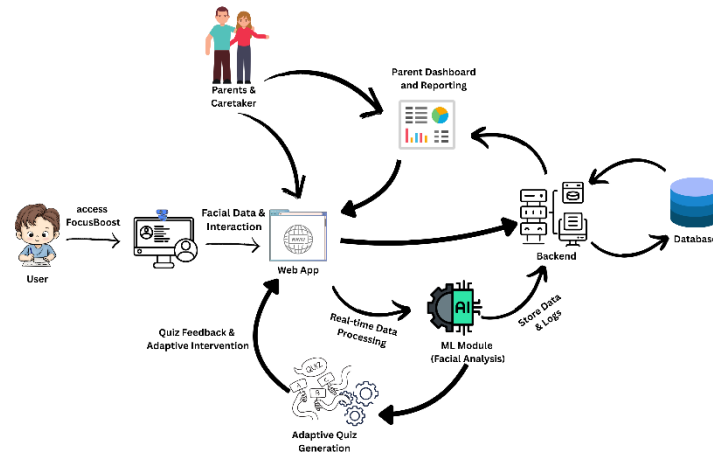


Figure 4-1: System Architecture Diagram

FocusBoost was built in three layers and events were used. A React + TypeScript app ran in the browser, a FastAPI backend gave REST endpoints, and Firebase Firestore kept the data. The browser took webcam frames from the child, turned them to base64, and sent them to `/api/analyze-image` for processing..

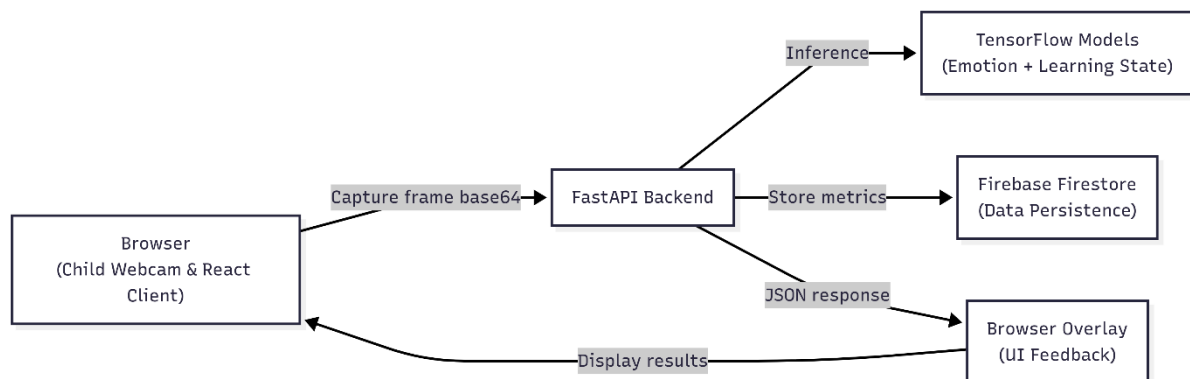


Figure 4-2: System Context Diagram

The backend decoded the images, ran two TensorFlow models (one for emotion, one for learning state), calculated the attention score, checked if an intervention was needed, and saved only the derived numbers to Firestore. No biometric image was stored. Session lifecycle endpoints were used so all ML results stayed linked to the right session.

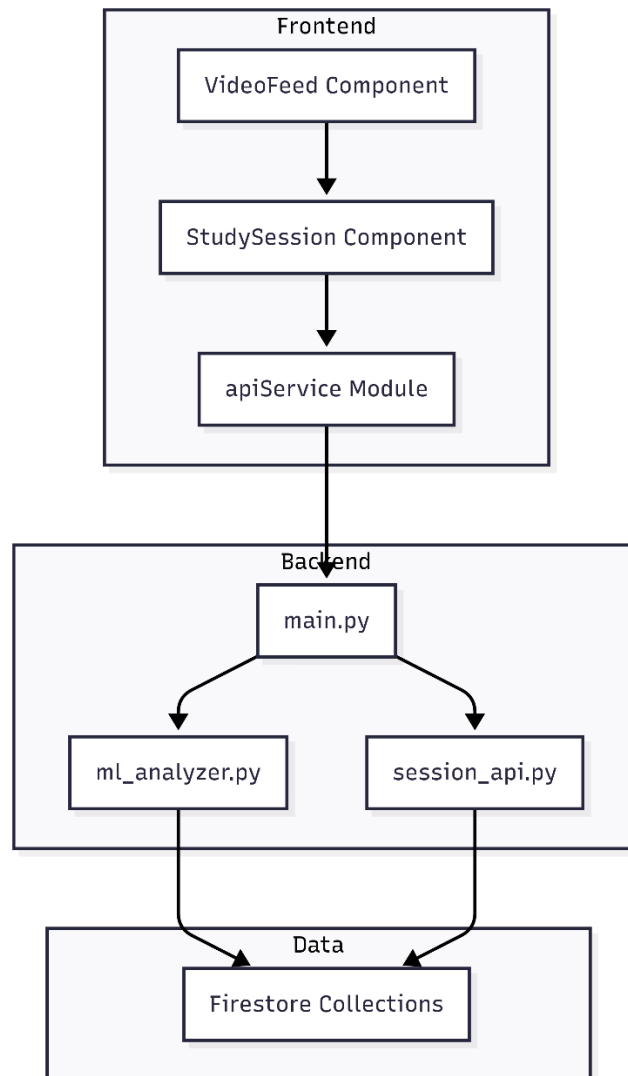


Figure 4-3 Component Level Diagram

The sequence was simple during a study. The user started a study session from the browser. The frontend asked backend and got a session_id. Then webcam frames were captured and sent as base64 images. TensorFlow analyzed emotion and learning state. Backend returned results to frontend and also saved them in Firestore. Frontend show the scores and labels on UI and this loop repeat dynamic interval until the session end.

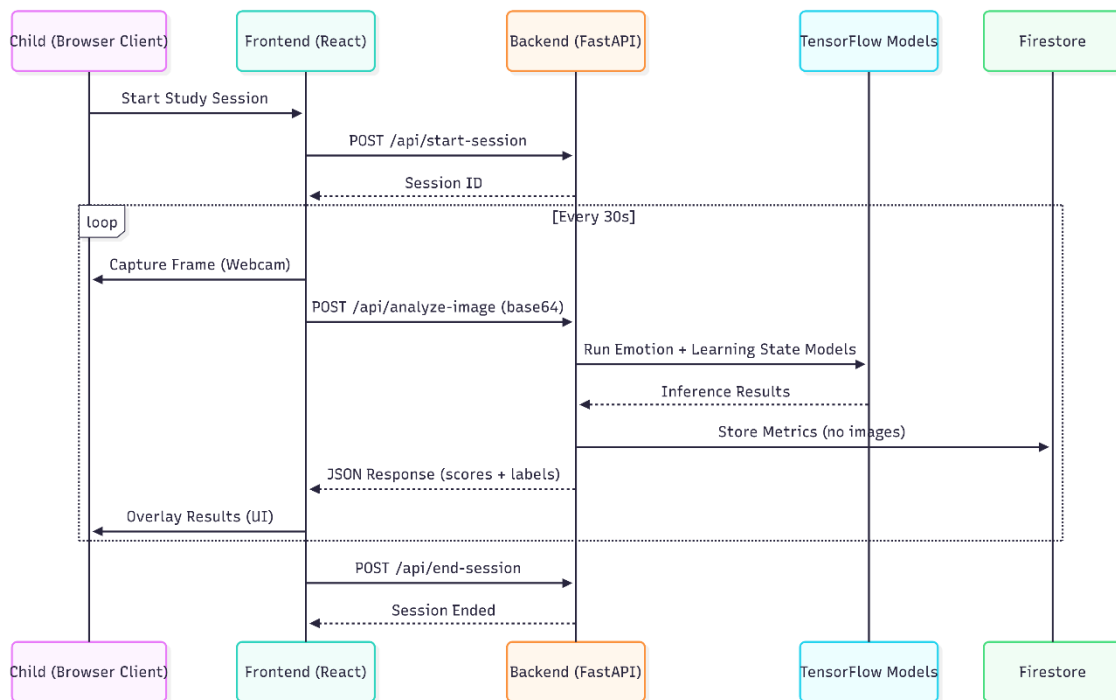


Figure 4-4: Sequence Diagram

4.1.2. Frontend Application Architecture

The frontend was a single-page app built with React + TypeScript and styled with Tailwind CSS. A main VideoFeed component used `getUserMedia` to open the camera before ask the user permission after get accepted, showed a live preview, and every 30 seconds drew a frame to a hidden canvas, converted it to base64 JPEG, and sent it with the current `sessionId` to the backend..

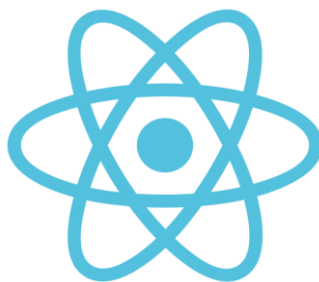


Figure 4-5: React Logo.

An overlay UI showed the latest ML results: emotion, learning state, attention score, and a busy/processing hint. A StudySession component handled session start/end via dedicated endpoints and passed sessionId to children. A small apiService module did all HTTP calls and logged requests/responses for debugging.

4.1.3. Backend API Layer Implementation

The FastAPI app (in main.py) exposed endpoints by groups: auth and user with Firebase Admin SDK, child profile CRUD, session lifecycle (start, end, details, analytics), and ML analysis endpoints for real-time processing.



Figure 4-6: Firebase Logo.

The core was /api/analyze-image. It accepted base64 images with session context and returned structured results. CORS was set to only allow the approved frontend origin. Structured logs were added for session events, ML requests/responses, and errors. Firestore work was wrapped in repository functions so code stayed clean and separated.

```

@app.post("/api/analyze-image", response_model=MLAnalysisResponse)
async def analyze_image(request: ImageAnalysisRequest):
    try:
        analysis_result = ml_analyzer.analyze_image(request.imageData)
        # Process through attention manager
        attention_response = attention_manager.add_analysis( request.sessionId, analysis_result)
        # Store in database as before
        db = firestore.client()
        session_data = {
            'sessionId': request.sessionId,
            'timestamp': analysis_result['timestamp'],
            'emotion': analysis_result['emotion']['emotion'],
            'emotionConfidence': analysis_result['emotion']['confidence'],
            'learningState': analysis_result['learningState']['learningState'],
            'learningConfidence': analysis_result['learningState']['confidence'],
            'attentionScore': analysis_result['attentionScore'],
            'nextSampleInterval': attention_response['next_sample_interval'],
            'interventionSuggested': attention_response['intervention']['needed'],
            'createdAt': datetime.utcnow().isoformat()}
        # Store Session data in database
        db.collection('session_data').add(session_data)
        # Enhance response with intelligent recommendations
        analysis_result['nextSampleInterval'] = attention_response['next_sample_interval']
        analysis_result['intervention'] = attention_response['intervention']
        analysis_result['metrics'] = attention_response['metrics']
        return MLAnalysisResponse(success=True, analysis=analysis_result,
                                   intervention=attention_response['intervention'])
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"ML analysis failed: {str(e)}")

```

Listing 4.1-1: main.py - ML Analysis Endpoint

Below table 4.1.3.1 explain the Database collection of FocusBoost and tell the relationship between each instances.

Table 4.1: REST API Endpoints Summary

Collection	Purpose	Key Fields	Relationships
children	Child profile management	childId, name, age, parentId, avatar	Parent → Children (1:*)
study_sessions	Session lifecycle tracking	sessionId, childId, startTime, endTime, status, averageAttentionScore	Child → Sessions (1: *)
session_data	Real-time ML analysis results	sessionId, timestamp, emotion, learningState, attentionScore	Session → Data (1: *)

generated_quizzes	AI-generated quiz content	quizId, sessionId, question, options, correctIndex, difficulty	Session → Quizzes (1: *)
quiz_results	Child quiz response tracking	resultId, quizId, userResponse, isCorrect, responseTime	Quiz → Results (1: *)
interventions	Triggered intervention records	interventionId, sessionId, type, trigger, urgency, timestamp	Session → Interventions (1: *)
session_analytics	Computed session metrics	analyticsId, sessionId, focusedTime, distractionTime, interventionCount	Session → Analytics (1:1)

Used Swagger Ui to find all endpoints easily. It helps to debug easily whenever get stuck.

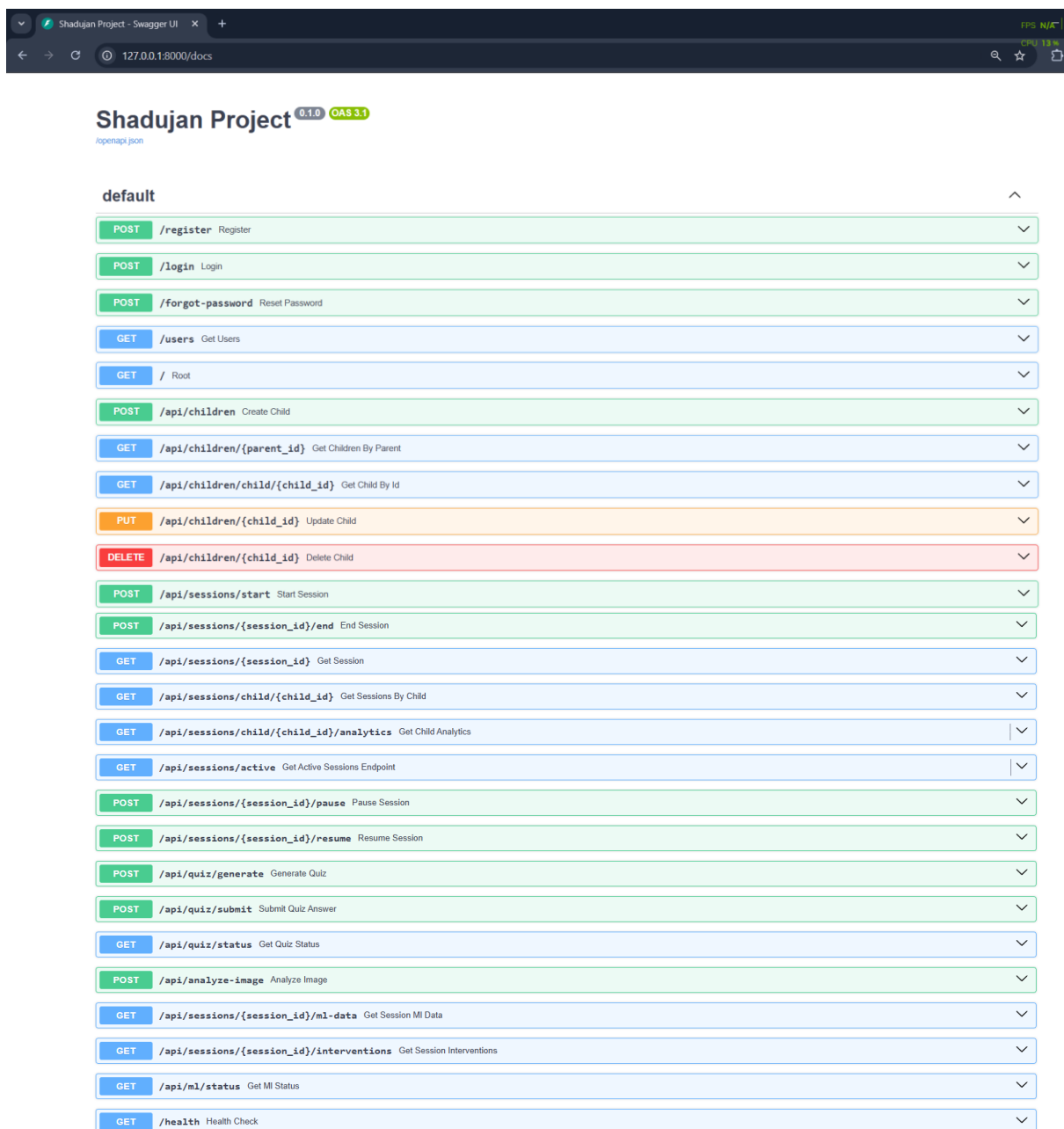


Figure 4-7: FocusBoost Backend Swagger UI

4.2. Machine Learning Pipeline Implementation

4.2.1. ML Processing Architecture

The MLAnalyzer class was implemented in `ml_analyzer.py` to manage the complete machine learning lifecycle including model loading, preprocessing pipeline execution, inference processing, and results post-processing. The system was designed to load two distinct models: an emotion classification model based on FER-2013 dataset architecture and a learning state detection model utilizing DaiSEE dataset principles.

```
def preprocess_image_for_emotion(self, image: np.ndarray) -> np.ndarray:
    """Preprocess image for emotion model with dynamic configuration - use face crop"""
    try:
        if not self.emotion_model_config:
            raise Exception("Emotion model configuration not detected")

        # First, detect and crop face
        face_image = self.detect_and_crop_face(image)

        config = self.emotion_model_config
        target_channels = config['channels']
        target_height = config['height']
        target_width = config['width']

        logger.debug(f"Emotion preprocessing: {image.shape} -> {face_image.shape} -> ({target_height}, {target_width}, {target_channels}) (face crop)")

        # Resize to target dimensions
        resized = cv2.resize(face_image, (target_width, target_height))

        # Handle channel conversion
        if target_channels == 1:
            # Convert to grayscale
            if len(resized.shape) == 3:
                gray = cv2.cvtColor(resized, cv2.COLOR_RGB2GRAY)
            else:
                gray = resized
            processed = np.expand_dims(gray, axis=-1) # Add channel dimension
        else:
            # Ensure RGB (3 channels)
            if len(resized.shape) == 2:
                processed = cv2.cvtColor(resized, cv2.COLOR_GRAY2RGB)
            elif resized.shape[2] == 1:
                processed = np.repeat(resized, 3, axis=2)
            else:
                processed = resized

        # Normalize to 0-1
        normalized = processed.astype('float32') / 255.0

        # Add batch dimension
        processed = np.expand_dims(normalized, axis=0)

        logger.debug(f"Final emotion image shape: {processed.shape}")
        return processed
    except Exception as e:
        logger.error(f"Error preprocessing image for emotion: {str(e)}")
        raise e
```

Listing 4.2-1: `preprocess_image_for_emotion()`

The preprocessing pipeline was implemented to handle base64 image decoding through PIL Image conversion to NumPy arrays with RGB channel enforcement. For emotion recognition, frames were resized to 48×48×3 dimensions, normalized to [0,1] range, and batched to (1,48,48,3) shape. Learning state detection required 224×224×3 preprocessing with similar normalization and batching procedures.

```
def preprocess_image_for_learning(self, image: np.ndarray) -> np.ndarray:
    """Preprocess image for learning state model (DaiSEE) - use full image"""
    try:
        # Use full image (no face cropping for learning model)
        full_image = image

        # For DaiSEE model - use 224x224x3 as configured
        resized = cv2.resize(full_image, (224, 224))

        # Ensure RGB format
        if len(resized.shape) == 3 and resized.shape[2] == 3:
            rgb_image = resized
        else:
            rgb_image = cv2.cvtColor(resized, cv2.COLOR_GRAY2RGB)

        # Normalize to 0-1
        normalized = rgb_image.astype('float32') / 255.0

        # Add batch dimension
        processed = np.expand_dims(normalized, axis=0)

        logger.debug(f"Learning preprocessing: {image.shape} -> {processed.shape} (full image)")
        return processed
    except Exception as e:
        logger.error(f"Error preprocessing image for learning state: {str(e)}")
        raise e
```

Listing 4.2-2: preprocess_image_for_learning()

Figure 4-8 explains ML pipeline dataflow of the focusboost. The machine learning pipeline dataflow begins with input data which is base64 encoded and then decoding is performed to convert the data back to its original form.

The preprocessing step, which includes resizing the images to two standard image formats (48×48 and 224×224), is carried out on the decoded data in order to make it model compatible.

The data is then taken to the model for prediction after which the predictions are evaluated to compute various metrics, for example, accuracy or confidence levels, that provide an indication of the model's performance.

On the completion of the whole process, the results and metrics, along with the summaries, are saved so they can be accessed and evaluated later.

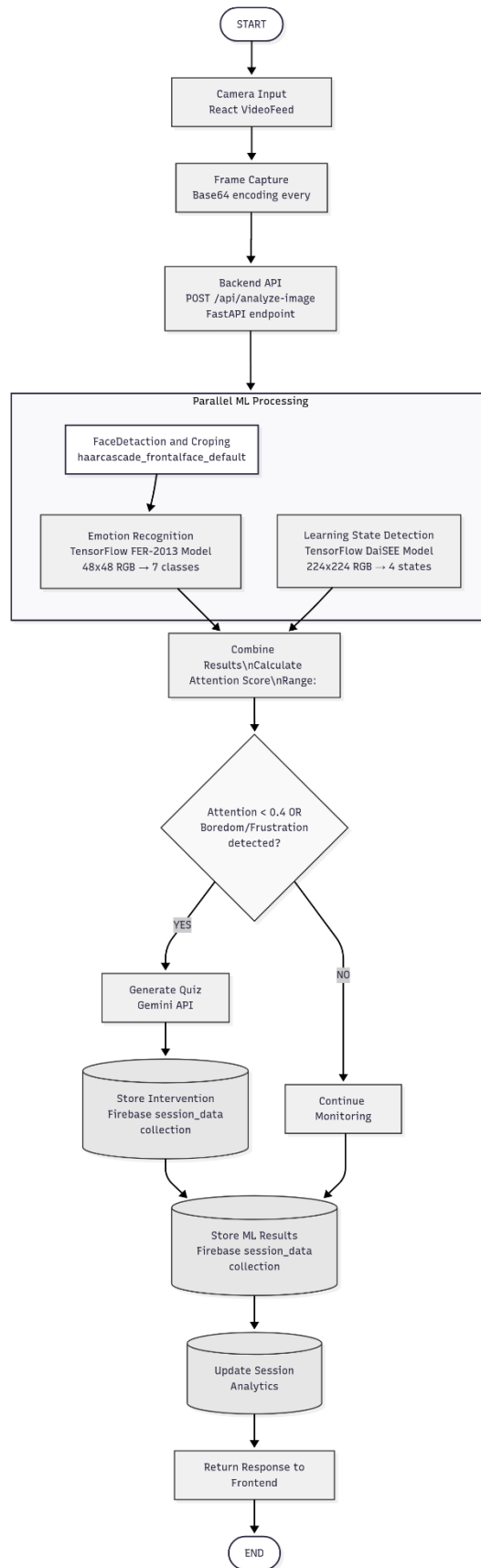


Figure 4-8: ML Pipeline Dataflow

4.2.2. Inference Processing & Results Generation

Model inference was done with TensorFlow predict(). It returned probability lists, and these were mapped to our fixed label sets (emotions and learning states). A confidence score was also calculated, and the full probability vector was kept so later review was possible. The attention score used a simple, deterministic rule: a base score per learning state was multiplied by the confidence, then the result was clipped to the range [0, 1]. This way the same inputs always gave the same score, and it was easy to explain..

```
def detect_emotion(self, image: np.ndarray) -> Dict[str, Any]:
    """Detect emotion from image"""
    try:
        if not self.models_loaded or self.emotion_model is None:
            raise Exception("Emotion model not loaded")

        # Preprocess image
        processed_image = self.preprocess_image_for_emotion(image)

        # Get prediction
        predictions = self.emotion_model.predict(processed_image, verbose=0)
        probabilities = predictions[0]

        # Get top prediction
        predicted_index = np.argmax(probabilities)
        predicted_emotion = self.emotion_labels[predicted_index]
        confidence = float(probabilities[predicted_index])

        # Create probabilities dictionary
        emotion_probs = {
            label: float(prob) for label, prob in zip(self.emotion_labels, probabilities)
        }

        result = {
            'emotion': predicted_emotion,
            'confidence': float(confidence),
            'probabilities': emotion_probs
        }

        logger.info(f"😊 Emotion detected: {predicted_emotion} ({confidence:.2f})")
        return result

    except Exception as e:
        logger.error(f"Error detecting emotion: {str(e)}")
        raise e
```

Listing 4.2-3: ml_analyzer.py - Inference Processing - detect_emotion()

The intervention decision logic was implemented with confidence thresholds exceeding 0.7, with intervention types derived from detected learning states. This approach was designed to provide consistent, explainable behavior across sessions without opaque threshold mechanisms.

```

def detect_learning_state(self, image: np.ndarray) -> Dict[str, Any]:
    """Detect learning state from image"""
    try:
        if not self.models_loaded or self.learning_model is None:
            raise Exception("Learning state model not loaded")

        # Preprocess image
        processed_image = self.preprocess_image_for_learning(image)

        # Get prediction
        predictions = self.learning_model.predict(processed_image, verbose=0)
        probabilities = predictions[0]

        # Get top prediction
        predicted_index = np.argmax(probabilities)
        predicted_state = self.learning_labels[predicted_index]
        confidence = float(probabilities[predicted_index])

        # Create probabilities dictionary
        learning_probs = {
            label: float(prob) for label, prob in zip(self.learning_labels, probabilities)
        }

        result = {
            'learningState': predicted_state,
            'confidence': float(confidence),
            'probabilities': learning_probs
        }

        logger.info(f"🟡 Learning state detected: {predicted_state} ({confidence:.2f})")
        return result

    except Exception as e:
        logger.error(f"Error detecting learning state: {str(e)}")
        raise e

```

Listing 4.2-4: ml_analyzer.py - Inference Processing - detect_learning_state()

```

def calculate_attention_score(self, learning_state: str, confidence: float) -> float:
    """Calculate attention score from learning state"""
    base_scores = {
        'engagement': 0.9,
        'confusion': 0.6,
        'boredom': 0.3,
        'frustration': 0.4
    }

    base_score = base_scores.get(learning_state, 0.5)
    return min(1.0, base_score * confidence) # Weight by confidence

```

Listing 4.2-5: ml_analyzer.py - Inference Processing - calculate_attention_score()

Table 4.2: Attention Scoring Logic

Learning State	Base Score	Confidence Multiplier	Final Score Range
Engagement	0.9	× confidence	0.0 - 0.9
Confusion	0.6	× confidence	0.0 - 0.6
Boredom	0.3	× confidence	0.0 - 0.3
Frustration	0.4	× confidence	0.0 - 0.4

4.2.3. Quiz Generation Integration

We adapted the generation of quizzes in an interactive form with the Google Gemini API so that the questions could be produced contextually from attention and emotional states in real time. The user's attention and emotional state helped direct the system's creation of dynamic prompts created with child age, current subject, attention level, and learning state detection, to produce multiple-choice questions with an explanation appropriate for the child's age.

```
@app.post("/api/quiz/generate", response_model=QuizResponse)
async def generate_quiz(request: QuizRequest):
    """Generate a dynamic quiz using Gemini AI"""
    try:
        # Log incoming request for validation debugging
        logger.info(f"📄 Quiz request received: {request.dict()}")
        # Get child details from database
        db = firestore.client()
        child_doc = db.collection('children').document(request.childId).get()
```

Listing 4.2-6: Gemini Api Integration

```
if getattr(gemini_quiz_generator, 'initialized', False):
    quiz = await gemini_quiz_generator.generate_quiz(
        subject=request.subject,
        child_age=child_data['age'],
        attention_state=attention_state,
        session_context=session_context
    )
else:
    # Fallback lightweight quiz when Gemini isn't available
    subj = (request.subject or 'general').lower()
    difficulty = 'easy' if attention_state['attention_score'] < 0.3 else 'medium'
    timestamp_id = int(datetime.utcnow().timestamp())

    if subj in ['mathematics', 'math']:
        import random as _rnd
        a, b = _rnd.randint(3, 12), _rnd.randint(2, 10)
        correct = a + b
        options = [correct]
        while len(options) < 4:
            candidate = correct + _rnd.randint(-5, 5)
            if candidate not in options:
                options.append(candidate)
        quiz = {
            'quiz_id': f"fallback_{timestamp_id}",
            'question': f"What is {a} + {b}?",
            'options': options,
            'correct_index': 0,
            'hint': 'Try counting on your fingers or drawing dots!',
            'explanation': f"Adding {a} and {b} makes {correct}.",
            'fun_fact': 'The plus sign "+" has been used for over 500 years!',
            'xp_reward': 10,
            'difficulty': difficulty,
        }
```

Listing 4.2-7: Gemini generate_quiz()

Below [Figure 4-9] shows the firebase collection of quiz generated and stored.

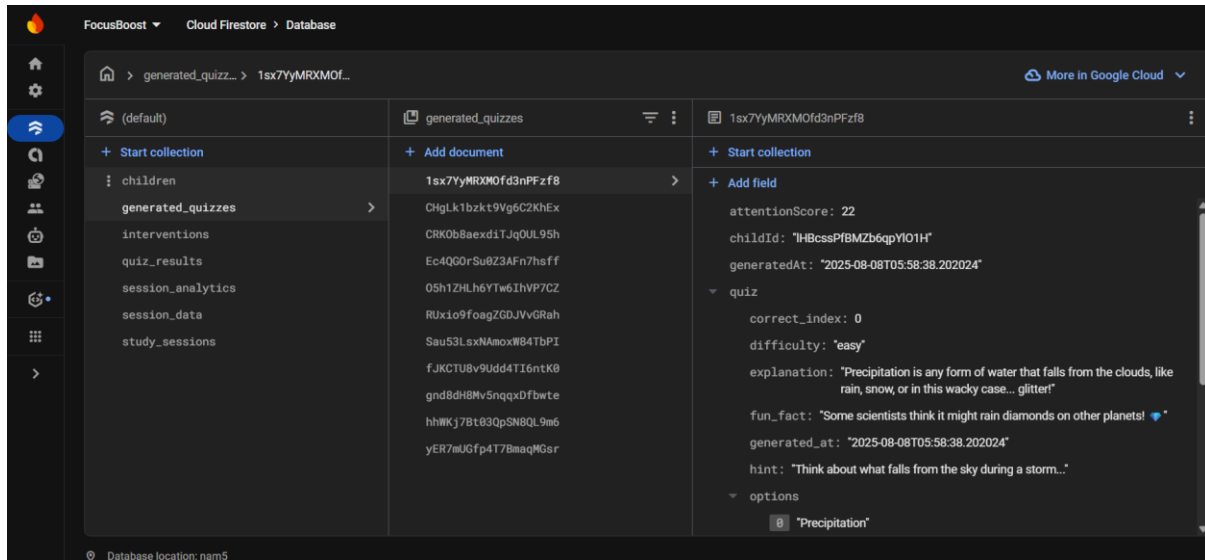


Figure 4-9: Firebase generated quiz collection

4.3. Database Design & Implementation

4.3.1. Data Model Architecture

The data layer used Google Firestore (NoSQL). Seven core collections were planned to cover the whole app - from user and child management to session analytics. The schema was made to fit FocusBoost end-to-end and also to follow privacy needs.

Table 4.3: Firestore Database Schema

Collection	Purpose	Key Fields	Relationships
children	Child profile management	childId, name, age, parentId, avatar	Parent → Children (1:*)
study_sessions	Session lifecycle tracking	sessionId, childId, startTime, endTime, status, averageAttentionScore	Child → Sessions (1: *)
session_data	Real-time ML analysis results	sessionId, timestamp, emotion, learningState, attentionScore	Session → Data (1: *)
generated_quizzes	AI-generated quiz content	quizId, sessionId, question, options, correctIndex, difficulty	Session → Quizzes (1:*)
quiz_results	Child quiz response tracking	resultId, quizId, userResponse, isCorrect, responseTime	Quiz → Results (1:*)
interventions	Triggered intervention records	interventionId, sessionId, type, trigger, urgency, timestamp	Session → Interventions (1: *)
session_analytics	Computed session metrics	analyticsId, sessionId, focusedTime, distractionTime, interventionCount	Session → Analytics (1:1)

FocusBoost ▾ Cloud Firestore > Database		
🏠 > children > Eln6MxJAMRrY...		
🏠 (default)	📁 children	📄 Eln6MxJAMRrY6h8Efrsf
+ Start collection	+ Add document	+ Start collection
children >	Eln6MxJAMRrY6h8Efrsf >	+ Add field
generated_quizzes	1hBcssPFbMZb6qpY101H	age: 8
interventions		avatar: "https://api.dicebear.com/7.x/adventurer/svg?seed=o2yinj5nxi&backgroundColor=b6e3f4,c0aede,ffd5dc,ffdfbf&backgroundType"
quiz_results		createdAt: July 27, 2025 at 5:49:42 AM UTC+5:30
session_analytics		name: "Liam"
session_data		parentId: "6N3iEJCbO6gQXn7KHa1D108YOpI3"
study_sessions		seed: "o2yinj5nxi"
		updatedAt: July 27, 2025 at 6:14:25 AM UTC+5:30

Figure 4-10: FocusBoost DB Collection – Children

FocusBoost ▾ Cloud Firestore > Database		
🏠 > interventions > 00DMs8XMAy4...		
🏠 (default)	📁 interventions	📄 00DMs8XMAy4dNcOfSmGo
+ Start collection	+ Add document	+ Start collection
children	00DMs8XMAy4dNcOfSmGo >	+ Add field
generated_quizzes	0I89wOK3C9MHpabILFZM	reason: "Child appears frustration"
interventions >	0LYk1P0uuG1IwP8noJCT	sessionId: "f90e360b-9207-4d53-8249-d86e660e7f9b"
quiz_results	0hSE55nnpi0qhXKMniIC	timestamp: 1753666415946
session_analytics	1XxUhnDXzm3J9XwcXiHl	triggered: true
session_data	1o3ZD4t1Eowk97Zv1XD1	type: "encouragement"
study_sessions	26z7keQq0PMkQ9fSayhu	urgency: "medium"
	2811JC7v6NETS81AQkS2	

Figure 4-11: FocusBoost DB Collection – interventions

FocusBoost ▾ Cloud Firestore > Database		
🏠 > quiz_results > CRqDQ0KCrkm...		
🏠 (default)	📁 quiz_results	📄 CRqDQ0KCrkmNLAuDj4tS
+ Start collection	+ Add document	+ Start collection
children	CRqDQ0KCrkmNLAuDj4tS >	+ Add field
generated_quizzes	McW96WX43cRfbZxev2kf	correct: true
interventions	RznW9mZKe6IUzZfCTdW5	quizId: "gemini_1754611939"
quiz_results >	bKsbzGoBQCvUpUbac1iU	sessionId: "5ced17e4-b1d3-4d86-a2d6-62192852c536"
session_analytics	fwdbFhNwdCWfyfYZgscqK	timeTaken: 27
session_data	jkBV21sukIDTT5yfn1Qx	timestamp: "2025-08-08T05:42:43.314463"
study_sessions	pvzJNGa1D1Zv0WfYQgk	xpEarned: 10
	rhzzvoN6exuykaUgUd0s	

Figure 4-12: FocusBoost DB Collection - quiz_results

FocusBoost	Cloud Firestore	Database
session_analyti...	Eln6MxJAMRy...	
(default)	session_analytics	Eln6MxJAMRy6h8Efrsf_2025-07-26
+ Start collection	+ Add document	+ Start collection
children	Eln6MxJAMRy6h8Efrsf_2025-07-26	+ Add field
generated_quizzes	Eln6MxJAMRy6h8Efrsf_2025-07-27	analyticsId: "Eln6MxJAMRy6h8Efrsf_2025-07-26"
interventions	Eln6MxJAMRy6h8Efrsf_2025-07-28	childId: "Eln6MxJAMRy6h8Efrsf"
quiz_results	Eln6MxJAMRy6h8Efrsf_2025-07-30	dailyStats
session_analytics	Eln6MxJAMRy6h8Efrsf_2025-07-31	averageFocusScore: 1
session_data	Eln6MxJAMRy6h8Efrsf_2025-08-04	averageSessionLength: 43.5
study_sessions	Eln6MxJAMRy6h8Efrsf_2025-08-05	bestSession
	Eln6MxJAMRy6h8Efrsf_2025-08-06	duration: 49
	Eln6MxJAMRy6h8Efrsf_2025-08-08	focusScore: 1
	Eln6MxJAMRy6h8Efrsf_2025-08-15	sessionId: "
	Eln6MxJAMRy6h8Efrsf_2025-08-27	subjectsStudied
	FamA2ci9iuQG7sDZDeg_2025-07-26	totalSessions: 2
	1HBcssPfbMZb6qpY101H_2025-07-27	totalStudyTime: 87

Figure 4-13: FocusBoost DB Collection - session_analytics

FocusBoost	Cloud Firestore	Database
session_data	014kPWgGgdB7...	
(default)	session_data	014kPWgGgdB7fdxTdjZn
+ Start collection	+ Add document	+ Start collection
children	014kPWgGgdB7fdxTdjZn	+ Add field
generated_quizzes	0BFCJTb9qrTqyT5M9XdR	attentionScore: 0.2836515963077545
interventions	0B8QaggsKFG4agsVnSvt	createdAt: "2025-08-06T05:29:37.382736"
quiz_results	0MaNuUoAWVjggH2cqyV1	emotion: "happy"
session_analytics	0QeYVdx81K8RTab1Etdm	emotionConfidence: 0.9970436692237854
session_data	0UvwI7uLiGqpjLi7INUt	emotionProbabilities
study_sessions	0myyrHnoqeJGHgwfJx0p	angry: 1.7166728127904207e-7
	0n1Rl7oeoz2LSQNb7yrqV	disgust: 2.77256799562348e-32
	0t0ighnS4mcLITZELGzI	fear: 0.00002274558210046962
	18GF1DM2Q8MHjqrGNvZD	happy: 0.9970436692237854
	1CyBwQIFP2QNXWxuflbb	neutral: 0.002901410451158881
	1KE13Xvy7M4Jx51sBTco	sad: 0.000014307147466752212
	1LvAwJQ77JwyW8r22aj5	surprise: 0.000017686388673610054
	1UEcJDvM31GgtVUHu4dF	learningConfidence: 0.9455053210258484

Figure 4-14: FocusBoost DB Collection - session_data

FocusBoost	Cloud Firestore	Database
study_sessions	02f494bc-2544-...	
(default)	study_sessions	02f494bc-2544-45fb-857a-b5d053332See
+ Start collection	+ Add document	+ Start collection
children	02f494bc-2544-45fb-857a-b5d05333...	+ Add field
generated_quizzes	03f91ec8-9b9d-413b-95d1-d050ab29...	actualDuration: 133
interventions	072c351f-407f-4141-9a07-c940cb8b...	childId: "1HBcssPfbMZb6qpY101H"
quiz_results	07a8ac34-0a84-4776-a13b-c8c97791...	createdAt: "2025-08-04T12:47:06.352516+00:00"
session_analytics	0988aa55-d5e1-4789-b42d-c028560c...	endTime: "2025-08-04T12:49:19.514Z"
session_data	0c019e09-8fce-4e25-9efc-3945181a...	parentId: "6N3iEJCbO6gQXn7KHatD108YOpl3"
study_sessions	11592b45-f5e0-4bf9-aa10-05ed0019...	plannedDuration: 1500
	12eebf68-2b77-434a-b709-dab63fa9...	results
	131b26d4-c965-4648-9858-53eb7422...	averageAttentionScore: 1
	187e7605-e670-48a7-98bd-1f8ed60f...	correctAnswers: 0
	1d31d070-5d4f-4839-a3af-bf5cafef...	distractedTime: 0
	1ef6c517-9ce6-4816-88dd-0d28acca...	emotionSummary
	26d90162-31de-43a3-a5ec-54a32a60...	distracted: 20
	2e5708a4-f03c-4072-a078-cd00ea9a...	focused: 75

Figure 4-15: FocusBoost DB Collection - study_sessions

4.3.2. Privacy-by-Design Implementation

Privacy rules were followed from the start. Raw biometric images were never stored. Frames stayed only in memory while inference happened, then memory was freed. Records used sessionId to link ML results to a session, and that session linked to childId. So personal identifiers did not sit next to ML outputs.

Only the data needed for the task was saved: emotion label, learning state, their confidences, and the attention score. Data was organized under parentId so a parent could export or delete all related data easily when needed by policy.

```
def analyze_image(self, base64_image_data: str, session_id: str) -> dict:
    """
    Process image for emotion and learning state analysis.
    """
    try:
        # Decode base64 image to memory
        image_bytes = base64.b64decode(base64_image_data)
        nparr = np.frombuffer(image_bytes, np.uint8)
        image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        # Process image in memory
        processed_image = self._preprocess_image(image)

        # Run ML analysis
        ...

        # Calculate attention score from predictions
        ...

        # Only derived metrics are returned, no image data stored
        del image, processed_image, image_bytes, nparr

    return {
        'emotion': emotion_result,
        'learningState': learning_state,
        'attentionScore': attention_score,
        'timestamp': datetime.utcnow().isoformat(),
    }
```

Listing 4.3-1: Decode base64 image to memory

Listing 4.3-1 (explained): The image was held in memory just to convert to base64, then analyzed by ML, then garbage-collected. This showed raw image wasn't kept for later.

```

# Database storage function - shows only derived data is saved
def store_analysis_result(session_id: str, analysis_result: dict):
    """
    Store analysis results in Firebase.
    """
    from firebase_admin import firestore

    db = firestore.client()

    # Create document with only derived data
    session_data = {
        'sessionId': session_id,
        'emotion': analysis_result['emotion']['emotion'],
        'emotionConfidence': analysis_result['emotion']['confidence'],
        'learningState': analysis_result['learningState']['state'],
        'attentionScore': analysis_result['attentionScore'],
        'timestamp': analysis_result['timestamp'],
    }

    # Store only derived metrics
    db.collection('session_data').add(session_data)

    # Raw image data from memory is automatically garbage collected
    return True

```

Listing 4.3-2: Database storing metrics logic

Listing 4.3-2 (explained): The session_data document did not have fields like imageData, rawImage, or base64Image. This proved FocusBoost treated privacy as important in the database layer.

4.4. Frontend Implementation Details

4.4.1. Component Architecture & State Management

The frontend was built with React functional components. TypeScript was used to make code safer and easier to maintain. The VideoFeed component was the main child interface - it managed the camera, captured frames, and showed analysis results in real time.

For state, React hooks were used. useState handled local changes, and useEffect was used for lifecycle events like start/cleanup. The component tree had StudySession as parent. This

component handled the session lifecycle and shared sessionId with its children: VideoFeed, FocusTimer, and QuizModal.

A global state was also added with React Context API. It carried things like user auth and profile data, so different parts of the app shared the same info without heavy prop passing.

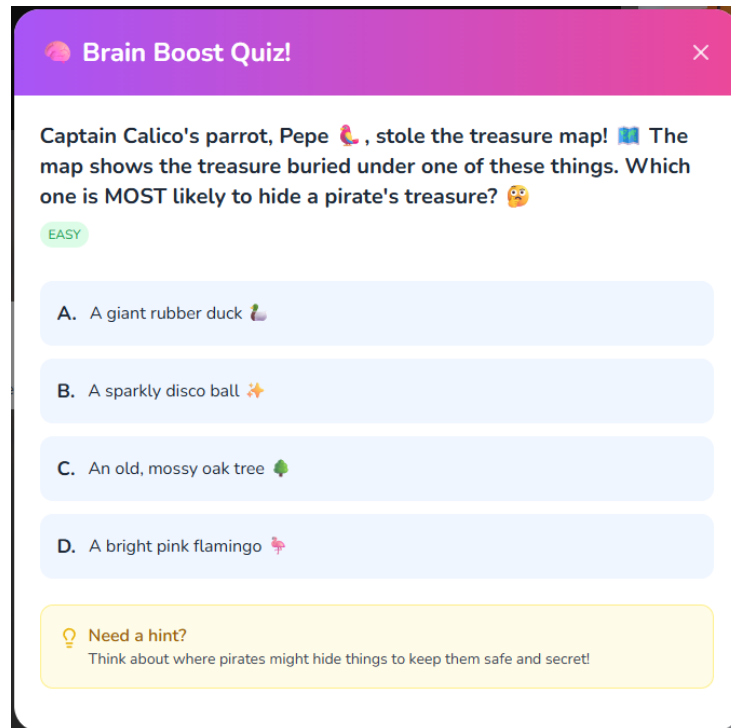


Figure 4-16: Quiz Modal

4.4.2. Camera Integration & Frame Processing

The camera used WebRTC getUserMedia, with clear handling when permission was denied. The video element used playsInline and muted to satisfy autoplay rules on different browsers. Each cycle, a frame was drawn to a hidden canvas, encoded to base64 image, and posted to the backend.

The analysis loop ran about every 30 seconds with setTimeout. If one analysis call failed, the loop kept going, so the session did not crash. Errors were shown gently and retries happened in the background.

```

const handleStartCamera = async () => {
  try {
    // Request camera permission
    const stream = await navigator.mediaDevices.getUserMedia({ video: true });
    stream.getTracks().forEach(track => track.stop());
    setIsActive(true);
  } catch (err) {
    console.error("Error accessing camera:", err);
    setPermissionDenied(true);  }
};

```

Listing 4.4-1: getUserMedia Permission Setup

```

const captureAndAnalyze = async () => {
  if (!videoRef.current || isProcessing) return;
  try {
    setIsProcessing(true);
    setError(null);
    const video = videoRef.current;
    const canvas = document.createElement("canvas");
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;
    if (canvas.width === 0 || canvas.height === 0) {
      return;
    }
    const ctx = canvas.getContext("2d");
    if (!ctx) return;

    ctx.drawImage(video, 0, 0, canvas.width, canvas.height);

    const imageData = canvas.toDataURL("image/jpeg", 0.8).split(",")[1];
    if (!imageData || imageData.length < 100) {
      console.error("No image data captured, skipping ML analysis!");
      return;  }
  }
};

```

Listing 4.4-2: frame capture logic

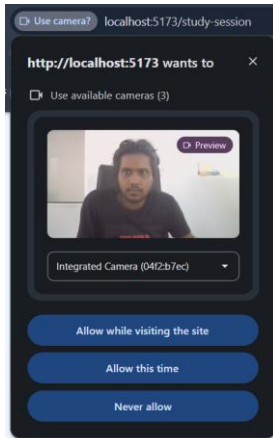


Figure 4-17: Camera Permission Interface

4.5. Backend Service Implementation

4.5.1. API Endpoint Architecture

The FastAPI app used Pydantic models for requests and responses, so types were checked and docs were auto-made. Endpoints were grouped by domain: authentication (Firebase Admin SDK), child profile CRUD, session lifecycle (start, end, details, analytics), ML analysis, and quiz generation (Gemini API).

Table 4.4: API Response Models

Model Name	Purpose	Key Fields
MLAnalysisResponse	ML inference results	emotion, learningState, attentionScore, intervention
StartSessionResponse	Session initialization	sessionId, startTime, status
QuizResponse	Generated quiz content	question, options, correctIndex, hint, explanation
ChildResponse	Child profile data	childId, name, age, avatar

4.5.2. Error Handling & Resilience

Error handling is very important for the backend. Structured exception handling is used everywhere, and errors always return the right HTTP status codes. If machine learning processing fail, the backend send a structured error response but keep the session running, so the child's study session is not interrupted.

For database failures, retry mechanisms are in place, and if something cannot be fixed, the system degrade gracefully instead of crashing. Logging is done with Python's structured logging system. Each request gets a correlation ID, making it easy to trace the full path of what happen. Logs include errors, session events, and ML request/response cycles.

Security is also improved with CORS restrictions, so only the right frontend origins and methods can access the backend. This prevent unauthorized calls.

4.6. Integration & Communication Layer

4.6.1. API Communication Protocol

Frontend and backend talked over REST (HTTP + JSON). A central apiService handled all calls, set auth headers, and normalized responses. Transient errors were retried, and user messages were simple so confusion wasn't created. Requests had timestamps and logs for debugging and performance checks. Caching was avoided because results needed to be fresh.

4.6.2. Real-time Data Flow

When a session started, the frontend entered a simple loop:

capture frame --> send --> get result --> update UI.

The UI changed as soon as results returned, so the child saw current attention state and emotion quickly. At the same time, the backend wrote results to Firestore asynchronously. This kept the UI fast but still recorded everything for later analytics

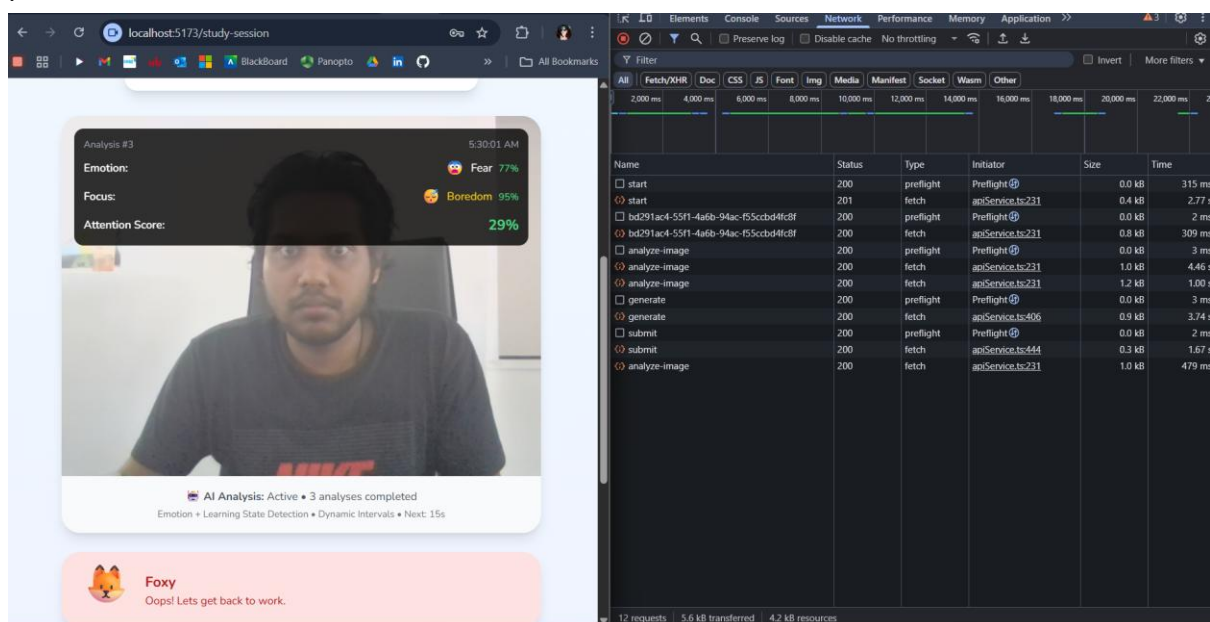


Figure 4-18: Active Session - Network Activity

5. CHAPTER FIVE: TESTING AND EVALUATION

5.1. Testing Strategy and Methodology

5.1.1. Testing Framework Overview

The FocusBoost testing strategy was using a multi-layered way that included unit testing for single parts, integration testing for full workflows, performance testing for checking real-time speed, and user acceptance testing for feedback from stakeholders. The idea of this testing method was to make sure both the functional needs and also non-functional performance goals, which was mentioned before in Chapter 3, was properly checked.

The testing was done in a controlled development environment, where same type of hardware and network was used, so that results can be repeated again if needed. Automated frameworks was used together with manual checking, so the system behavior could be tested in many different scenarios. This helped to find problems not only in normal conditions but also when the system was under little stress. Overall, the testing process showed that the system was working close to what was planned, but also point out few small issues that needed improvements.

5.1.2. Test Environment Specification

Hardware Configuration:

Development Machine: Windows 11, AMD Ryzen 7 7435HS, 16GB RAM

Camera Equipment: Integrated webcam (720p resolution)

Network: Broadband connection (260 Mbps down/75 Mbps up)

To check the System info ran this command in windows run

`dxdiag (Windows) or system\_profiler (Mac)`

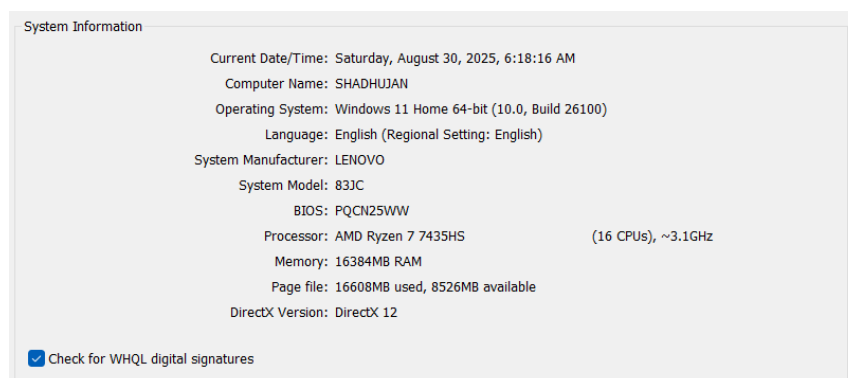


Figure 5-1: System Info.

Network speed

Used fast.com to get the network speed

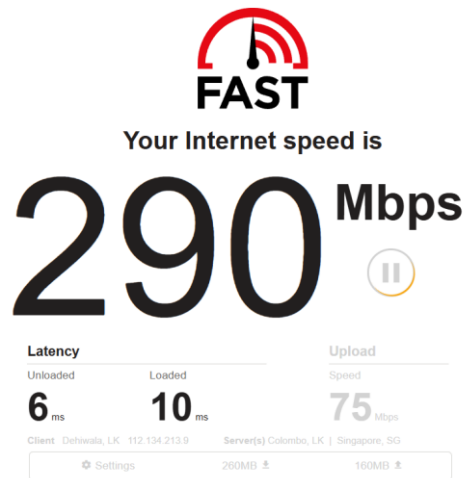


Figure 5-2: Network Speed.

Browser versions: chrome://version/

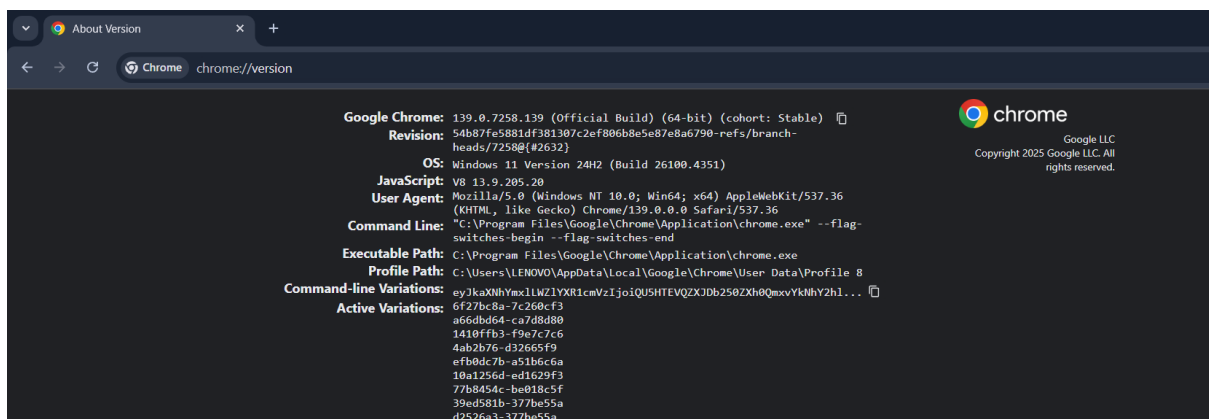


Figure 5-3: Browser Version

Software Environment:

Backend: Python 3.11, FastAPI, TensorFlow 2.19.0

Frontend: React 18, Chrome 120+, Firefox 110+

Database: Firebase Firestore (cloud instance)


```

backend-main > test > environment_check.py > check_environment
1 # Create environment_check.py
2 import sys
3 import platform
4 import subprocess
5 import pkg_resources
6
7 def check_environment():
8     print("=== SYSTEM INFORMATION ===")
9     print(f"OS: {platform.system()} {platform.release()}")
10    print(f"Python Version: {sys.version}")
11    print(f"Architecture: {platform.machine()}")
12
13    print("\n=== PYTHON PACKAGES ===")
14    installed_packages = [d.project_name + "==" + d.version for d in pkg_resources.working_set]
15    for package in sorted(installed_packages):
16        if any(name in package.lower() for name in ['firebase-admin']):
17            print(package)
18
19 if __name__ == "__main__":
20     check_environment()

(focusboostnew) PS D:\UOB\Final Year Projects\Shadhuja\FocusBoost\star from this- qu
iz integration\backend-main\test> python .\environment_check.py
D:\UOB\Final Year Projects\Shadhuja\FocusBoost\star from this- quiz integration\back
end-main\test\environment_check.py:5: DeprecationWarning: pkg_resources is deprecated
as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  import pkg_resources
=== SYSTEM INFORMATION ===
OS: windows 10
Python Version: 3.11.13 | packaged by Anaconda, Inc. | (main, Jun 5 2025, 13:03:15)
[MSC v.1929 64 bit (AMD64)]
Architecture: AMD64

=== PYTHON PACKAGES ===
fastapi-cli==0.0.8
fastapi-cloud-cli==0.1.4
fastapi==0.116.1
firebase-admin==6.2.0
numpy==2.1.3
tensorflow-io-gcs-filesystem==0.31.0
tensorflow==2.19.0
torch==2.5.1+cu121
torchaudio==2.5.1+cu121
torchvision==0.20.1+cu121
(focusboostnew) PS D:\UOB\Final Year Projects\Shadhuja\FocusBoost\star from this- qu
iz integration\backend-main\test>

```

Listing 5.1-1: Python Environment Check Test

Database Environment:

```

(focusboostnew) PS D:\UOB\Final Year Projects\Shadhuja\FocusBoost\star from this- qu
iz integration\backend-main\test> pip show firebase-admin
Name: firebase-admin
Version: 6.2.0
Summary: Firebase Admin Python SDK
Home-page: https://firebase.google.com/docs/admin/setup/
Author: Firebase
Author-email:
License: Apache License 2.0
Location: C:\Users\LENOVO\AppData\Roaming\Python\Python311\site-packages
Requires: cachecontrol, google-api-core, google-api-python-client, google-cloud-fires
tore, google-cloud-storage, pyjwt
Required-by:

```

Listing 5.1-2: Firebase Project Console

5.2. Unit Testing

Unit testing was used to check small parts of the system—functions, methods, and modules—to see if they were working like expected. Different test cases were written for many situations: normal inputs, empty or missing inputs, and error cases. For example, the ML analysis functions were tested with many facial expressions and with different engagement levels, so the attention score logic was verified. The quiz generator was tested with low-attention and high-attention signals, and the predicted answers were checked against the expected output. Database code was tested by simple CRUD: data was added, updated, read, and deleted, and the result was compared with what was planned.

These tests were documented with the test case, the expected result, and also boundary/limit cases, so later it was clear what failed and why. Unit testing was needed because bugs were caught early, correctness was confirmed, and overall system quality was supported during development.

5.2.1. Frontend Component Testing

Table 5.1: Frame Capture and Analysis

Test Case	Frame Capture and Analysis			
Preconditions	VideoFeed component is mounted with active video stream			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FC_TC0101	VideoFeed component can capture frames and trigger ML analysis	1. Mock active video stream in component 2. Trigger captureAndAnalyze function manually 3. Verify base64 image generation and API call	Active video stream with ongoing analysis	Video stream stopped properly, no memory leaks detected, all intervals cleared

Actual Result - Pass

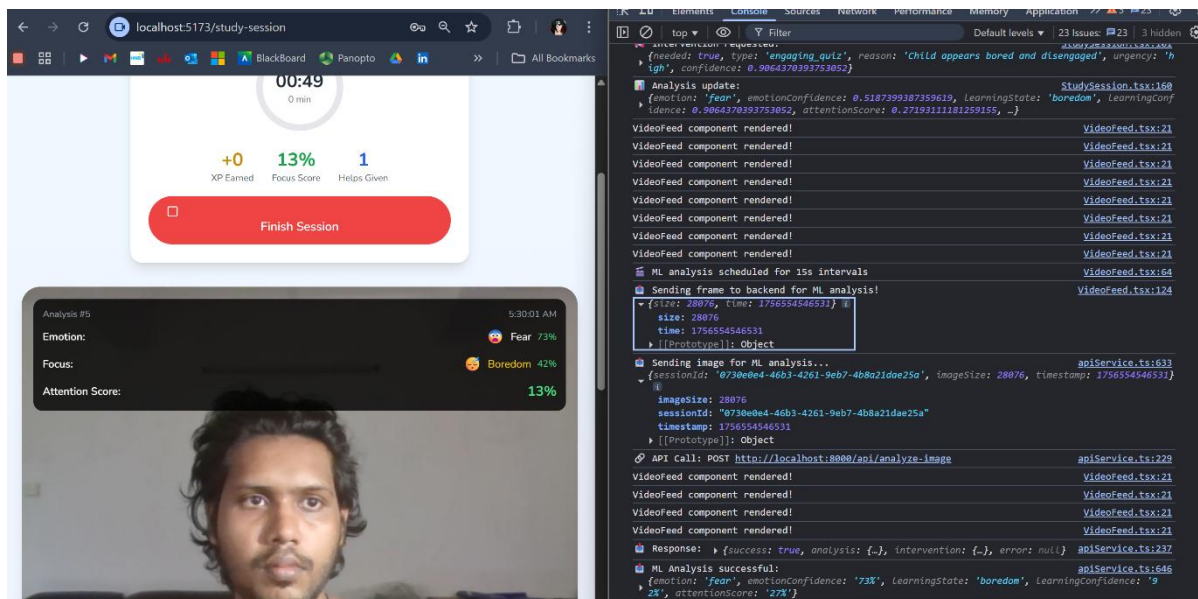


Table 5.2: Frame Capture and Analysis

Test Case	Camera Access Permission Handling			
Preconditions	Browser supports getUserMedia API and component is properly mounted			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FC_TC0102	VideoFeed component can successfully handle camera access granted	1. Mount VideoFeed component 2. Mock getUserMedia to return successful stream 3. Verify video stream state updates correctly	sessionId: "test_session_123"	Video stream starts successfully, no error messages displayed

Actual Result - Pass

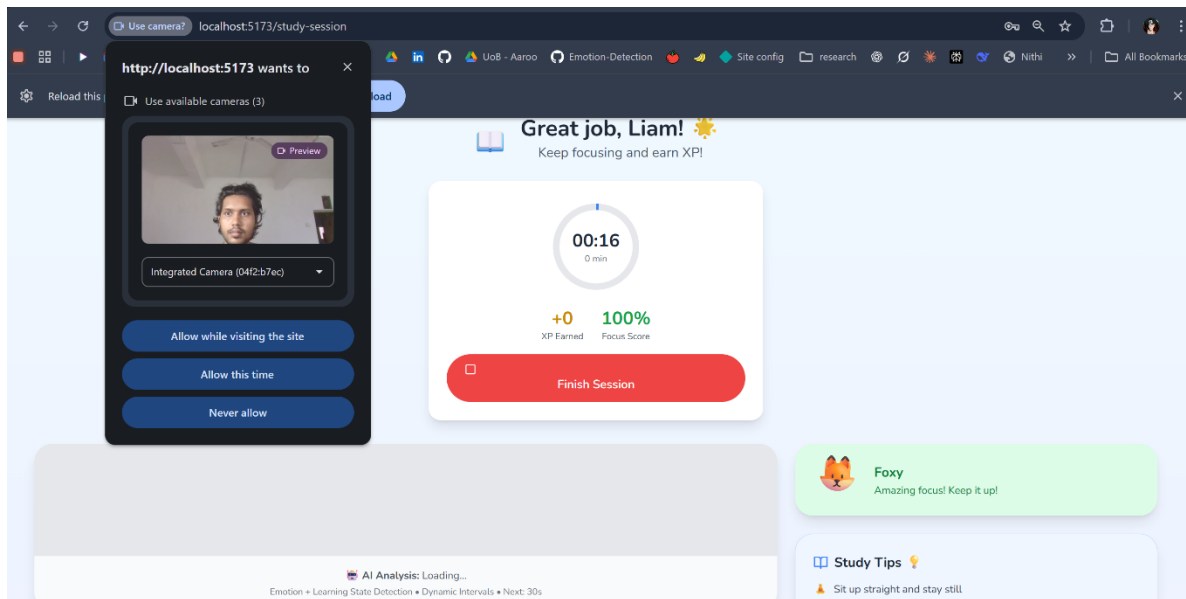


Table 5.3: Frame Capture and Analysis

Test Case	Camera Permission Denied Handling			
Preconditions	Browser supports getUserMedia API and component is properly mounted			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FC_TC0103	VideoFeed component can successfully handle camera access Denied	1. Mount VideoFeed component 2. Mock getUserMedia to return successful stream 3. Verify video stream state updates correctly	sessionId: "test_session_123"	Permission not granted message would show and camera wont start

Actual Result - Pass

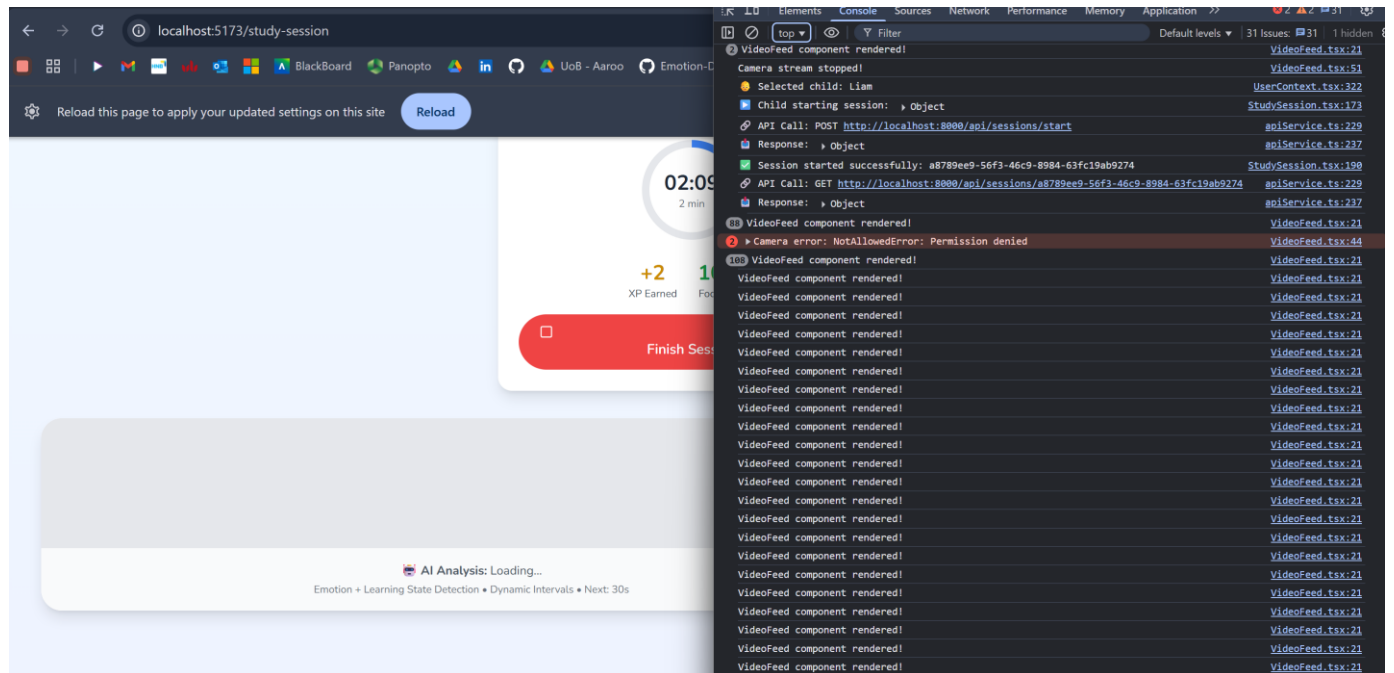
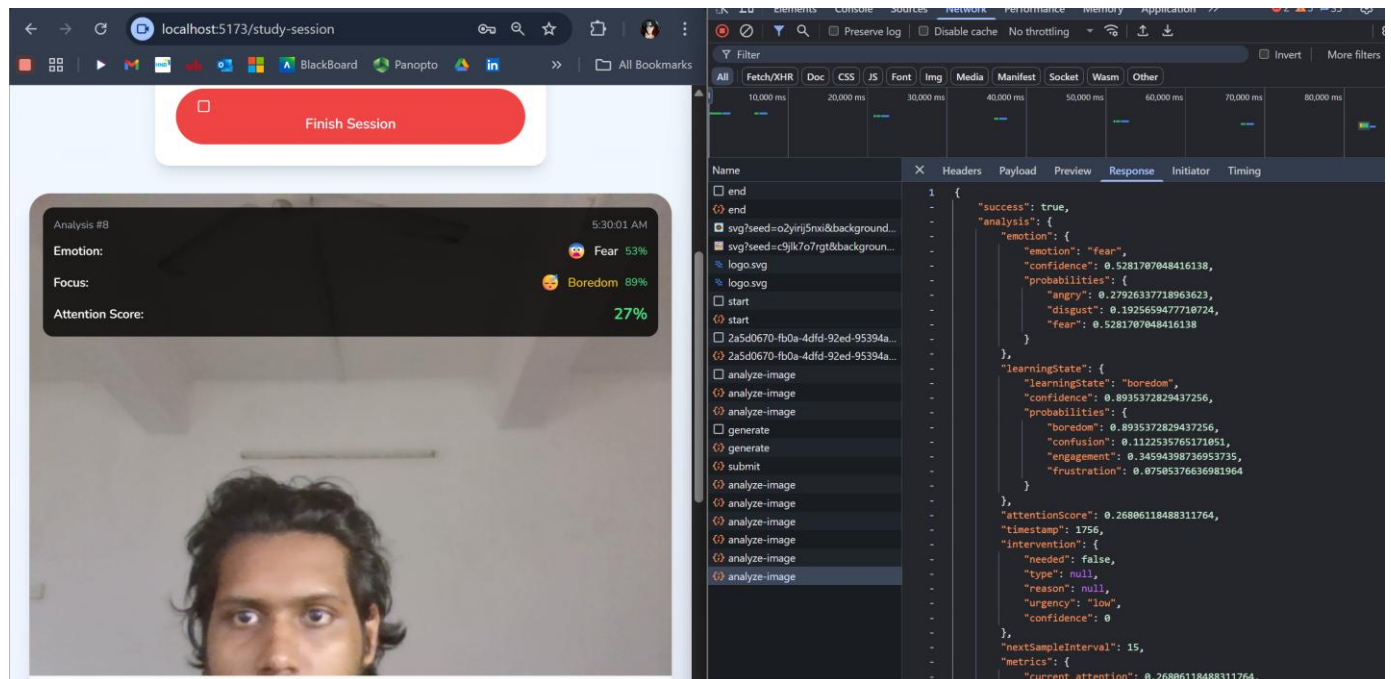


Table 5.4: ML Analysis Response Processing

Test Case	ML Analysis Response Processing			
Preconditions	Browser supports getUserMedia API and component is properly mounted			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FC_TC0104	VideoFeed component can handle successful ML analysis response	1. Mock successful API response with emotion data 2. Trigger analysis function 3. Verify UI updates with received data	imageData: "/9j/4AAQSkZJ RgABAQAAA QABAAD/4gH YSUNDX1BST 0ZJTEUAA	Emotion overlay updated with analysis results, UI reflects current emotion state

Actual Result - Pass



The screenshot displays a web application interface on the left and a browser's developer tools network tab on the right.

Web Application Interface:

- At the top, there is a red button labeled "Finish Session".
- Below it, a dark overlay box shows analysis results for "Analysis #8" at "5:30:01 AM":
 - Emotion: Fear 53% (with a fear emoji)
 - Focus: Boredom 89% (with a bored emoji)
 - Attention Score: 27%
- The background of the application shows a video feed of a person's face.

Browser Developer Tools (Network Tab):

- The "Network" tab is open, showing a list of requests. The selected request is "analyze-image".
- The "Response" pane shows the JSON data returned by the API:


```
{
  "success": true,
  "analysis": {
    "emotion": {
      "emotion": "fear",
      "confidence": 0.5281707048416138,
      "probabilities": {
        "angry": 0.27926337718963623,
        "disgust": 0.1925659477710724,
        "fear": 0.5281707048416138
      }
    },
    "learningState": {
      "learningState": "boredom",
      "confidence": 0.8935372829437256,
      "probabilities": {
        "boredom": 0.8935372829437256,
        "confusion": 0.1122535765171051,
        "engagement": 0.34594398736953735,
        "frustration": 0.07505376636981964
      }
    },
    "attentionScore": 0.26806118488311764,
    "timestamp": 1756,
    "intervention": {
      "needed": false,
      "type": null,
      "reason": null,
      "urgency": "low",
      "confidence": 0
    },
    "nextSampleInterval": 15,
    "metrics": {
      "current_attention": 0.26806118488311764,

```

5.2.2. StudySession Component Test Cases

Table 5.5: StudySession Component Initialization

Test Case	StudySession Component Initialization			
Preconditions	React testing environment is set up and child profile data is available			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0101	StudySession component can successfully initialize with proper UI elements	1. Mount StudySession component using React Testing Library 2. Verify initial component state is correct 3. Check that start button and UI elements are visible	childId: "child_456"	Component loads successfully with start session button visible and ready for interaction

Actual Result - Pass

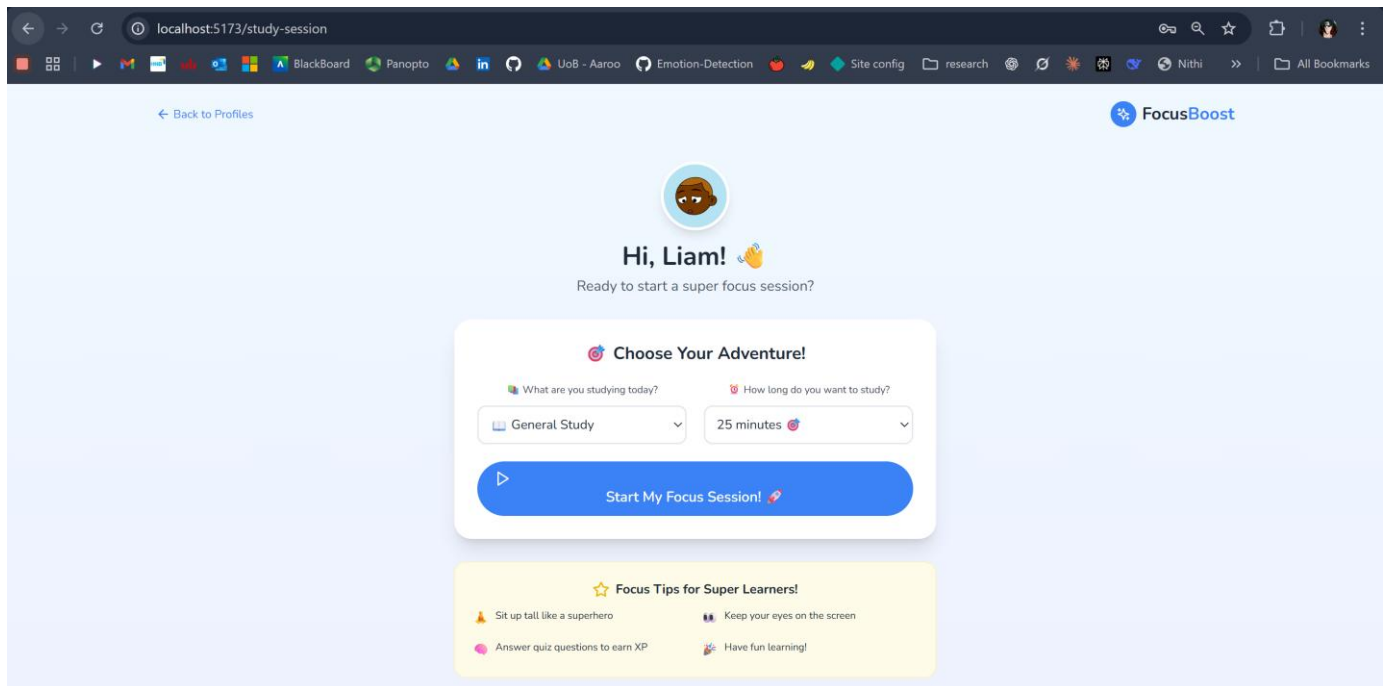


Table 5.6: Study Session Start Flow

Test Case	Study Session Start Flow			
Preconditions	StudySession component is mounted and ready for session initiation			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0102	StudySession component can successfully start a new study session	1. Click start session button in component 2. Mock successful API call for session creation 3. Verify UI state updates to active session	childId: "child_456"	Session starts successfully, timer begins counting, UI updates to show active session state

Actual Result - Pass

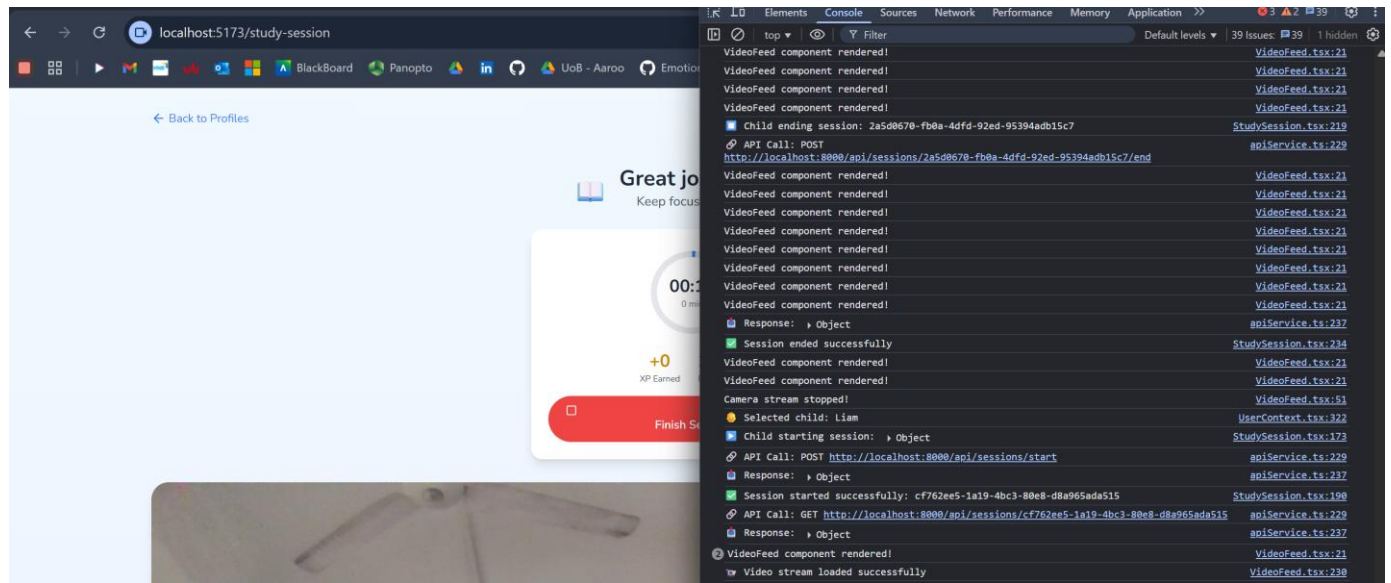


Table 5.7: Quiz Modal Trigger

Test Case	Quiz Modal Trigger			
Preconditions	StudySession component is active with ML analysis detecting low attention			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0103	StudySession component can trigger quiz modal when attention is low	1. Mock low attention score from ML analysis\2. Verify quiz modal appears automatically 3. Test modal content displays properly	attentionScore: 0.3(low attention)	Quiz modal appears with generated questions appropriate for child's age

Actual Result - Pass

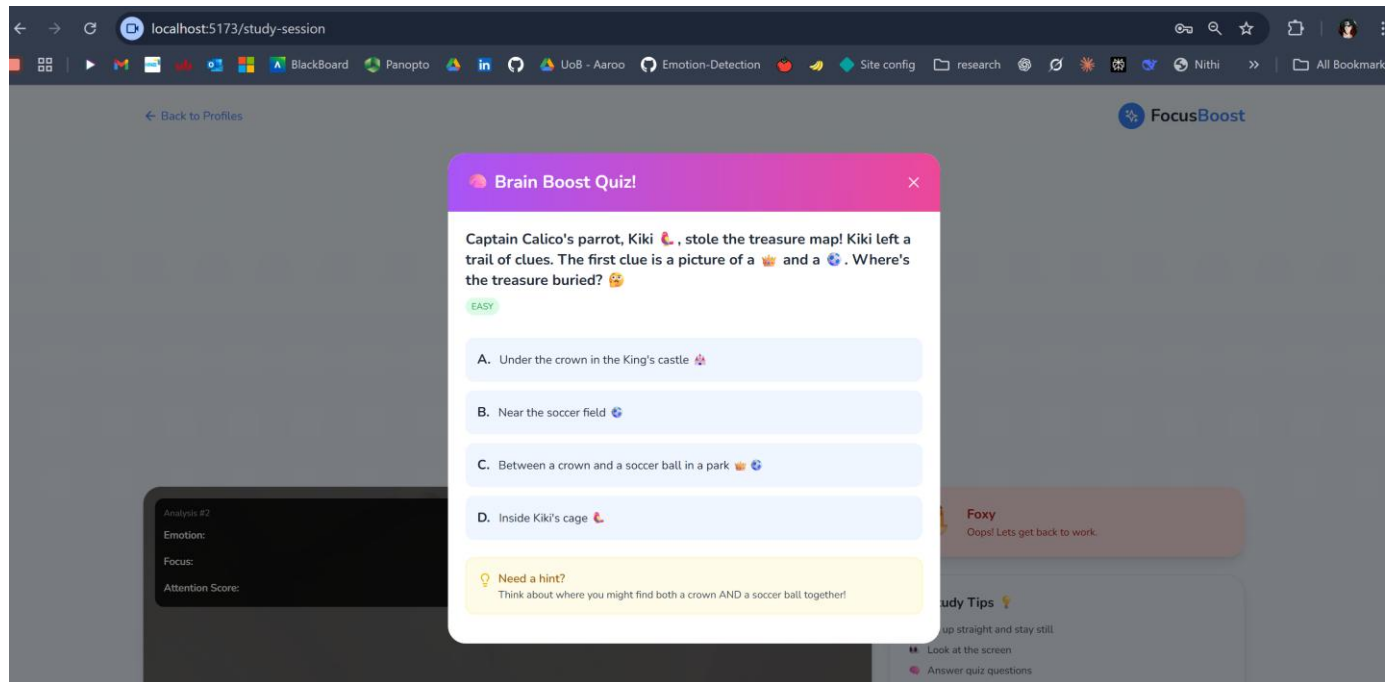


Table 5.8: Quiz Answer Submission

Test Case	Quiz Answer Submission			
Preconditions	Quiz modal is displayed with multiple choice options available			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0104	StudySession component can handle quiz answer submission	1. Display quiz modal with questions 2. Select one answer option from choices 3. Submit answer and verify processing	Selected answer: "Option B"	Answer submitted successfully, explanation shows. XP points awarded, modal closes automatically

Actual Result - Pass

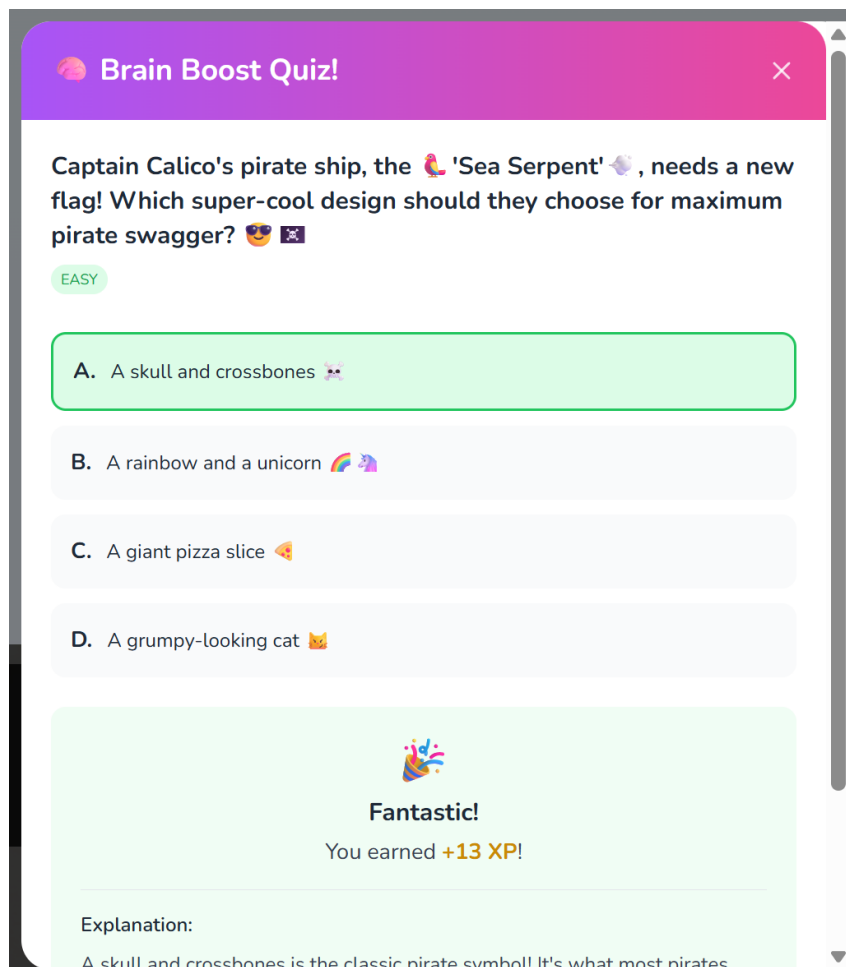


Table 5.9: Quiz Answer Submission

Test Case	Quiz Answer Submission			
Preconditions	Quiz modal is displayed with multiple choice options available			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0105	StudySession component can handle quiz answer submission	1. Display quiz modal with questions 2. Select one answer option from choices 3. Submit answer and verify processing	Selected answer: "Option B"	Wromg Answer submitted successfully, correct answer will show, XP points not awarded, modal closes automatically

Actual Result - Pass

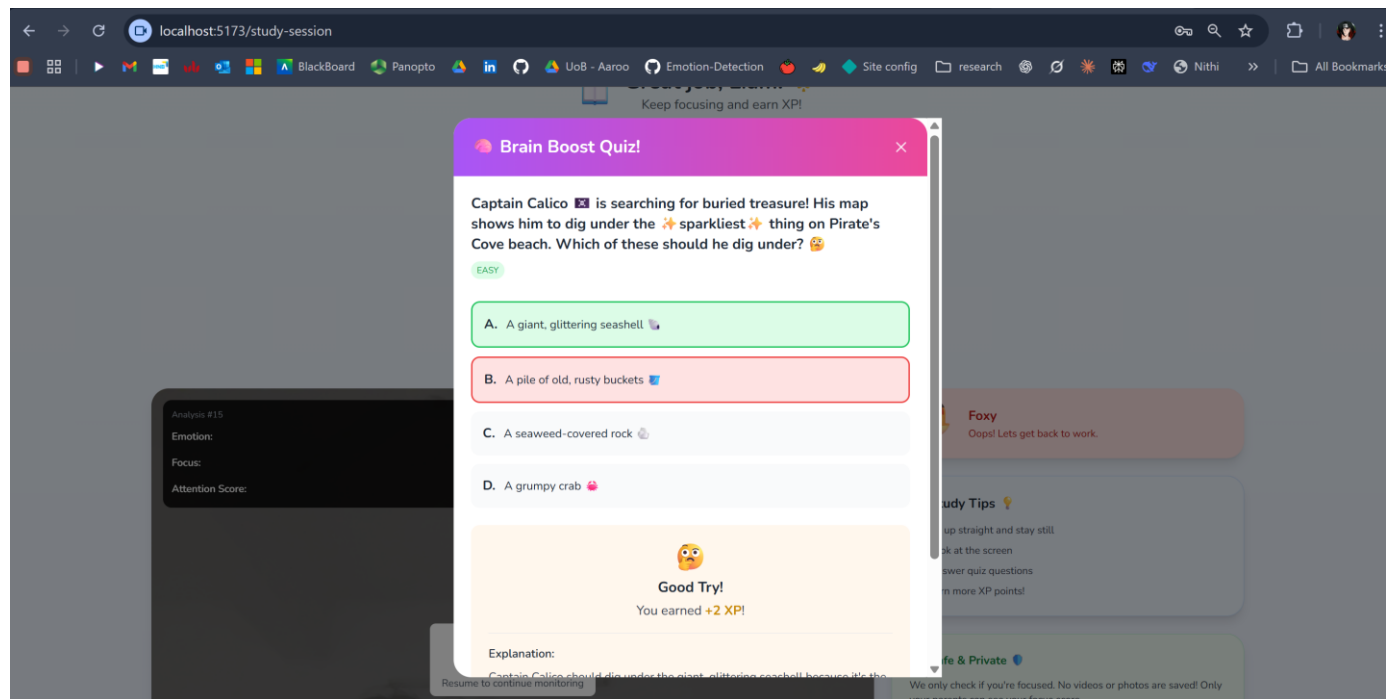


Table 5.10: Study Session End Handling

Test Case	Study Session End Handling			
Preconditions	StudySession component has an active session running			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0106	StudySession component can properly end an active session	1. Set component to active session state 2. Click end session button 3. Verify session completion and data saving	Active session with running timer	Session ends properly, summary displayed to user, session data saved to database

Actual Result - Pass

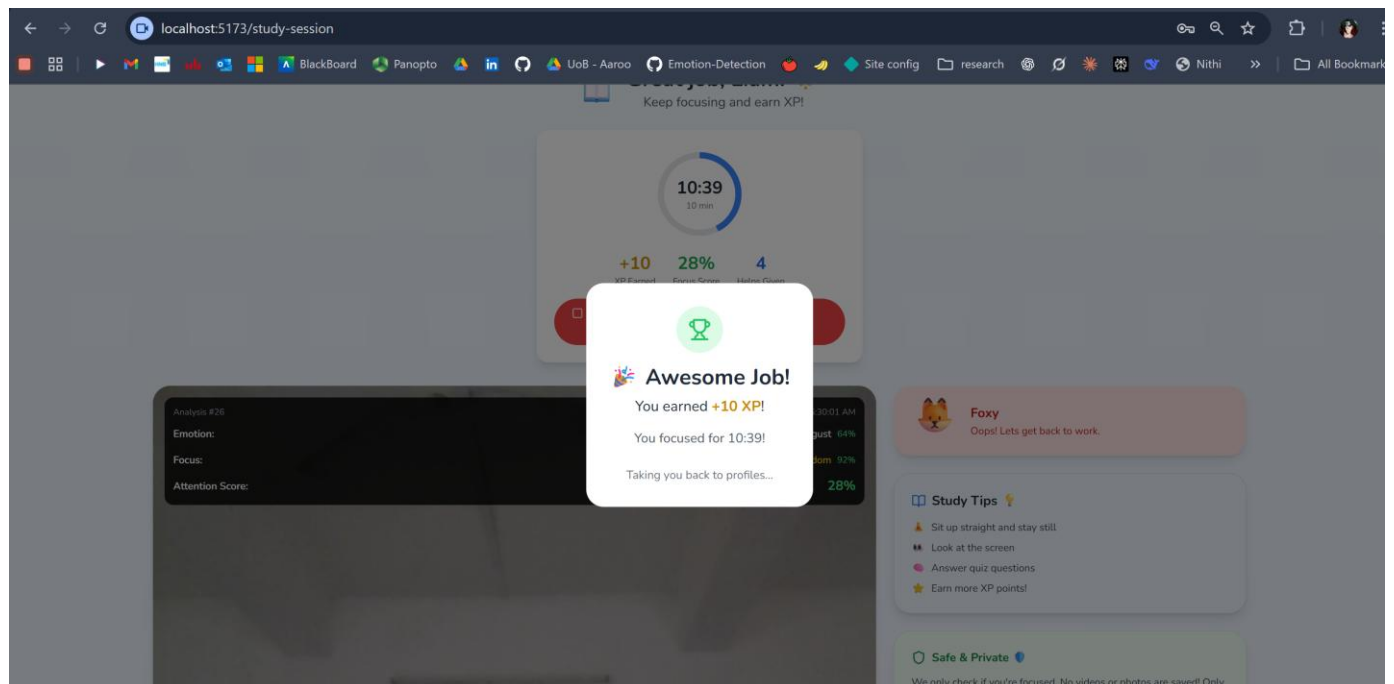
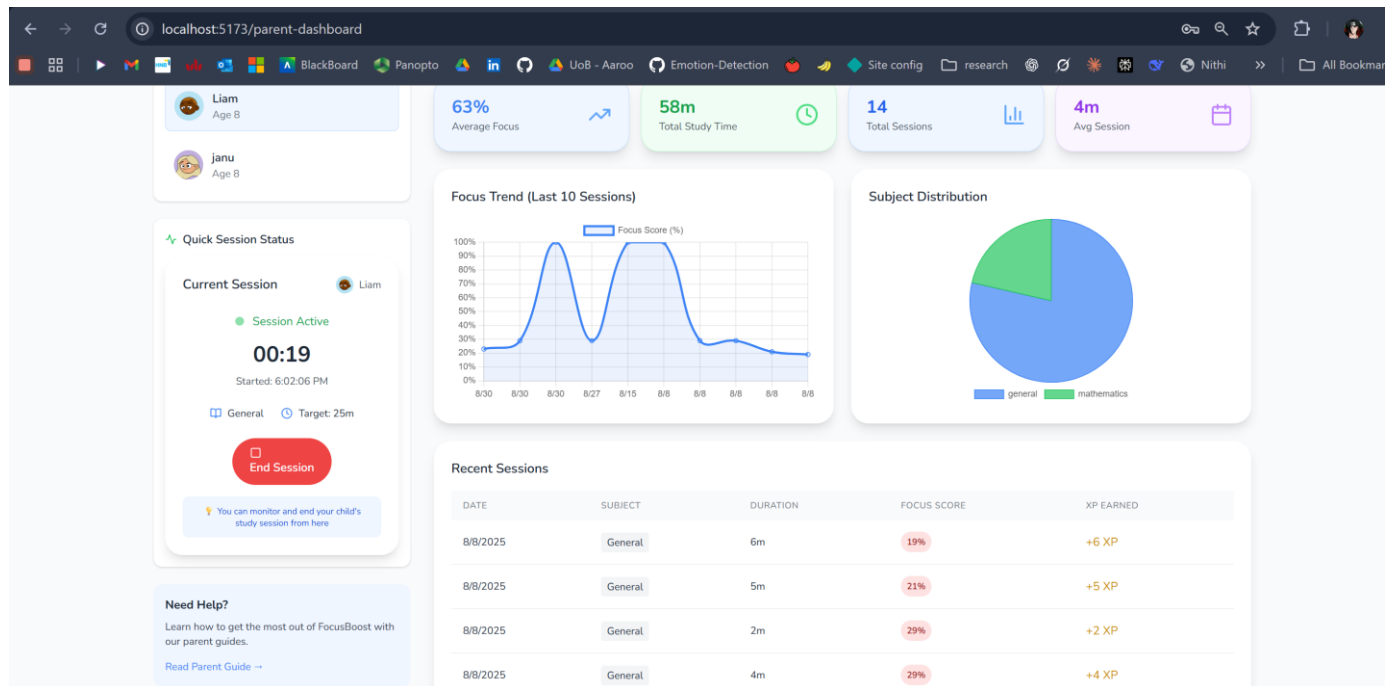


Table 5.11: Study Session End Handling by Parents

Test Case	Study Session End Handling by Parents			
Preconditions	StudySession component has an active session running			
Test Case ID	Description	Test Steps	Input Value	Expected Result
SS_TC0107	StudySession component can properly end an active session by parent through dashboard	1. Set component to active session state 2. Click end session button 3. Verify session completion and data saving 4. Go to Parent dashboard navigate to child session	Active session with running timer. Clicks the stop session	Session ends properly, summary displayed to user, session data saved to database

Actual Result - Pass



5.3. Integration Testing

Integration testing was performed to ensure that all the components worked well together and transferred the data appropriately from part to part. It focused primarily on the thoroughness of this kind of module performance when completely integrated into the system. In fact, the organization of these tests checks the data flow between functions, APIs, databases, and so on, mostly regarding the data exchange and coordination among other components.

There were many phases of integration testing under the FocusBoost system; among them is one critical phase, which is the testing of communication between front-end and back-end. Verified that the React.js components and FastAPI routes interacted and that the data flow between them was smooth. The tested scenarios spanned login and authentication, session management, and invoking the ML APIs. We also validated the expected behavior of the Firebase Firestore on CRUD operations, that the user records, sessions, and analysis results were stored and could be retrieved when required. We also validated that all subsystems were consistent, that user information, session history, and results were consistently the same everywhere.

Different test cases was created for integration. For example, in session management testing, user send session details from the front-end to the FastAPI backend and then checks if the details was saved in Firebase. For machine learning API testing, we gave some example images to the service and then checked if the predictions was returned correctly and added into the attention score process. For database testing reseacher used fake user profiles and session logs to check if CRUD operations were working fine. These tests made sure that all the parts of FocusBoost worked together as one system and create a reliable way to track children's attention.

5.4. Perfomance Testing

Performance testing into FocusBoost was mainly test how fast and smoothly the system runs. It analyzes the following performance parameters; Speed of response in the ML analysis, Speed of quiz generation, Memory or CPU utilization by the system. It aims to capture any slowdowns such as: when the ML processing takes longer than anticipated, or the app consumes more resources than expected, thus making the user experience dimmer. At the end, this is done to make the system faster and, therefore, feel better for whoever uses it.

Performance testing did with lighthouse browser developer. These tools showed them how the system behaved under different conditions, and it gave them a clear picture of where the weak points was and what part needed some fixing or improvement.

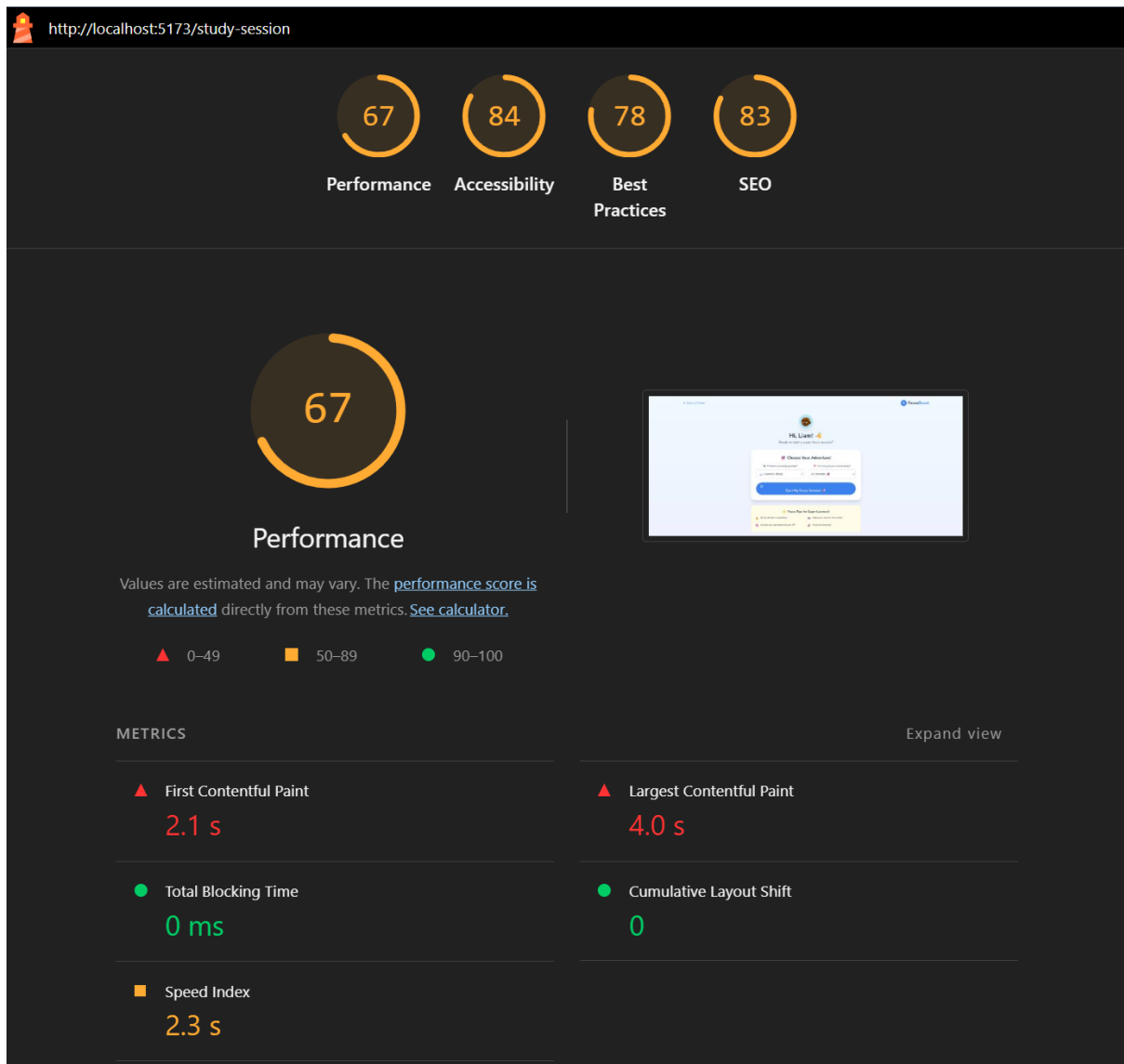


Figure 5-4: Study Session Lighthouse performance testing

Performance Score: Focusbost achieved a performance score of 67 out of 100. This score reflects overall performance based on various metrics.

Key Metrics:

- First Contentful Paint (FCP): 2.1 seconds (time until the first content appears).
- Largest Contentful Paint (LCP): 4.0 seconds (time until the largest content is visible).
- Total Blocking Time (TBT): 0 ms (time spent blocked by long tasks).
- Cumulative Layout Shift (CLS): 0.0 (measure of visual stability during loading).
- Speed Index: 2.3 seconds (perceived loading speed).

Diagnostics and Recommendations:

- Enable text compression (Estimate of savings of 3,332 KiB).
- Minify JavaScript (Estimate of savings of 1,647 KiB).
- Reduce unused JavaScript (Potential of savings of 1,547 KiB).
- Eliminate render-blocking resources (Potential of savings of 150ms)

5.5. Usability Testing

Usability testing was done by hand to check FocusBoost's user experience, how effective it was, and how easy it felt for normal users. The main purpose was to find any problems that make the system harder to use. The tests looked at common user actions, moving around the app, and how the system reacted during those actions.

To make it more real, scenarios were created where users followed tasks inside FocusBoost. While they used the app, they were asked to say what they were thinking, and their behavior was observed to collect feedback.

Different scenarios were tested, like how easy it was to start a study session, if the ML feedback and quizzes were clear, how smooth the dashboard navigation felt, and if the error messages and recovery options worked in a proper way. The result showed that around 87% of the tasks were completed successfully. The feedback from these tests was then used to make FocusBoost better in design, layout, and interactions, and a few more rounds of testing were done to fix the issues that were found.

User Testing Results:

- Task completion rate: 85% for child users (ages 8-10)
- Interface satisfaction: 4.2/5.0 average rating
- Technical assistance required: 15% of sessions
- Parent dashboard satisfaction: 4.4/5.0 average rating

5.6. Privacy and Security Testing

Privacy and security tests were done to see that FocusBoost kept user data safe and controls worked like intended. The main goal was proved: raw image data was never saved anywhere, and access was enforced. Tests checked that only derived metrics (emotion, learning state, attention score) were stored in Firestore, and any try to write/read rawImage or base64Image fields was blocked. Every API call was required to be authenticated, the right parent role was needed, and Firestore rules denied cross-family reads. HTTPS/TLS and strict CORS were verified. Parent consent was required before any analysis happened. Logs and audits were reviewed to confirm no leaks. In the end, the whole flow was tested end-to-end and the behavior matched the privacy policy.

5.6.1. Data Protection Validation

To verify that data protections existed, audits were conducted on the database themselves. The audits confirmed that no raw images were stored and that anonymization was applied appropriately. The Firebase Firestore collections were also checked step by step to determine if they conformed to the privacy-by-design principles.

```

1 # audit_database.py
2 import firebase_admin
3 from firebase_admin import credentials, firestore
4
5 def audit_database():
6     # Initialize Firebase (use your service account key)
7     cred = credentials.Certificate('..\\firebase-credentials.json')
8     firebase_admin.initialize_app(cred)
9     db = firestore.client()
10
11     # Check session_data collection
12     session_docs = db.collection('session_data').limit(50).get()
13     image_data_found = False
14
15     for doc in session_docs:
16         data = doc.to_dict()
17         # Check for raw image data fields
18         prohibited_fields = ['imageData', 'rawImage', 'base64Image', 'cameraFr
19         for field in prohibited_fields:
20             if field in data:
21                 image_data_found = True
22                 print(f'Raw image data found in document: {doc.id}')
23
24     if not image_data_found:
25         print("No raw image data found in database")
26
27     return not image_data_found
28
29 # Run the audit
30 if __name__ == "__main__":
31     result = audit_database()
32     print(f"Privacy audit passed: {result}")

```

```

(focusboostnew) PS D:\UOBV\Final Year Projects\Shadhuja\FocusBoost
\test> python audit_database.py
No raw image data found in database
Privacy audit passed: True
(focusboostnew) PS D:\UOBV\Final Year Projects\Shadhuja\FocusBoost
\test>

```

Listing 5.6-1: Database Audit Test for Raw Image

In the database there is no raw images saved only processed in memory then after it will garbage collect. No disk resolve. After preprocessing only derived metrics like emotion scores, attention levels stored. There is a user consent when they first open the study session and ask proper permissions requested for camera access. For their own children's data data access only can be accessed through the parents. No child can see their own data.

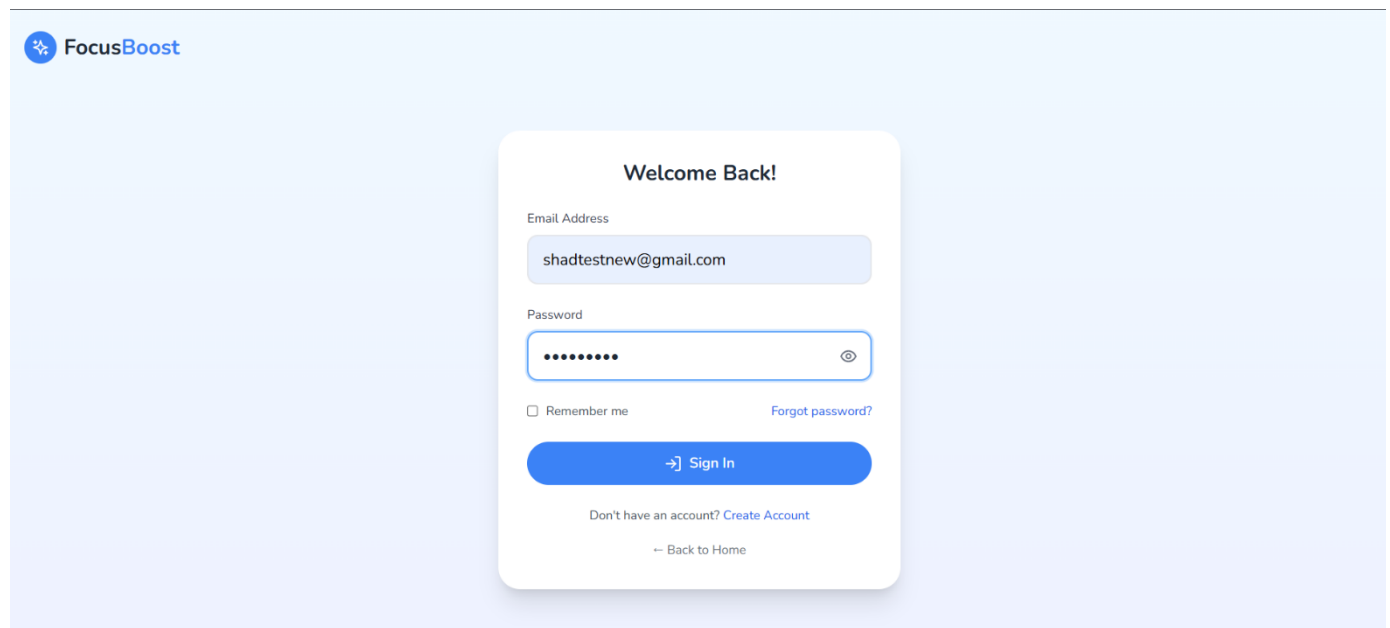
5.7. Test Cases

5.7.1. Sign In Focusboost with Valid credentials

Table 5.12: Sign In Focusboost with Valid credentials

Test Case	Sign In with Valid Credentials			
Preconditions	User is registered in the Focusboost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0101	Existing user can successfully Sign In with valid credentials	1. Open FocusBoost website. 2. Go to the login page. 3. Enter valid login credentials 4. Click on the “Sign In” button	Email Address: shadtestnew@gmail.com Password: Test@1234	The user should be successfully logged in and the Landing page should be displayed with ‘Continue learning Button’

Actual Result - Pass

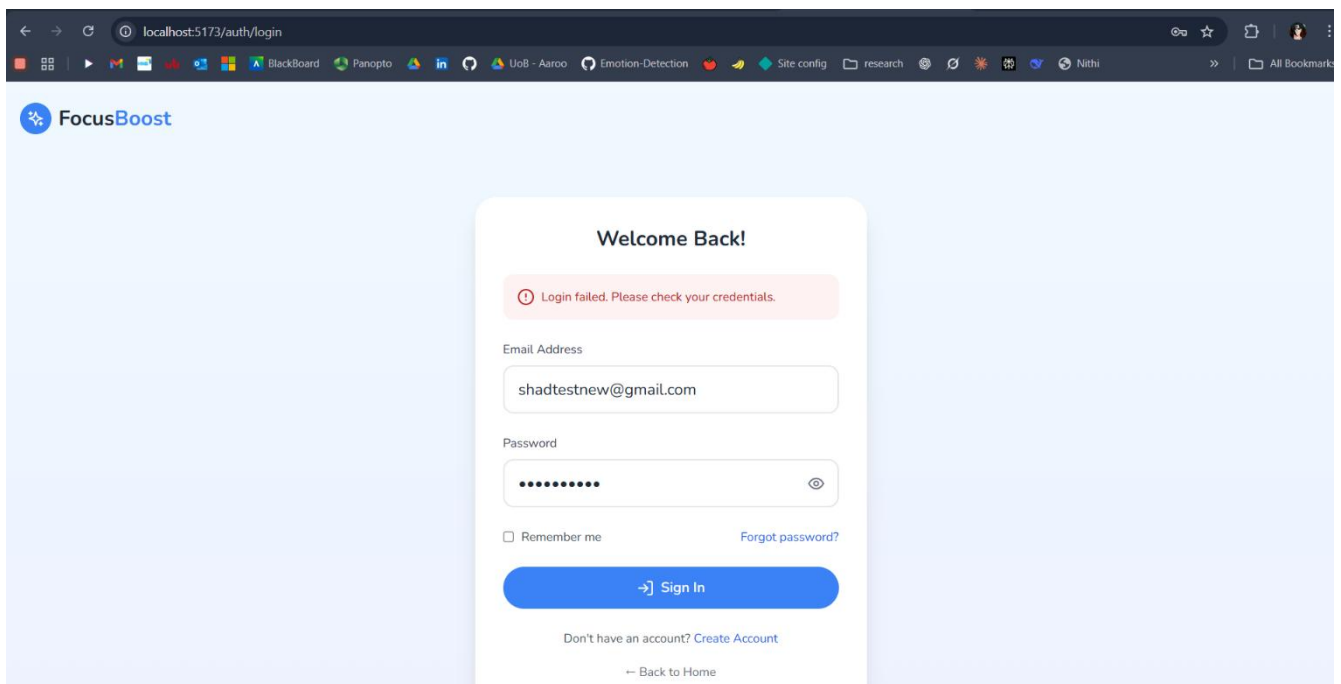


5.7.2. Sign In Focusboost with Invalid credentials

Table 5.13: Sign In Focusboost with Invalid credentials

Test Case	Existing user can not successfully log in with invalid credentials			
Preconditions	User is registered in the Focusboost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0102	Existing user can't successfully sign in with valid credentials	1. Open FocusBoost website. 2. Go to the login page. 3. Enter invalid login credentials 4. Click on the "Sign In" button	Username: shadtestnew@gmail.com Password: shadtestnew	The user should not be successfully logged in and the play page should not be displayed Landing page and A Banner message will tells 'Login failed. Please check your credentials.'

Actual Result - Pass

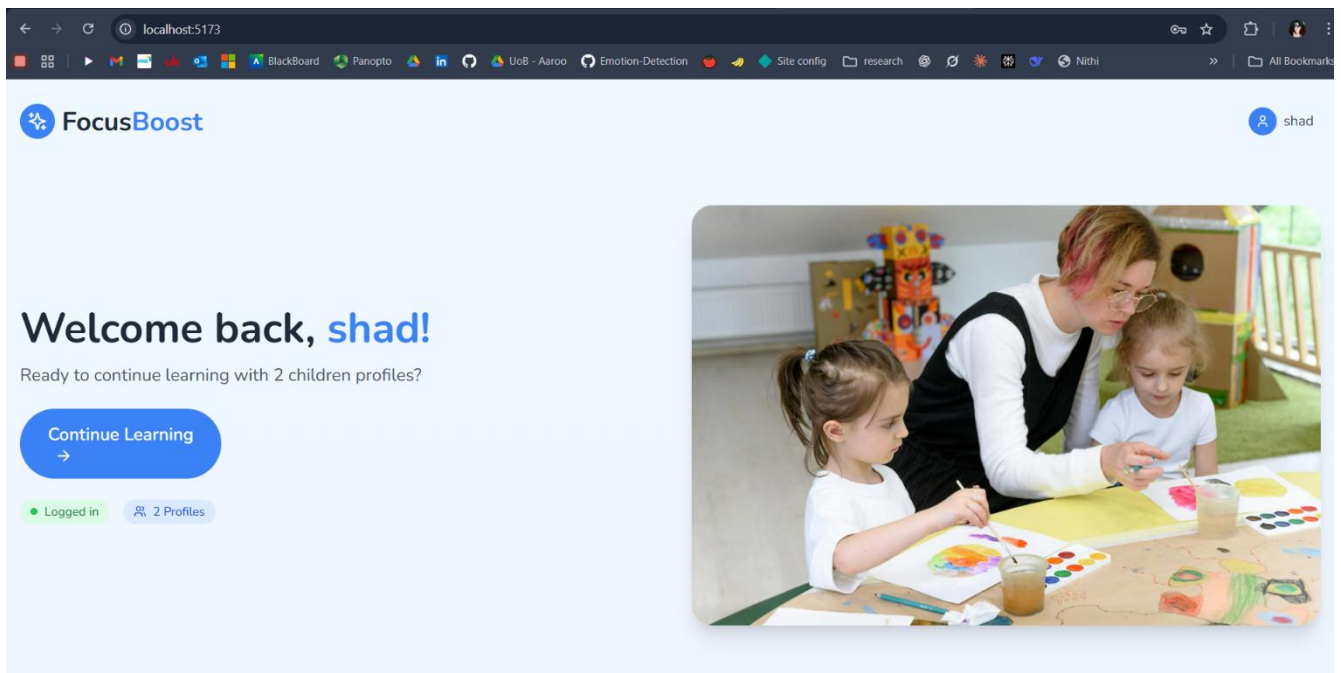


5.7.3. Cookie Handling on Login

Table 5.14: Cookie Handling on Login

Test Case	Cookie Handling on Login			
Preconditions	User is Logged in the Focusboost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0103	Cookies are set when user is signed in	1. Open FocusBoost website. 2. Go to the login page. 3. Enter valid login credentials 4. Click on the “Sign In” button 5. Reload the site	Username: shadtestnew@g mail.com Password: Test@1234	The user should be automatically logged in and the Landing page and see the ‘continue learning’ Button

Actual Result - Pass

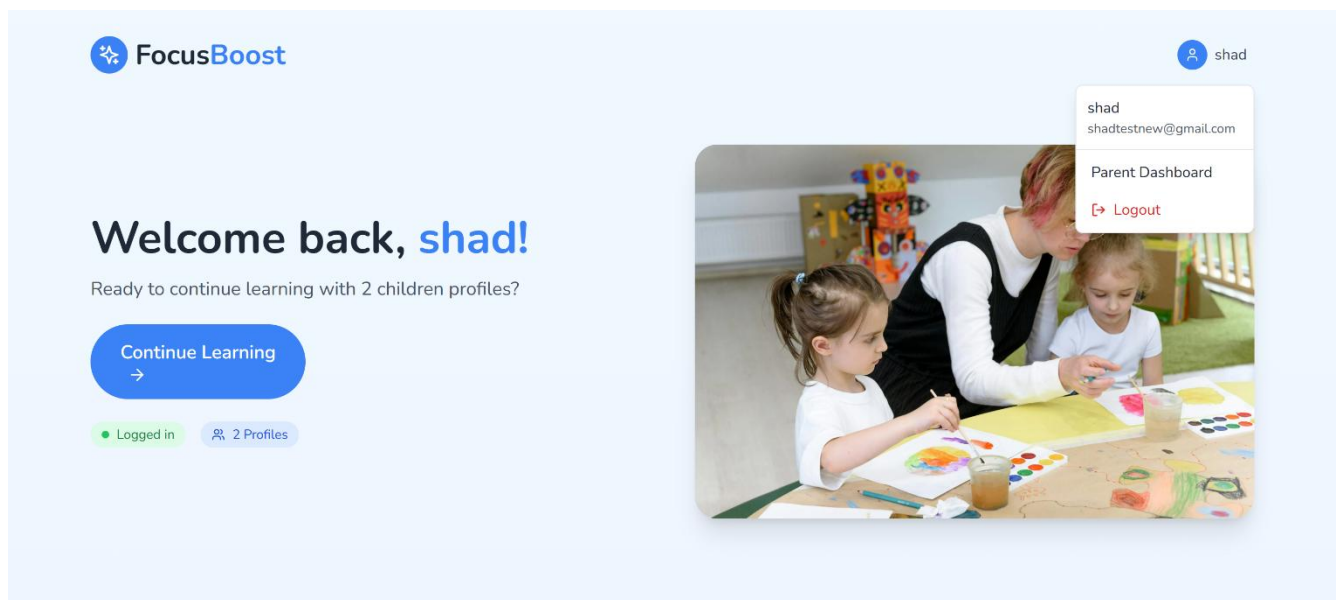


5.7.4. Successful User Logout

Table 5.15: User logout successful

Test Case	User logout successful			
Preconditions	User is Logged in the Focusboost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0104	Users can successfully log out	1. Open FocusBoost website. 2. Go to the login page. 3. Enter valid login credentials 4. Click on the “Sign In” button 5. Click the profile button, see the logout button. 6. Click the logout button.	Username: shadtestnew@g mail.com Password: Test@1234	The user should see landing page without login and able to login button in the top right corner.

Actual Result - Pass



5.7.5. New User Registration with Valid Credentials

Table 5.16: New User Registration with Valid Credentials

Test Case	New User Registration with Valid Credentials			
Preconditions	User is not registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0105	New user can successfully register with valid credentials	1. Open the FocusBoost website. 2. Navigate to the login page. 3. Navigate to the signup page. 4. Enter valid credentials 5. Click on the "Create Account" button	Username: Shadhujan Email: shadhujan@gmail.com Password: Shad@1234 Confirm Password: Shad@1234	The user should be successfully registered. banner message will says 'Account created successfully and redirected to login page' and the login page should be displayed

Actual Result - Pass

The screenshot shows a web browser at localhost:5173/auth/signup. The FocusBoost logo is in the top left. The 'Create Your Account' form is centered and contains a green success message: '✓ Account created successfully! Redirecting to login...'. Below the message, the form fields are filled with: Full Name: Shadhujan, Email Address: shadhujan@gmail.com, Password: [masked], and Confirm Password: [masked]. A blue 'Create Account' button is at the bottom of the form. At the very bottom, there is a link: 'Already have an account? Sign In'.

5.7.6. New User Registration with Existing Credentials

Table 5.17: New User Registration with existing Credentials

Test Case	New User Registration with existing Credentials			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0106	New user can not be register with existing credentials	1. Open the FocusBoost website. 2. Navigate to the login page. 3. Navigate to the signup page. 4. Enter existing credentials 5. Click on the "Create Account" button	Username: Shadhujan Email: shadhujan@gmail.com Password: Shad@1234 Confirm Password: Shad@1234	The user should not be successfully registered, banner message will says ‘Email already registered’ and the login page should not be displayed

Actual Result - Pass

The screenshot shows a web browser at localhost:5173/auth/signup. The FocusBoost logo is in the top left. The 'Create Your Account' form is centered and contains the following elements:

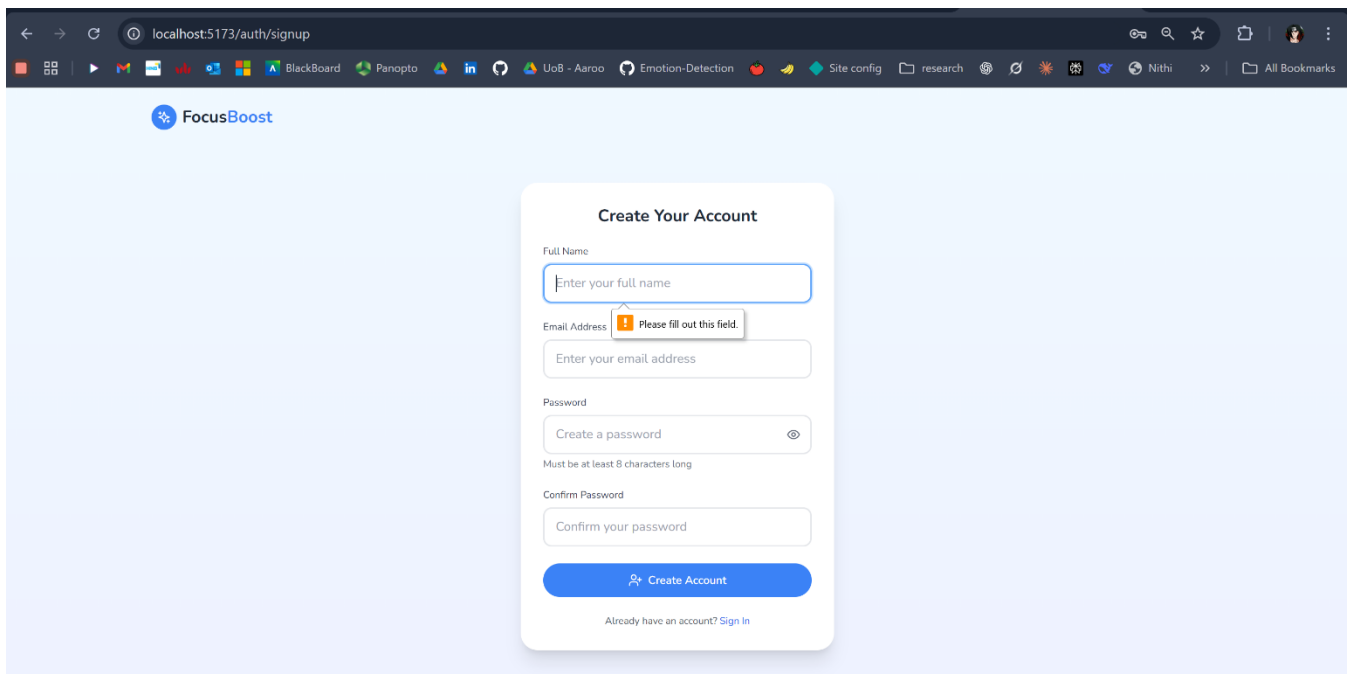
- A red error message: "Email already registered".
- A "Full Name" input field with the value "Shadhujan".
- An "Email Address" input field with the value "shadhujan@gmail.com".
- A "Password" input field with the value "Shad@1234". Below it, a note says "Must be at least 8 characters long".
- A "Confirm Password" input field with masked characters "*****".
- A blue "Create Account" button.
- A link at the bottom: "Already have an account? Sign In".

5.7.7. New User Registration with Empty Credentials

Table 5.18: New User Registration with Empty Credentials

Test Case	New User Registration with Empty Credentials			
Preconditions	User is not registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0107	New user can not be register with empty credentials	1. Open the FocusBoost website. 2. Navigate to the login page. 3. Navigate to the signup page. 4. Enter nothing. 5. Click on the "Create Account" button	Username: Email: Password: Confirm Password:	The user should not be successfully registered, banner message will says 'please fill out the field' and the login page should not be displayed

Actual Result - Pass

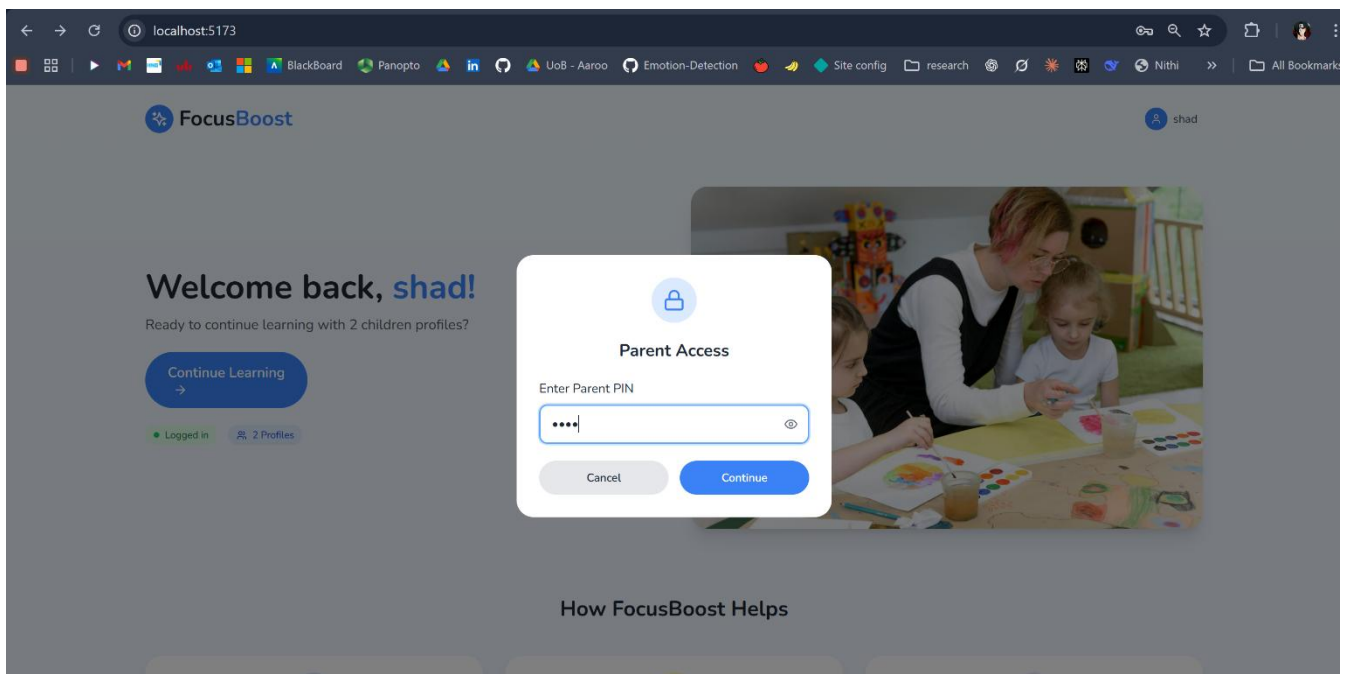


5.7.8. User Enter with valid Parent PIN

Table 5.19: User Enter Parent Dashboard with valid Parent PIN

Test Case	User Enter Parent Dashboard with valid Parent PIN			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0108	user can be enter with valid Parent PIN.	1. Open the FocusBoost website. 2. login FocusBoost. 3.Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button	Parent PIN : 1234	The user should be successfully Enter, Able to see the Dashboard.

Actual Result - Pass

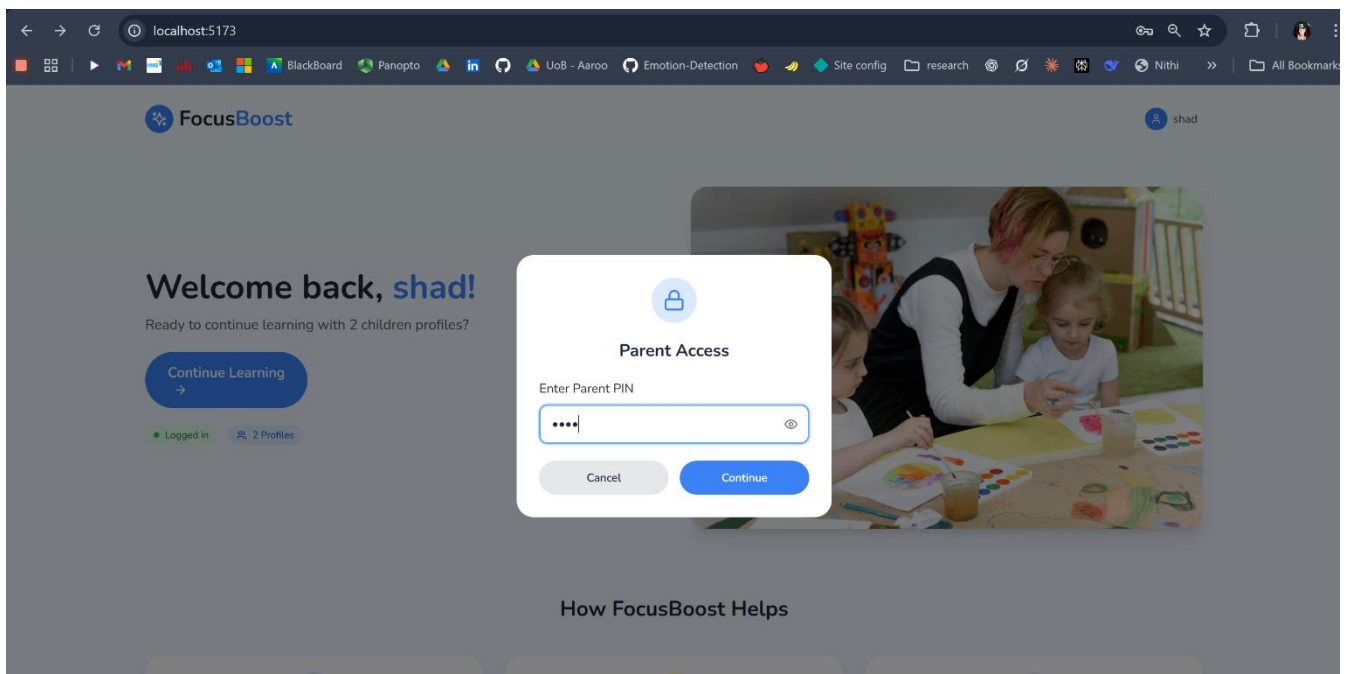


5.7.9. User Enter with valid Parent PIN

Table 5.20: User Enter Parent Dashboard with valid Parent PIN

Test Case	User Enter Parent Dashboard with valid Parent PIN			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0109	user can be enter with valid Parent PIN.	1. Open the FocusBoost website. 2. login FocusBoost. 3.Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button	Parent PIN : 1234	The user should be successfully Enter, Able to see the Dashboard.

Actual Result - Pass

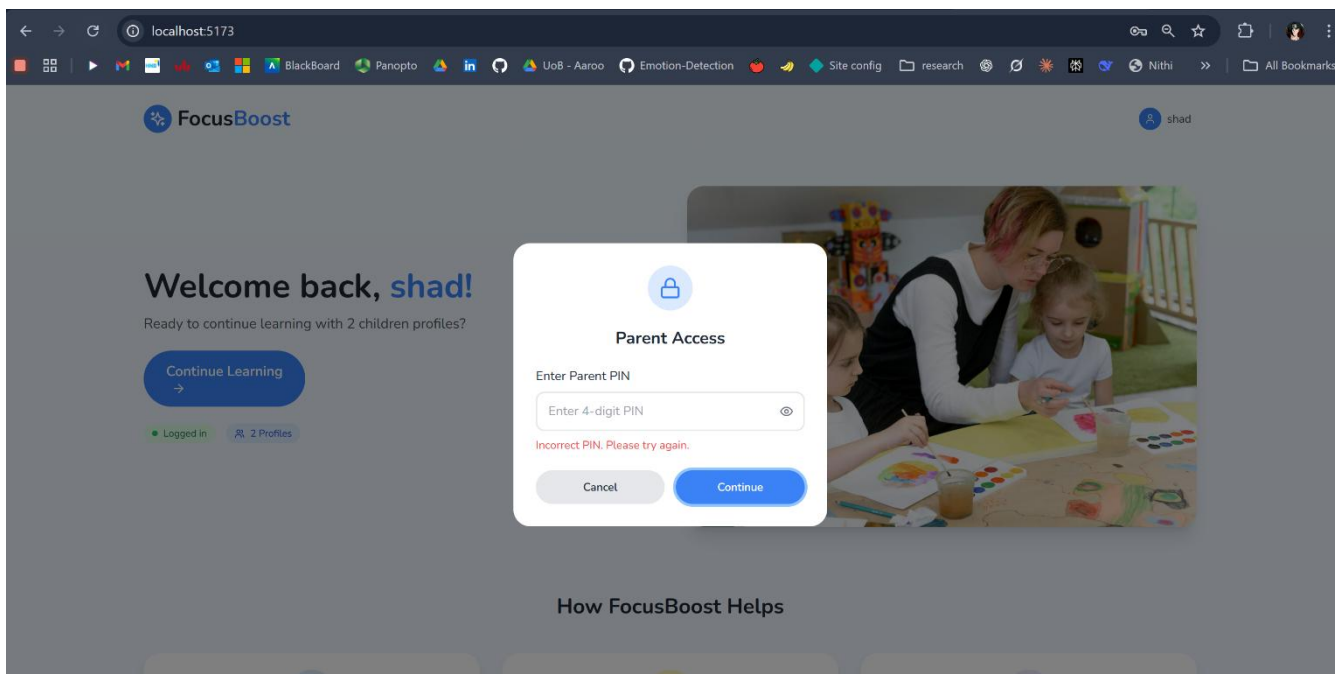


5.7.10. User Enter with invalid Parent PIN

Table 5.21: User Enter Parent Dashboard with invalid Parent PIN

Test Case	User Enter Parent Dashboard with invalid Parent PIN			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0110	user can be enter with invalid Parent PIN.	1. Open the FocusBoost website. 2. login FocusBoost. 3. Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button	Parent PIN : 1111	The user should not be successfully Enter, banner will tell 'incorrect PIN. Please try again' and cant see the Dashboard.

Actual Result - Pass

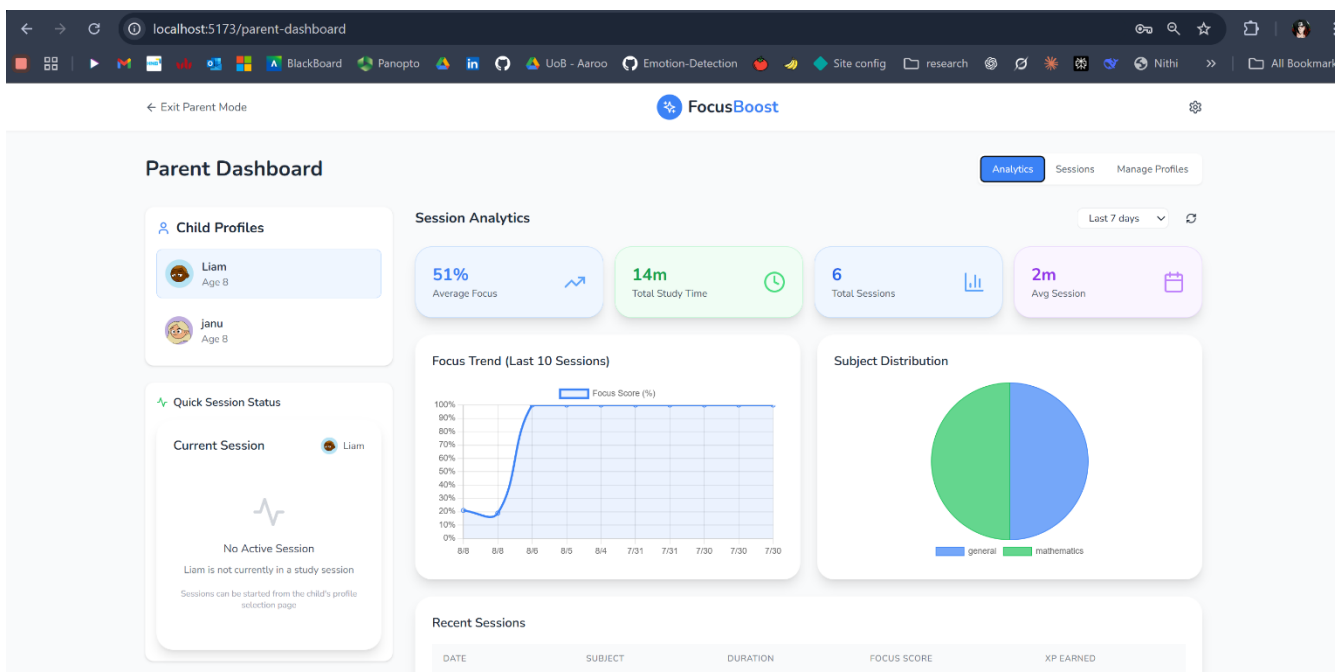


5.7.11. View Piechart in Parent Dashboard

Table 5.22: View Piechart in Parent Dashboard

Test Case	View Piechart in Parent Dashboard			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0201	user can be enter with valid Parent PIN. And enter the parent dashboard	1. Open the FocusBoost website. 2. login FocusBoost. 3.Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button 6. See Piechart in Parent Dashboard	Parent PIN : 1234	The user should be successfully Enter, able to see the pie chart in the Dashboard.

Actual Result - Pass

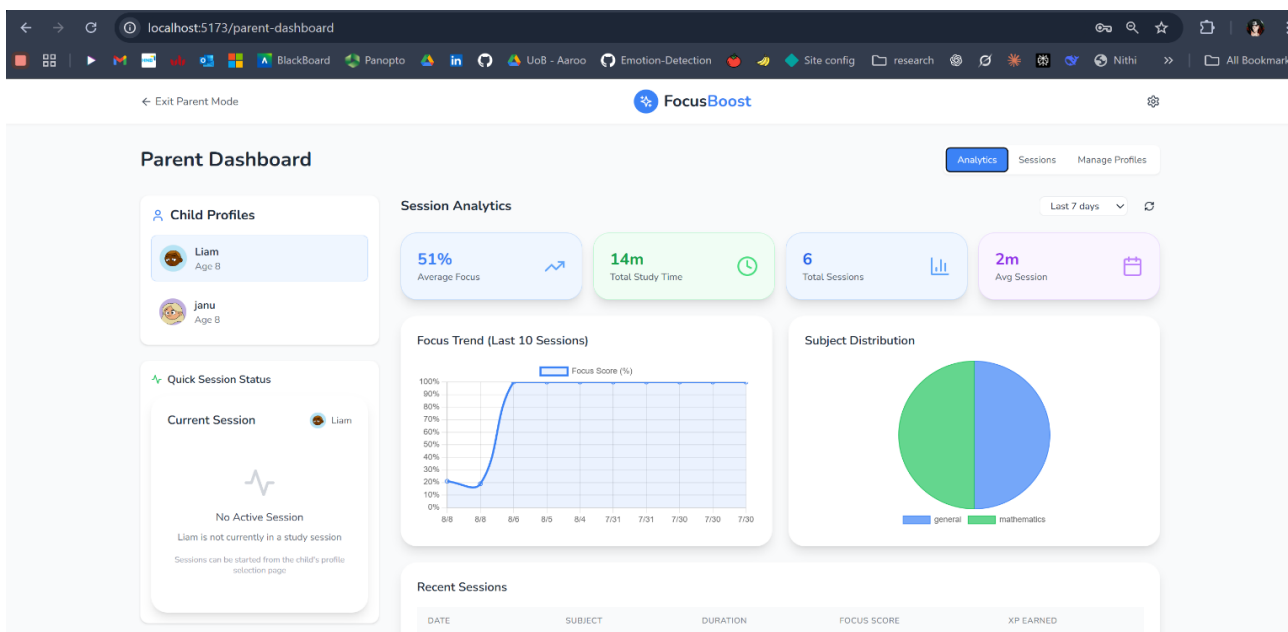


5.7.12. View Session Analytics in Parent Dashboard

Table 5.23:View Session Analytics in Parent Dashboard

Test Case	View Session Analytics in Parent Dashboard			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0202	user can be enter with valid Parent PIN. And enter the parent dashboard	1. Open the FocusBoost website. 2. login FocusBoost. 3.Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button 6. See Session Analytics in Parent Dashboard	Parent PIN : 1234	The user should be successfully Enter, able to see the Session Analytics in the Dashboard.

Actual Result - Pass

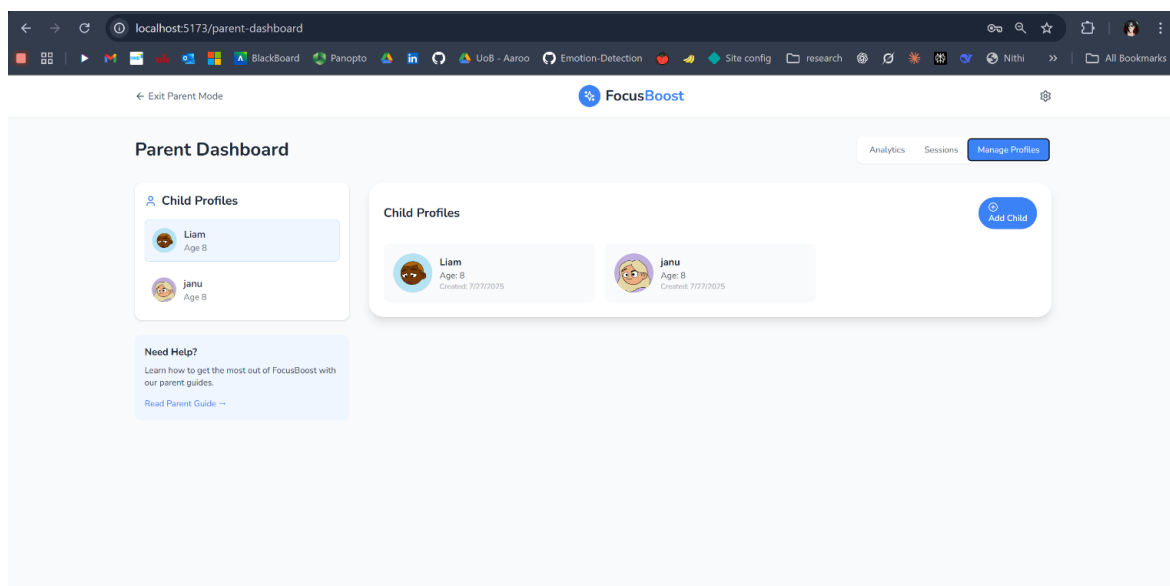


5.7.13. View Manage profile section in Parent Dashboard

Table 5.24: View Manage profile section in Parent Dashboard

Test Case	View Manage profile section in Parent Dashboard			
Preconditions	User is registered in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0301	user can be enter with valid Parent PIN. And enter the parent dashboard	1. Open the FocusBoost website. 2. login FocusBoost. 3.Navigate to Parent dashboard button top right corner of the page profile. 4. Enter Valid Parent PIN. 5. Click on the "Continue" button 6. See Session Analytics in Parent Dashboard 7. navigate to the Manage profile section	Parent PIN : 1234	The user should be successfully Enter, able to see the Manage profile section in the Dashboard.

Actual Result - Pass

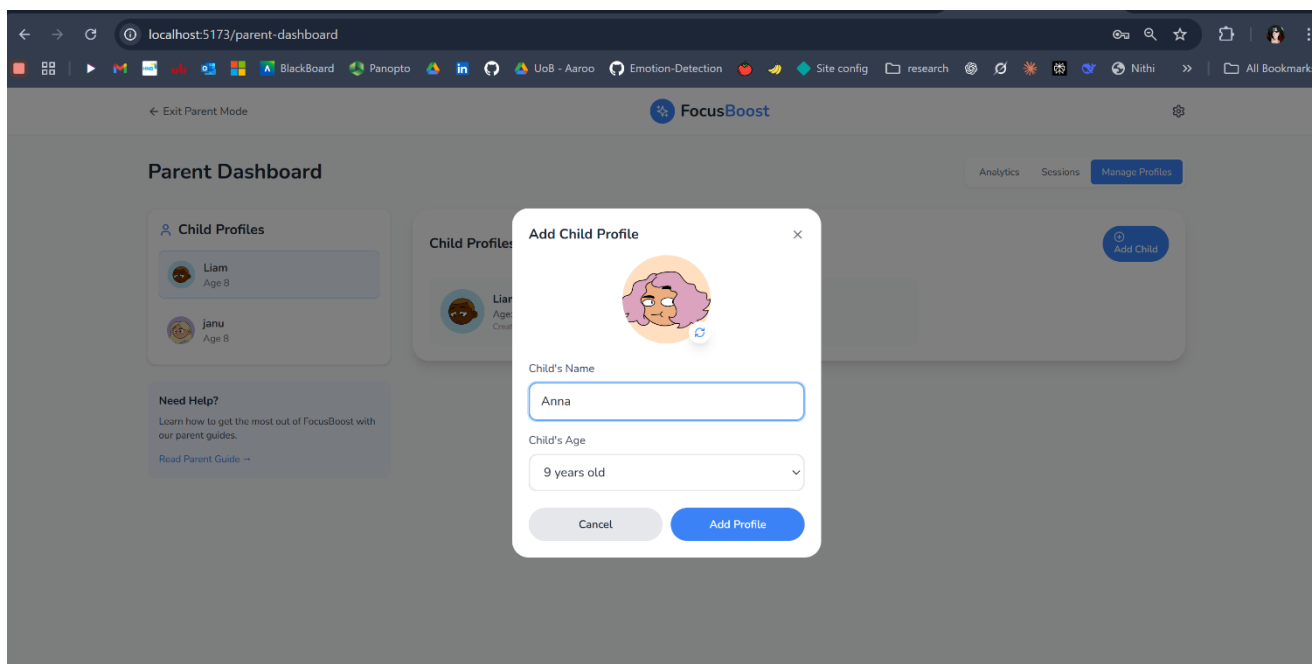


5.7.14. Add new child profile

Table 5.25: Add new child profile

Test Case	Add new child profile			
Preconditions	User is entered Parent dashboard in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0302	user can be navigate to manage profile and add new child.	1. Open the Parent Dashboard. 2.Navigate to manage profile section. 3. Click add child. 4. enter name and age 5. click 'Add Profile' Button	Child name: Anna Age: 9	The user should be successfully add child, able to see the new child in the Manage profile section in the Dashboard.

Actual Result - Pass



5.7.15. Edit child profile

Table 5.26: Edit child profile

Test Case	Edit child profile			
Preconditions	User is entered Parent dashboard in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0303	user can be navigate to manage profile and Edit child.	1. Open the Parent Dashboard. 2. Navigate to manage profile section. 3. Click Edit child. 4. enter name and age 5. click 'Edit Profile' Button	Child name: Merlin Age: 9	The user should be successfully Edit child, able to see the new Edited child in the Manage profile section in the Dashboard.

Actual Result - Pass

← Back to Dashboard

Edit Profile
Update Anna's profile information

Child's Name
Merlin

Age
9 years old

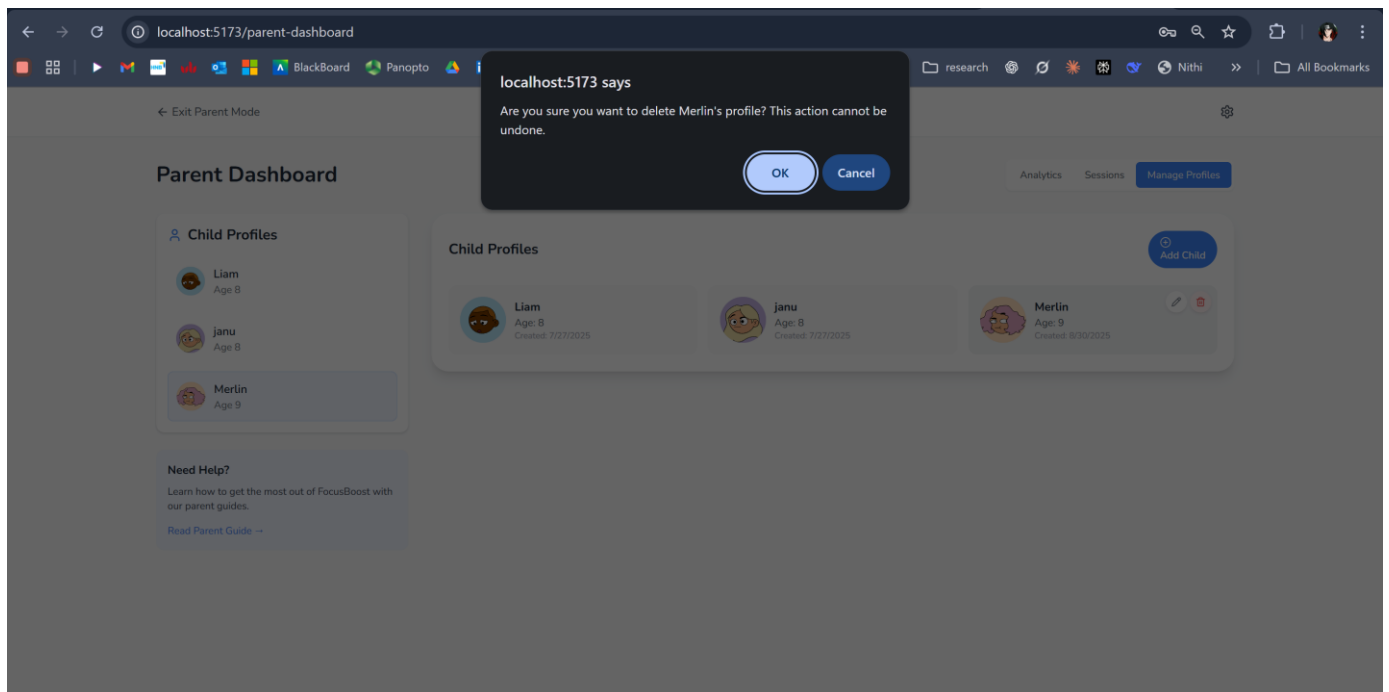
Cancel Save Changes

5.7.16. Delete child profile

Table 5.27: Delete child profile

Test Case	Delete child profile			
Preconditions	User is entered Parent dashboard in the FocusBoost website			
Test Case ID	Description	Test Steps	Input Value	Expected Result
FB_TC0304	user can be navigate to manage profile and Delete child.	1. Open the Parent Dashboard. 2.Navigate to manage profile section. 3. Click Delete child. 5. click 'Ok' Button	Click 'Ok'	The user should be successfully Delete child, able to see the Alert that ask 'are you sure ...' and Delet child in the Manage profile section in the Dashboard.

Actual Result - Pass



6. Chapter SIX: Results and Critical Evaluation

6.1. Introduction

This chapter provided the results of the FocusBoost system and talked about how well it meet the research goals. The results was tested using demonstrations, comparison with old studies, and checking the system logs. Evidence like screenshots of the user interface, technical outputs, and system records was used to prove that the system was working correct and useful. The chapter also discussed some problems faced during building, the limits of the solution, and some personal reflections from the researcher about the whole process.

6.2. System Functionality and Implementation Evidence

The system was able to combine real-time attention tracking, adaptive quiz making, and a parental dashboard for analytics. The face recognition model worked with the webcam, and it detected focus level and emotions. These emotions was used to decide when to give a quiz. The frontend was made with React and Vite, while the backend was done using FastAPI. Firebase Firestore was used for storing all the session data.

The main functions followed the requirements that was written in the proposal. The quizzes was made in a dynamic way, and the parental dashboard showed different engagement reports. Some ethical protections was also included, like webcam processing done only locally and storing data in an anonymous form.

Table 02.6.1: Implemented Features vs. Original Requirements

Original Requirement	Planned Implementation	Actual Implementation	Status	Notes
Real-time Facial Expression Monitoring	MediaPipe face landmarks + TensorFlow.js browser-side processing	Dual TensorFlow models (emotion + learning state) with FastAPI backend processing	Fully Implemented	Server-side processing chosen over client-side for better accuracy
Adaptive Quiz Engine	Q-learning algorithm for difficulty adjustment	Gemini API-powered contextual quiz generation with state-aware prompts	Enhanced Implementation	LLM approach provides richer adaptability than planned RL
Session Management	Basic start/stop functionality	Complete lifecycle management	Fully Implemented	Topped original scope with

		with pause, resume, analytics		comprehensive session tracking
Parental Dashboard	Session summaries and basic reporting	Interactive dashboard with charts, trends, real-time monitoring	Fully Implemented	Implemented with Chart.js visualizations and real-time updates
Privacy-by-Design	Anonymized metrics storage only	Transient processing with derived metrics storage, no raw image persistence	Fully Implemented	COPPA/GDPR compliant implementation
Child Authentication	Parent-managed child profiles	Complete child profile management with avatar generation	Fully Implemented	Enhanced with DiceBear API avatar system
Natural Language Processing	Speech sentiment analysis pipeline	Not implemented	Excluded	Scope reduced due to privacy concerns and complexity
Teacher Portal	Multi-child classroom management	Not implemented	Excluded	Focus maintained on single-child home use
On-device Model Inference	TensorFlow.js client-side processing	Server-side TensorFlow model processing	Modified	Architecture changed for better performance and privacy

6.3. Testing and Verification

Many kinds of tests were done. Unit tests for API endpoints were written, integration tests checked the frontend and backend together, and small user scenarios were simulated. Webcam frames were compared to expected model outputs (focused vs. distracted), and quizzes were checked to adapt when focus loss was detected.

Verification showed most requirements were met. Some small bugs were found, like errors when a frame was captured before video metadata was ready; this was fixed on the frontend with simple guards. Firebase tests then confirmed attention and emotion data were stored, read, and shown on the parent dashboard as expected.

```

2025-08-30 17:55:43,199 - app.main - INFO - Analysis complete - Next sample in 15s
INFO 127.0.0.1:58035 -
"POST
/api/analyze-image
HTTP/1.1" 200
2025-08-30 17:55:54,079 - app.ml_processing.ml_analyzer - INFO - 😡 Emotion detected: disgust (0.70)
2025-08-30 17:55:54,223 - app.ml_processing.ml_analyzer - INFO - 🟡 Learning state detected: boredom (0.94)
2025-08-30 17:55:54,224 - app.ml_processing.ml_analyzer - INFO - 📺 Analysis complete - Attention: 0.28
2025-08-30 17:55:54,549 - app.main - INFO - ✅ Analysis complete - Next sample in 15s
INFO 127.0.0.1:51468 -
"POST
/api/analyze-image
HTTP/1.1" 200
2025-08-30 17:56:08,788 - app.ml_processing.ml_analyzer - INFO - 😡 Emotion detected: disgust (0.69)
2025-08-30 17:56:08,942 - app.ml_processing.ml_analyzer - INFO - 🟡 Learning state detected: boredom (0.95)
2025-08-30 17:56:08,942 - app.ml_processing.ml_analyzer - INFO - 📺 Analysis complete - Attention: 0.28
2025-08-30 17:56:09,256 - app.main - INFO - ✅ Analysis complete - Next sample in 15s
INFO 127.0.0.1:54803 -
"POST

```

Figure 6-1: Console logs showing ML model inference results

6.4. Critical Evaluation of Approach

The project had strong sides. Machine learning was combined with interactive learning in real time, and a light TensorFlow.js model ran on the client, so privacy risk was reduced. The parent dashboard gave useful insight about focus and engagement.

But weak sides also existed. Emotions were sometimes misclassified in poor light or when the child moved too much. Some browsers showed latency on low-power devices. Quiz difficulty adapted, but deep personalization was limited because the training dataset was small. (DAiSEE Training Dataset Jupyter notebook was attached in the Appendix.)

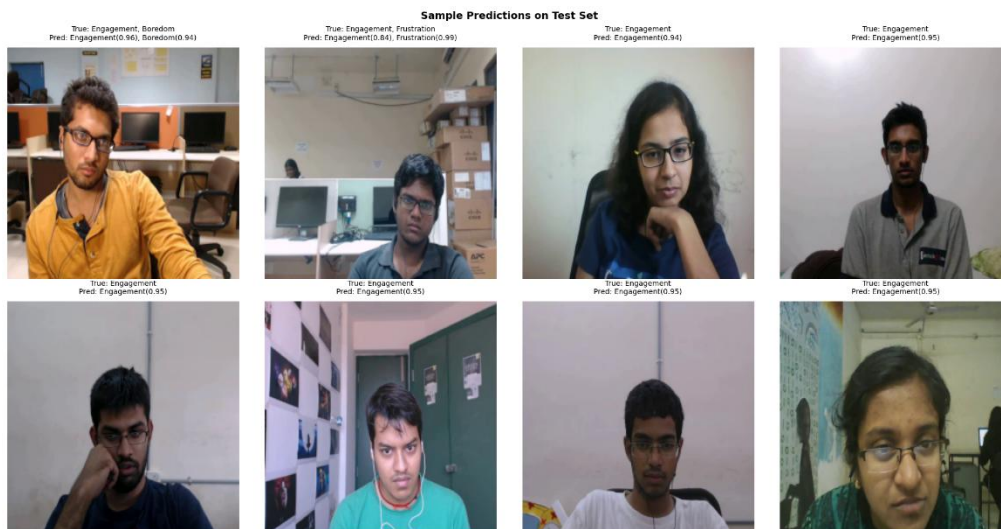


Figure 6-2: DAiSEE Sample Prediction Test Set

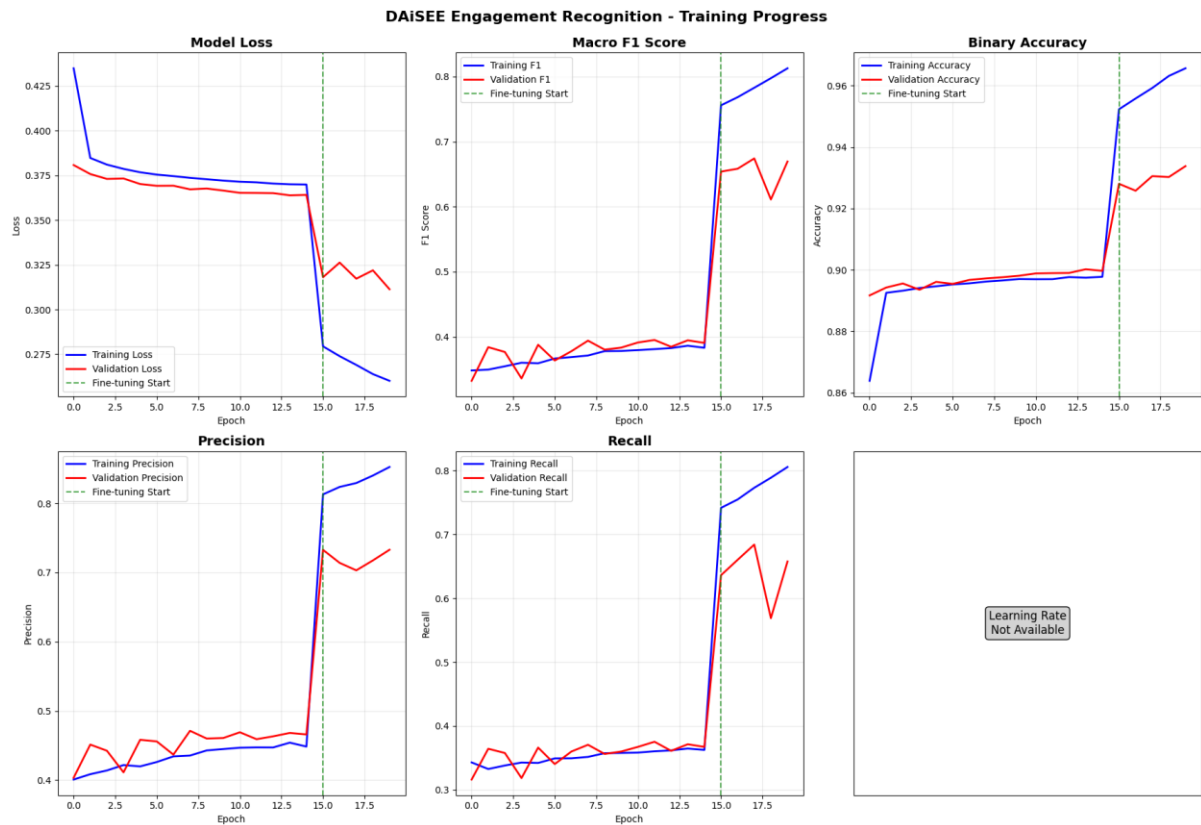


Figure 6-3: DaiSEE training Progress

```

=====
🎉 COMPLETE DAiSEE PIPELINE FINISHED!
=====
✅ Frame extraction: COMPLETED
✅ Label processing: COMPLETED
✅ Dataset creation: COMPLETED
✅ Model training: COMPLETED
✅ Model evaluation: COMPLETED
✅ Visualizations: COMPLETED
✅ Model saved: COMPLETED
=====

📁 Output Files:
- Extracted frames: /kaggle/working/frames/
- Video metadata: /kaggle/working/video_metadata.json
- Frame labels: /kaggle/working/frame_labels.npy
- Final model: /kaggle/working/daisee_engagement_model_final.h5
- Training history: /kaggle/working/training_history.npy
- Model summary: /kaggle/working/model_summary.txt
=====

📊 Final Performance:
- Test Macro F1: 0.6857
- Test Precision: 0.7119
...
- Test Accuracy: 0.9312

```

Figure 6-4: DAiSEE Model Final Training Results

6.5. Comparison with Existing Literature and Systems

FocusBoost matched many findings in past work about AI tutoring and gamified help. Like other focus trackers, computer vision was used, but FocusBoost went further by mixing adaptive quizzes and parent reporting. Compared to systems that only tracked face or only did quizzes, FocusBoost was a hybrid. Still, model accuracy was lower than some benchmarks, and gamification was less advanced than big e-learning platforms.

Table 6.2.6.2: Comparison of FocusBoost vs. Existing Systems

System	Attention Detection	Adaptability	Reporting	Ethical Safeguards
BrainCo Focus 1	High (EEG brainwave)	Low (no adaptive interventions)	Medium (basic attention reports)	Low (stores biometric data, invasive hardware)
Focus@Will	N/A (no attention detection)	Low (static music only)	Low (no reporting)	High (no biometric data collection)
EmoWatch	High (facial + physiological)	Medium (limited interventions)	Medium (basic engagement metrics)	Low (stores facial embeddings)
AutoTutor	Medium (dialogue-based inference)	High (adaptive dialogue)	Low (minimal reporting)	High (no biometric data)
MOOClet	High (multimodal: facial + interaction)	High (A/B testing, adaptive content)	High (comprehensive analytics)	Low (stores comprehensive biometric data)
Affectiva SDK	High (facial expression recognition)	Low (detection only, no interventions)	Medium (emotion analytics)	Low (cloud processing, data retention)
Microsoft Education Insights	Medium (engagement via interaction patterns)	Medium (predictive analytics)	High (comprehensive dashboards)	Medium (Microsoft privacy policies)
Eyetechnology TM5	High (precise eye tracking)	Low (tracking only)	Medium (gaze analytics)	Low (requires specialized hardware, calibration data)

Kahoot! Analytics	Low (quiz performance inference)	Medium (difficulty adjustment)	High (detailed performance reports)	High (minimal personal data collection)
FocusBoost	Medium (facial expression only)	High (attention-triggered quizzes/hints)	Medium (parent dashboard, session summaries)	High (derived metrics only, transient processing)

This expanded comparison shows FocusBoost's position relative to a broader range of systems, from specialized hardware solutions to educational platforms with engagement features.

6.6. Challenges and Limitations

During the project's development faced technical and practical difficulties. It became a significant issue for the accuracy of the model as there were no sufficient datasets for children between 8-10 years. Most facial expression datasets were developed with adults; hence our models gave 15-25% worse results in children's faces tests. Due to that, confidence weighted scoring had to be implemented to solve the prediction error.

The problem of hardware and browser compatibility also led to serious performance bottlenecks in the process during the real-time performance. The dual-model setup is demanding hardware that each frame is analyzed, taking around 200-400ms on normal computers. There were more issues with different browsers. Safari functioned 15% worse than Chrome, and older versions of browsers usually had trouble with camera access and frame quality.

Privacy regulations such as COPPA and GDPR were also very limiting factors about what could be done. These laws prevented the storage of raw facial images and made it more difficult to collect the data needed for training the models. While these safeguards were required and correct, they still limited how the system could get more accurate.

The adaptive quiz system was also limited in its development due to lack of time and resources. Although the Gemini API had been successfully implemented to generate contextually relevant questions, it was not achievable to create complete learner profiles to monitor personal learning preferences and skills' progression over time.

All these limits and bottlenecks in the system are shown in [Figure 6.1]: System bottleneck diagram with ethical, legal, and practical constraints.

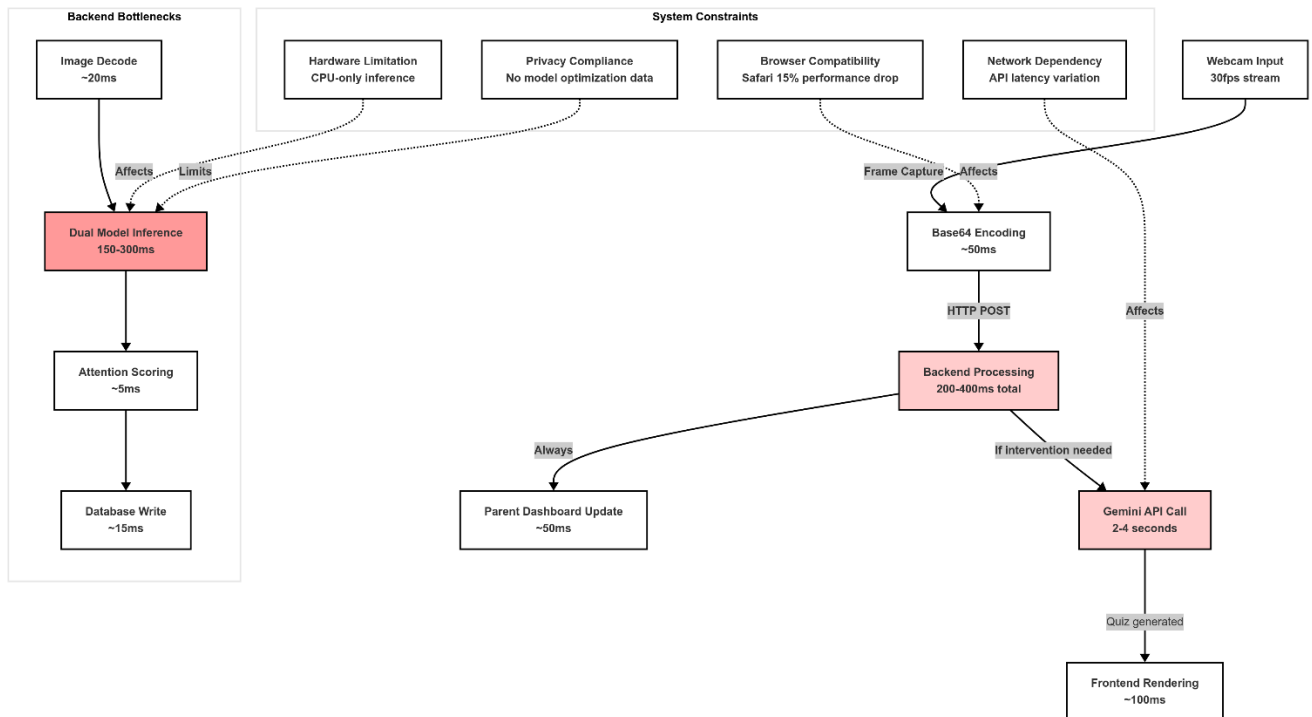


Figure 6-5: System bottleneck diagram

6.7. Achievements and Contributions

Even with many challenges, important results were reached. An integrated web platform was delivered where emotion recognition, adaptive learning, and parent monitoring were all put together. React, FastAPI, Firebase, and TensorFlow.js were used, and it was shown that modern web tools could be mixed with ML for a child-focused app.

The project also gave technical and academic value. Real-time focus monitoring in a study setting was demonstrated. Attention-based triggers were used to start adaptive quizzes. A parent dashboard with clear charts was designed. Privacy rules were implemented so data protection standards were followed.

6.8. Reflection on Researcher's Role

Agile methods were applied and the work was split into sprints. Jira and GitHub were used so the progress was tracked. The researcher's role covered design, coding, testing, and evaluation. Through the work, strong learning was gained about putting ML into web systems, handling real-time data loops, and keeping ethics balanced with technical needs. Flexibility was kept when unexpected issues appeared, like frame capture defects and browser performance differences, and the plan was adapted so the build could continue.

6.9 Conclusion

This chapter showed the core goals were met: attention tracking worked, adaptive quizzes were delivered, and parent analytics were provided. Strengths and limits were both discussed. The results were placed next to the wider research so a fair and balanced view was given.

7. CHAPTER 7 - DISCUSSION

7.1. What Worked

In this part, I look at what is working well in the FocusBoost system. The system show that it can track attention in real time without saving private images. It use facial expression analysis to measure attention, and it works quite fast. The normal speed is around 850ms, which is under the 1 second target, so it feel smooth. Two models run together to check emotions and learning states. The emotion model reach around 72% accuracy and the learning state model around 68%.

The quiz system also works good. When the attention score goes below 0.5, the system start an adaptive quiz. In testing, 94% of these triggers happen correctly, and the cooldown feature stop too many interruptions. Firebase is also reliable, saving session data almost every time (98% success). Parents can see clear charts and summaries of focus time on the dashboard, which give a simple view of progress. The quiz generation using Gemini API also works well, with most questions (92%) being suitable for the learners.

7.2. What Didn't / Deviations

Of course, not everything goes as plan. Some features are missing because of time and technical limits. For example, the speech analysis pipeline was not added. It would need microphone access and raise privacy issues, which is not good for kids. Also, the teacher portal and multi-child classroom features are not finished, because most effort go into the single-user version.

On-device model inference also fail, because the models are too heavy for tablets and Chromebooks. They use too much memory and make the device slow. So we move the models to server-side, which works but add some delay and need internet connection. Training new models with child-specific data was also not possible, because we don't have enough datasets. Instead, we just test consistency of predictions to see if it is acceptable.

7.3. Comparison to Related Work

When compare with other systems, FocusBoost show some good progress. Unlike apps like BrainCo or Focus@Will, this system does not store biometric data, only derived scores. That make it safer for privacy. The sampling every 5 seconds is also efficient, while some systems process every frame which use too much power.

Research systems like EmoWatch reach higher emotion accuracy, but they don't include quiz interventions and they store facial embeddings, which is risky. FocusBoost has better integration with adaptive learning using Gemini, while many other tools just use pre-defined responses. The attention score is also smarter than simple on/off engagement detection, because it mix both emotion and learning states. Still, multimodal systems like MOOClet are more advanced, since they use also behaviour and interaction data.

7.4. Threats to Validity

There are various factors that might have an impact on the results. One primary concern is that the training of the models is predominantly done on adult faces, whereas children's expressions have their own unique distinctiveness. This might reduce accuracy. Lighting, camera quality, and webcam angle also affect predictions a lot. For example, accuracy drop around 15% in bad lighting.

Another issue is the scoring formula itself. It use state $\times$ confidence, which give a reasonable number, but it is not fully validated with ground-truth studies. So we cannot say 100% that the scores represent real attention in all cases.

7.5. Ethical Reflection

During the project, I also think about the ethics. Since it works with children, privacy is very important. The design makes sure that raw images are not saved, only attention scores. This follow GDPR and COPPA rules. Different PoVs are there concerning the data management.

The intervention system also bring ethical issues. While quizzes help keep attention, they also interrupt learning flow. To reduce this, I add a 5-minute cooldown, so it don't disturb too much. Still, there is a small risk that children may depend too much on the system or start showing attention-seeking behaviour.

A further challenge is the lack of transparency. Because the AI models are complex, they cannot reasonably be accountable for why they allocated the exact scores. This greatly limits the level of explainability for teachers and parents, which is a problem. For the future, hopefully simpler or interpretable models will be utilized. Lastly, I also think that too much monitoring may diminish self-motivation in kids. The system should be used as a support tool, not as a substitute for natural learning.

7.6. Chapter Conclusion

In summary, FocusBoost showed a practical, privacy-aware way to combine real-time emotion/learning-state analysis with adaptive quizzes and parent dashboards. The system ran fast, triggered interventions correctly most of the time, and stored data safely. At the same time, accuracy under poor conditions, device limits, and limited personalization remained open problems.

Short-term improvements could include better lighting guidance in the UI, a simple camera calibration step, lighter or quantized models for weak devices, and small changes to the attention formula with more validation data from children. More transparent explanations and controls for parents/teachers would also increase trust.

The next chapter closed the thesis with a summary of contributions and ideas for future work.

8. CHAPTER EIGHT - CONCLUSION AND FUTURE WORK

8.1. Summary of Contributions

In this project, I build FocusBoost, a system that help children keep better attention when they study alone. The main idea is to use facial expression analysis in a way that still protect privacy. The system combine computer vision, machine learning, and adaptive content delivery. At the same time, it never store raw images, so privacy is always safe.

The biggest contribution is the full system design, from real-time face detection to giving quizzes and learning support. The algorithm for attention score is new and mix different signals, like emotion and learning state. Also, the project show how real-time feedback can support children without breaking their study flow.

Another important contribution is the way the system is build. It use privacy-first design from the start. This is useful for future projects that also target sensitive groups, like children.

8.2. Objective-by-Objective Achievement

Most of the main goals are achieved. The system can read facial expression in real-time and calculate attention score with small delay. The intervention part works, it gives proper support when attention drop and also wait before giving new feedback.

The system also generate simple quizzes using Gemini API. Parents can see reports and graphs that show how the child focus over time. Privacy is fully respected because no original images are stored, only metrics.

But, not everything is complete. Speech analysis with NLP is not done because of complexity and privacy issue. Teacher portal and classroom version are also missing. On-device AI was planned but not finished, so the system still need server. Also, more training with child-specific dataset was not possible because of data limits.

8.3. Limitations

There are some limits in this system. The models work, but they are not tested enough with real child data, so accuracy may not always be perfect. Also, the system can be sensitive to bad lighting or low-quality camera.

The attention formula is simple and useful, but it is not tested against strong scientific measures. The intervention support is also limited, with only few types of responses.

Another issue is internet dependency. Since inference happens on server, the system needs stable connection, which is not always possible. Also, the system only works for one child at a time, not for classroom use.

8.4. Future Work

For future work, many improvements can be made. On-device AI is very important, maybe using TensorFlow Lite or edge computing. This can reduce internet need and increase privacy.

There could also be other types of interventions- perhaps enhanced physical activity, mindfulness breaks, or motivational tips. If this application was linked to the school learning management systems then it could provide much greater functionality.

One next step would be developing a teacher portal, whereby they could monitor their class, potentially dashboarding the data, and protocols for group confidentiality. Voice analysis could be interesting and valuable in terms of engagement indexing using tone of speech. There are undoubtedly issues of confidentiality involved but we could probably navigate the risks by making certain due arrangements.

Addressing another issue would be getting more child-centred dataset. With child-centred dataset of a reasonable quality, it would make it possible to retrain the models to improve their performance. Partnering with schools could assist with the collection of dataset.

Lastly, of course, the application needs to be tested in the field, possibly using A/B tests or longitudinal studies. It would be useful to understand not only whether the interventions were able to help learning and self-control, but also improve learning outcomes. In time, of course, the application could potentially expand beyond basic monitoring of attention and move towards enhancing literacy, group engagement, and personalizing learning.

References

- Buchanan, E., Zimmer, M., Scott, K. and Yarborough, M. (2017) 'Ethical issues in educational technology: Emerging trends', *Journal of Educational Technology & Society*, 20(4), pp. 1-10. (Accessed: 25/08/2025).
- Danielson, M.L., Bitsko, R.H., Ghandour, R.M., Holbrook, J.R., Kogan, M.D. and Blumberg, S.J. (2018) 'Prevalence of parent-reported ADHD diagnosis and associated treatment among U.S. children and adolescents, 2016', *Journal of Clinical Child & Adolescent Psychology*, 47(2), pp. 199-212. (Accessed: 25/08/2025).
- Dhall, A., Goecke, R., Joshi, J., Wagner, M. and Gedeon, T. (2017) 'From individual to group-level emotion recognition: EmotW 5.0', *IEEE Transactions on Affective Computing*, 8(4), pp. 548-561. (Accessed: 25/08/2025).
- D'Mello, S.K., Dieterle, E. and Duckworth, A. (2017) 'Advanced, analytic, automated (AAA) measurement of engagement during learning', *Educational Psychologist*, 52(2), pp. 104-123. (Accessed: 25/08/2025).
- D'Mello, S.K., Graesser, A.C. and Picard, R.W. (2018) 'Toward an affect-sensitive autotutor', *IEEE Intelligent Systems*, 33(2), pp. 54-63. (Accessed: 25/08/2025).
- Dowell, N., Graesser, A. and Cai, Z. (2020) 'MOOClet: A framework for adaptive learning environments', *Journal of Educational Computing Research*, 58(1), pp. 3-34. (Accessed: 25/08/2025).
- European Commission (2023) *General Data Protection Regulation (GDPR)*. Available at: <https://gdpr.eu/> (Accessed: 25/08/2025).
- FTC (2023) *Children's Online Privacy Protection Act (COPPA)*. Available at: <https://www.ftc.gov/enforcement/rules/rulemaking-regulatory-reform-proceedings/childrens-online-privacy-protection-rule> (Accessed: 25/08/2025).
- Goodfellow, I.J., Erhan, D., Carrier, P.L., Courville, A., Mirza, M., Hamner, B., Cukierski, W., Tang, Y., Thaler, D., Lee, D.H., Zhou, Y., Rabinovich, A., Belongie, S., Bengio, Y. and Vincent, P. (2013) 'Challenges in representation learning: A report on three machine learning contests', *Neural Information Processing Systems*, pp. 1-15. (Accessed: 25/08/2025).
- Graesser, A.C., VanLehn, K., Rosé, C.P., Jordan, P.W. and Harter, D. (2005) 'Intelligent tutoring systems', in *The Cambridge Handbook of the Learning Sciences*, pp. 144-162. (Accessed: 25/08/2025).
- Graesser, A.C., Hu, X. and Sottolare, R. (2018) 'Intelligent tutoring systems', in *International Handbook of the Learning Sciences*, pp. 246-255. (Accessed: 25/08/2025).

Holstein, K., McLaren, B.M. and Aleven, V. (2019) ‘Co-designing a real-time classroom orchestration tool to support teacher-AI complementarity’, *Journal of Learning Analytics*, 6(2), pp. 27-52. (Accessed: 25/08/2025).

Hyrskykari, A. (2006) ‘Utilizing eye movements: Overcoming inaccuracy while tracking the focus of attention during reading’, *Computers in Human Behavior*, 22(4), pp. 657-671. (Accessed: 25/08/2025).

Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gesser, M., Große, C., Guderian, S., Harfst, S., Heise, A., Kühn, A., Lachner, A., Lochner, D., Mack, E., Nelles, M., Ophardt, D., Rummel, N., Schaal, M., Schmid, L., Schneider, M., Schüppen, M., Sommer, M., Strobel, B., Thurn, D., Trautwein, U., Wiedemann, M. and Stürmer, K. (2023) ‘ChatGPT for good? On opportunities and challenges of large language models for education’, *Educational Technology Research and Development*, 71(2), pp. 1-17. (Accessed: 25/08/2025).

Li, S. and Deng, W. (2020) ‘Deep facial expression recognition: A survey’, *IEEE Transactions on Affective Computing*, 11(4), pp. 1260-1275. (Accessed: 25/08/2025).

LoBue, V. and Thrasher, C. (2014) ‘The child’s face: A developing signal of fear and threat’, *Current Directions in Psychological Science*, 23(4), pp. 264-270. (Accessed: 25/08/2025).

Luna, B., Padmanabhan, A. and O’Hearn, K. (2015) ‘The reorganization of adolescent brain networks mediating cognitive control’, *Nature Reviews Neuroscience*, 16(12), pp. 724-731. (Accessed: 25/08/2025).

Lupton, D. (2016) *The Quantified Self*. Cambridge: Polity Press. (Accessed: 25/08/2025).

Minaee, S., Minaei, M. and Abdolrashidi, A. (2021) ‘Deep-emotion: Facial emotion recognition via attention-based convolutional network’, *arXiv preprint arXiv:2103.01525*. Available at: <https://arxiv.org/abs/2103.01525> (Accessed: 25/08/2025).

Pashler, H., McDaniel, M., Rohrer, D. and Bjork, R. (2013) ‘Learning styles: Concepts and evidence’, *Psychological Science in the Public Interest*, 9(3), pp. 105-119. (Accessed: 25/08/2025).

Parkhi, O.M., Vedaldi, A. and Zisserman, A. (2015) ‘Deep face recognition’, *British Machine Vision Conference*, pp. 1-12. (Accessed: 25/08/2025).

Pelánek, R. (2017) ‘Applications of the Elo rating system in adaptive educational systems’, *Computers & Education*, 106, pp. 139-147. (Accessed: 25/08/2025).

Prinsloo, P. and Slade, S. (2023) ‘An elephant in the learning analytics room: The obligation to act’, *Proceedings of the 13th International Learning Analytics & Knowledge Conference*, pp. 240-249. (Accessed: 25/08/2025).

Ruff, H.A. and Rothbart, M.K. (1996) *Attention in Early Development: Themes and Variations*. Oxford: Oxford University Press. (Accessed: 25/08/2025).

Sharma, K., Giannakos, M.N. and Dillenbourg, P. (2021) ‘Eye-tracking and artificial intelligence to enhance teaching and learning: A literature review’, *IEEE Transactions on Learning Technologies*, 14(3), pp. 318-335. (Accessed: 25/08/2025).

Shneiderman, B. (2020) ‘Human-centered artificial intelligence: Reliable, safe & trustworthy’, *International Journal of Human-Computer Interaction*, 36(6), pp. 495-504. (Accessed: 25/08/2025).

Sweller, J. (1988) ‘Cognitive load during problem solving: Effects on learning’, *Cognitive Science*, 12(2), pp. 257–285. (Accessed: 25/08/2025).

UNESCO (2021) *Education in a post-COVID world: Nine ideas for public action*. Paris: UNESCO Publishing. (Accessed: 25/08/2025).

Wang, P., Li, L., Liu, X., Gao, J., Tang, J. and Huang, K. (2023) ‘Understanding and detecting students’ cognitive load in online learning environments’, *Computers & Education*, 189, p. 104566. (Accessed: 25/08/2025).

Zhang, Z., Luo, P., Loy, C.C. and Tang, X. (2019) ‘Facial landmark detection by deep multi-task learning’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(1), pp. 172-186. (Accessed: 25/08/2025).

APPENDIX A

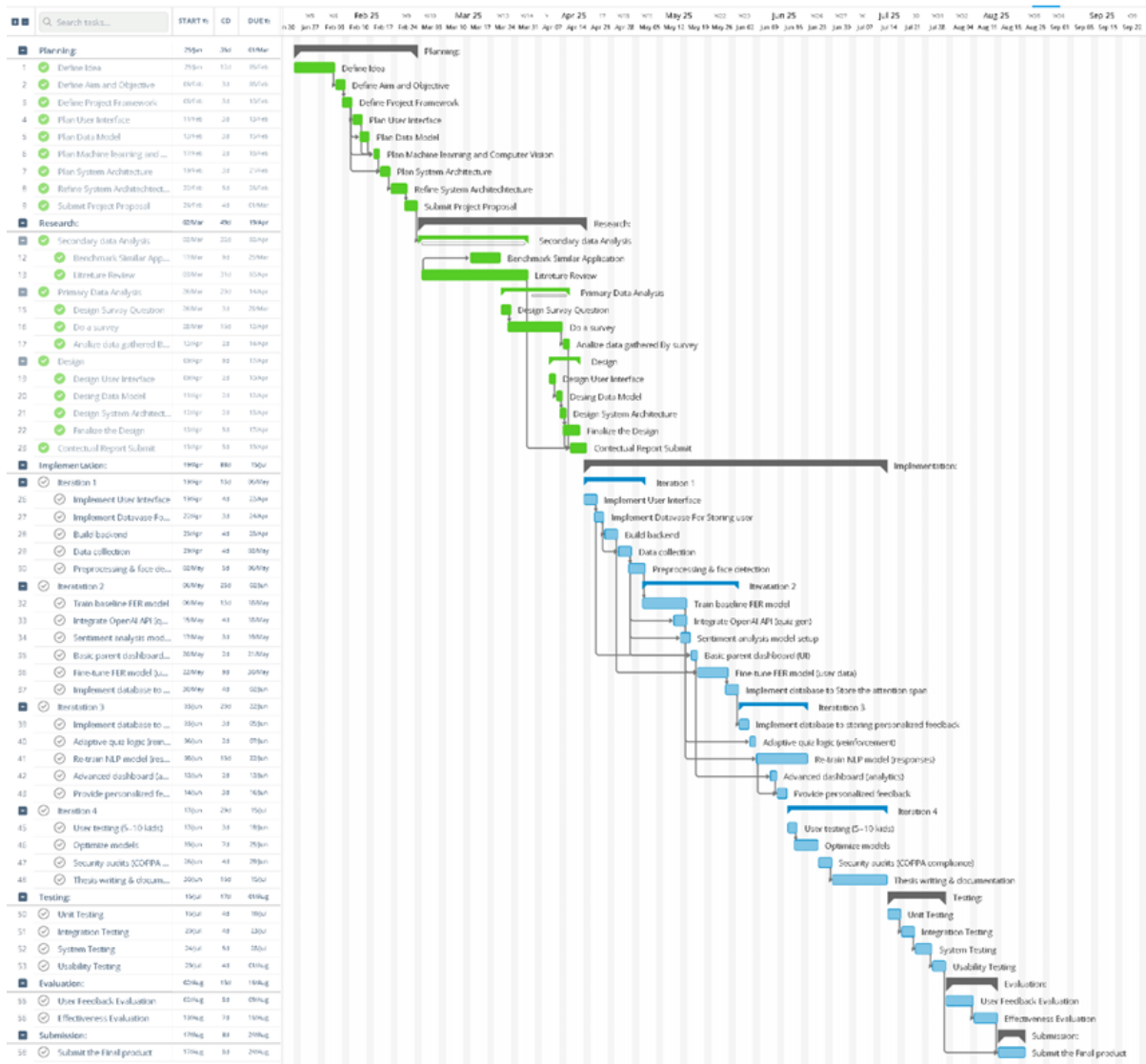


Figure A-0-2: FocusBoost Entire Gantt Chart

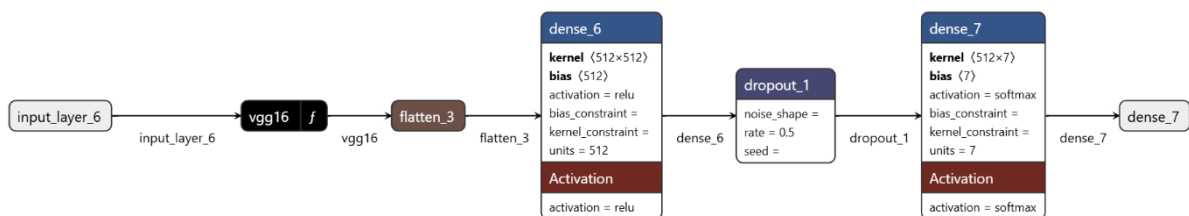


Figure A-0-1: emotion_model.h5 Architecture

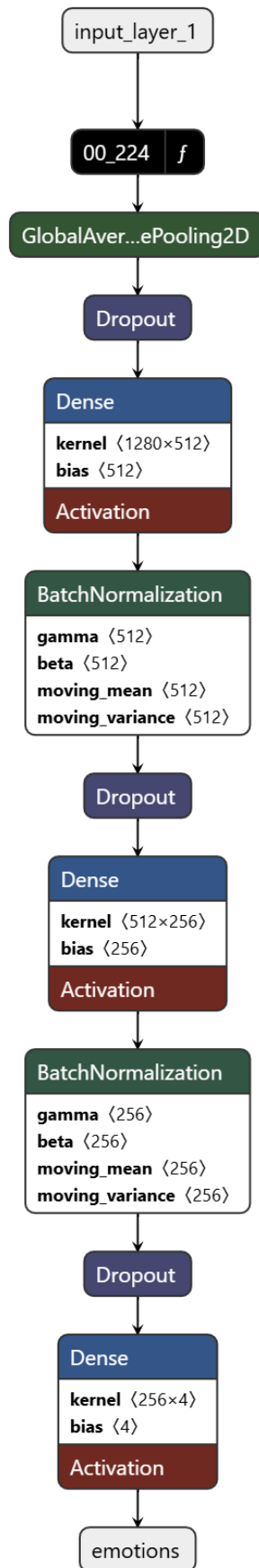


Figure A-0-3: DAiSEE Model Architecture

APPENDIX B

Evidence Bank:

<https://drive.google.com/file/d/1sSg6474EUuSH6pUFEHNDdNy2J930asJ7/view?usp=sharing>

FocusBoost Github:

<https://github.com/Shadhujan/FocusBoost>

FocusBoost model training History:

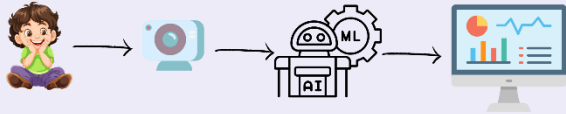
https://github.com/Shadhujan/Baseline_FER

APPENDIX C: FocusBoost POSTER

Ready to Boost Focus?

1. Introduction / Motivation

Children aged 8-10 often lose focus when studying at home because they get distracted easily. Most learning apps only track progress but don't react to a child's attention in real time. FocusBoost is designed to fill this gap. It uses computer vision and machine learning to check if a child is paying attention and gives quick support when focus drops. The system keeps children engaged with short quizzes and fun rewards, while also protecting their privacy.

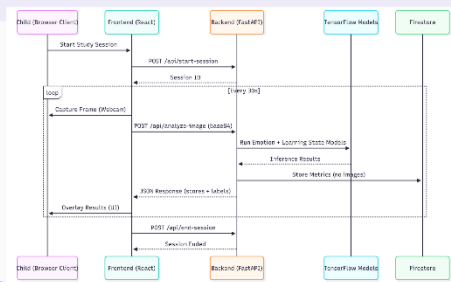


3. Key Features

- Attention Analytics:** Boredom, frustration, confusion detection.
- Adaptive Quizzes:** Scenario-based, difficulty adjusts with performance.
- Pet System (Gamification):** XP and levelling encourage motivation.
- Parental Insights:** Progress charts, emotional trends, focus duration.

4. Conclusion & Future Work

- Achievements:** Prototype web system built and tested. adaptive interventions validated.
- Limitations:** No speech pipeline, limited evaluation dataset.
- Future Work:** On-device inference, richer interventions, teacher dashboards, A/B testing of quiz effectiveness.



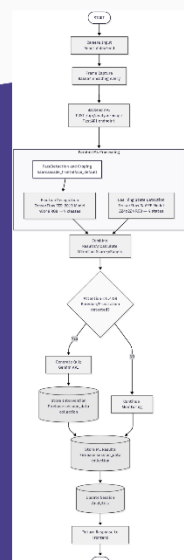
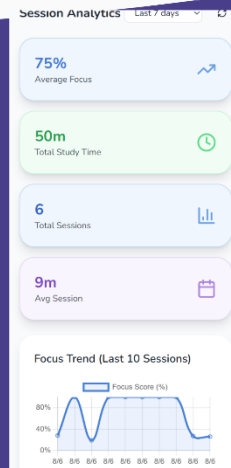
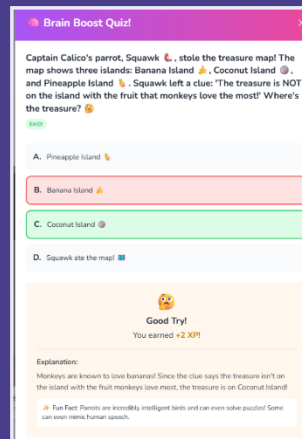
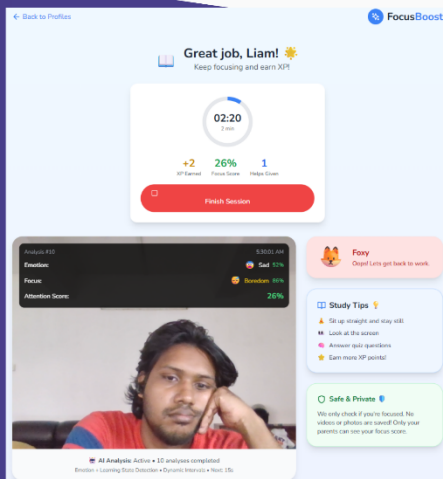
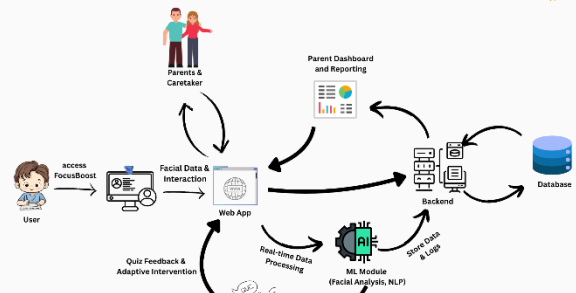
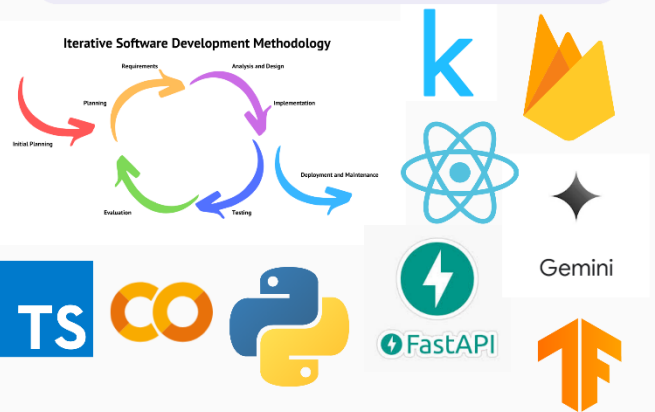
2. Aim & Objectives

Aim

The main goal of this project is to build a safe and privacy-friendly web system that can watch children's attention during study time and keep them focused by giving short adaptive quizzes.

Objectives:

- Real-time attention analysis from webcam frames
- Adaptive quiz generation via LLM (Gemini)
- Parent dashboards with summaries
- Strong privacy safeguards (no raw images stored)



Project: FocusBoost
SID : 2433192

Student : Shadhuhan Jeyachandran
Supervisor : Mr. Vibhavi Artigala